

Estándares de Desarrollo de Software

Documento Versión: 1.2

Gestión de Seguridad Electrónica

Fecha generación: 29/11/2025

CONTROL DE VERSIÓN			
VERSIÓN	**FECHA DE ACTUALIZACIÓN**	**CARGO**	**DESCRIPCIÓN**
1.0	18/06/2025	Arquitecto de Soluciones	Emisión del Documento
1.1	19/11/2025	Documentador Técnico	Ajustes de forma, ampliación del estándar de Logs, criptografía
1.2	29/11/2025	Control de Arquitectura	Correcciones generales al documento según observaciones.

Tabla de contenido

1 Introducción 8
1.1 Propósito 8
1.2 Alcance 9
1.3 Glosario, definiciones, Acrónimos y abreviaciones 10
2 Estándar de Logs 11
2.1 Cumplimiento Normativo 12
2.2 Importancia específica para RENIEC 13
2.3 Razones técnicas 13
2.4 Formatos de Logging 14
2.4.1 Syslog RFC 5424(Formato Moderno) 14
2.4.2 Structured Logging(JSON) 15
2.4.3 JSON Logging 17
2.4.4 Structured Logging (Logfmt) 20
2.4.5 OpenTelemetry Logs 23
2.5 Logs de Auditoría 27
2.5.1 Mecanismos de integridad, cifrado y retención 27
2.5.2 Formatos recomendados 27
2.6 Logs de Trazabilidad 28
2.6.1 Control de flujo y aseguramiento de datos críticos 28
2.6.2 Formatos recomendados 28
2.7 Logs de Observabilidad 28
2.7.1 Estándares y frameworks de observabilidad adoptados globalmente 29
2.7.2 Formatos recomendados 29
2.8 Consideraciones sobre Bases de Datos No Relacionales y Backup de Logs 29
2.9 Desafíos de Logging para microservicios 31
2.10 Retención de Logs por nivel de Criticidad 31
2.10.1 Tiempos mínimos de retención según nivel de criticidad 32
2.11 Finalidad de los Logs 33
3 Estándar de Timestamp y Zonas Horarias 33
3.1 Principios Generales 33
3.2 Zona Horaria Institucional (UTC) 34
3.3 Formato de Timestamps (ISO 8601) 34
3.3.1 Especificación técnica del formato 34
3.3.2 Reglas de formato obligatorias 35
3.3.3 Razones institucionales para adoptar ISO 8601 35

3.3.4 Consideraciones para sistemas heredados 35
3.4 Sincronización del Tiempo (NTP) 36
3.4.1 Requisitos técnicos de la sincronización NTP 36
3.5 Reglas de uso y Conversión del Tiempo 37
3.5.1 Reglas Institucionales Ampliadas 37
4 Estándar de Controles Criptográficos 38
4.1 Principios Generales 38
4.1.1 Practicas Criptográficas Recomendadas 39
4.2 Cifrado 39
4.2.1 Cifrado Simétrico 40
4.2.2 Cifrado Asimétrico 40
4.3 Funciones Hash 41
4.3.1 Gestión segura de contraseñas 42
4.4 Blockchain 42
4.4.1 Aplicación en RENIEC (Implícita) 42
4.5 Enmascaramiento de los Datos 43
4.6 Consideraciones adicionales 43
5 Estándar de Tratamiento de Datos Confidenciales 43
5.1 Principios Generales 44
5.2 Importancia específica para RENIEC 44
5.3 Aspectos clave de la ISO 27001 45
5.4 Protección de Datos Personales, Ley 29733 45
6 Estándar de Desarrollo Seguro 46
6.1 Principios de Desarrollo Seguro 46
6.2 Requisitos de Desarrollo Seguro 47
6.2.1 Validación y sanitización de entradas 47
6.2.2 Registro y monitoreo 48
6.2.3 Manejo seguro de errores 48
6.2.4 Configuración segura por defecto 49
6.3 Principios de Seguridad en el Ciclo de Vida del SIIRC 49
6.3.1 Seguridad por diseño 49
6.3.2 Principio de menor privilegio 50
6.3.3 Seguridad en capas (defensa en profundidad) 50
6.4 Controles Organizacionales Complementarios 50
6.5 Puntos Claves para el Desarrollo 51
6.6 DevSecOps + Seguridad (DevSecOps) 51
6.6.1 Alcance del Estándar DevSecOps 52
6.6.2 Componentes de Apoyo del Ecosistema DevSecOps 52

6.6.3 El Pipeline DevSecOps: Fases, Prácticas y Controles 53
6.6.4 Estrategias de Despliegue Seguro: Despliegues Canary 55
6.6.5 Versionado y CI/CD - Estrategia Integral 56
6.7 Estándares de pseudocódigo 57
7 Estándar de Autenticación y Autorización 58
7.1 Autenticación y Autorización 58
7.2 OpenID Connect 58
7.2.1 Diagrama de Flujo de autenticación con OIDC 59
7.2.2 Ventajas de OpenID Connect para SIIRC 59
7.2.3 Elementos Principales del protocolo OIDC 60
7.3 OAuth 2.0 60
7.3.1 Flujo Client Credentials (Servicios internos de SIIRC) 61
7.3.2 Parámetros y cabeceras utilizados en OAuth 2.0 61
7.4 EIDas 62
7.5 Identidad Autosoberana 63
7.5.1 Aplicación al SIIRC de RENIEC 63
8 Estándar de REST Services 64
8.1 Cumplimiento Normativo y Marco Legal Peruano 64
8.2 Justificación Estratégica 65
8.3 Métodos HTTP - Especificación Completa 65
8.3.1 Implementación Técnica 65
8.3.2 Manejo de respuestas y Errores SIIRC 66
8.3.3 Estructura Gestión de Errores 67
8.4 Patrones de Diseño REST para Microservicios 68
8.4.1 Patrón API Gateway - Justificación Arquitectónica 68
8.4.2 Patrón Saga para Transacciones Distribuidas 69
8.5 Principios REST Fundamentales 70
8.5.1 Stateless 70
8.5.2 Uniform Interface 70
8.5.3 Resource-Based 70
8.5.4 Ventajas Arquitectónicas 71
8.6 Nombrado de microservicios 71
8.6.1 Estructura del Nombramiento 71
8.6.2 Componentes del Nombre 71
8.7 Versionado de APIs - Estrategia de Evolución Continua 72
8.7.1 Estrategia de Versionado por URL Path 72
8.7.2 Política de Deprecación Empresarial 72
8.8 Métricas de Éxito y KPIs Ejecutivos 73

9 Estándar de base de datos 73
9.1 Propósito de un buen diseño de la base de datos 73
9.2 Tipo de base de datos a diseñar 73
9.3 Convención de nombres 75
9.4 Diccionario de datos 76
9.5 Criterios de claves primarias 76
9.6 Tipos de datos para cada atributo. 77
9.7 Tablas de consulta 77
9.8 Normalizar/desnormalizar según el tipo de Base de Datos 77
9.9 Modelos físicos por partes 78
9.10 Vistas e índices 78
10 Estándar de pruebas 78
10.1 Alcance 78
10.2 Métricas de pruebas 80
10.3 Criterios de Aceptación de las pruebas 80
10.4 Propósito del Proceso de Pruebas de Software 80
10.4.1 Estándares de Pruebas 80
10.4.2 Nivel de Pruebas de Proceso 81
10.4.3 Nivel de Pruebas de Bloque 81
10.4.4 Nivel de Pruebas de Sistema 82
10.5 Metodologías y marcos de referencia 83
10.5.1 Metodología BDD 83
10.5.2 Metodología ATDD 83
10.5.3 Marco de referencia de pruebas ISTQB 83
10.5.4 Software Engineering Body of Knowledge (SWEBOK) 83
11 Estándar de Implementación de Inteligencia Artificial (IA) 84
11.1 Principios generales 84
11.2 Razones técnicas 85
11.3 Aprendizaje supervisado 86
11.4 Aprendizaje no supervisado 86
11.5 Aprendizaje por refuerzo 86
11.6 Visión por computadora 87
11.7 Vectorización de archivos 87
11.8 Aseguramiento de calidad (Q/A) con LLM 88
11.9 Gobernanza y cumplimiento 89
12 Revisión y Actualización 89
12.1 Frecuencia de Revisión 90
12.2 Responsables 90

12.3 Control de Cambios 90
13 Referencias Bibliográficas y Normativas 90
13.1 Libros y Publicaciones Académicas 90
13.2 Sitios Web y Documentos Técnicos 91
13.3 Normativas y Legislación 91
13.4 Conferencias y Opiniones Expertas 91
13.5 Sitios Web Institucionales y Normativos 92
13.6 Normas ISO 92
13.7 OWASP (Open Web Application Security Project) 92
13.8 NIST (National Institute of Standards and Technology) 92
13.9 Políticas Internas 92

1 Introducción

La transformación digital que atraviesa el Estado peruano exige el desarrollo de plataformas tecnológicas que sean resilientes, interoperables y altamente seguras. En este contexto, el Sistema Integral de Identificación, Registro y Certificación, conocido como SIIRC, constituye el eje central de los procesos digitales que lidera la Registraduría Nacional de Identificación y Estado Civil (RENIEC). Este sistema agrupa funcionalidades críticas como el Registro Civil, la Identificación, los Servicios Electorales, la Certificación Digital, los Servicios de Información y la gestión del Vínculo de Parentesco, convirtiéndose en un componente esencial de la infraestructura estatal orientada a la gestión de la identidad y la ciudadanía.

El desarrollo e implementación de una plataforma de esta magnitud requiere contar con una base normativa y técnica sólida, que permita garantizar la coherencia, la seguridad y la calidad en cada una de sus fases. Este documento tiene como propósito establecer los lineamientos técnicos obligatorios que deben observarse en el ciclo de vida completo de los componentes del SIIRC, desde el diseño y desarrollo, hasta la integración, operación y mantenimiento. Estos estándares cubren temas fundamentales como los mecanismos de autenticación y autorización, el diseño e implementación de servicios REST, el desarrollo seguro, la trazabilidad mediante registros de eventos, los controles criptográficos aplicables, el tratamiento de datos personales y el cumplimiento normativo tanto a nivel nacional como internacional.

Además, este conjunto de estándares está alineado con marcos de referencia globalmente reconocidos, tales como la familia de normas ISO/IEC 27000, los lineamientos del NIST, y las buenas prácticas promovidas por OWASP, permitiendo así que el SIIRC se mantenga a la vanguardia en materia de gobernanza tecnológica, protección de datos y confianza digital. Al adoptar este estándar, RENIEC busca consolidar una plataforma que no solo cumpla con los requisitos funcionales de sus procesos misionales, sino que también ofrezca garantías técnicas y jurídicas en cuanto al uso responsable y seguro de la información ciudadana.

1.1 Propósito

El propósito de este documento es definir un marco normativo y técnico que oriente el desarrollo, la integración y la operación segura y eficiente de la plataforma SIIRC – Sistema Integral de Identificación, Registro y Certificación, liderada por RENIEC. Esta guía busca garantizar que todas las soluciones tecnológicas desarrolladas para SIIRC cumplan con criterios consistentes de seguridad, interoperabilidad, trazabilidad, calidad del software y protección de datos personales.

Este estándar está diseñado para cubrir todas las dimensiones técnicas relevantes en el desarrollo de soluciones digitales dentro de la plataforma SIIRC. En primer lugar, establece las bases para una autenticación y autorización robusta, a través de estándares como OAuth 2.0 y OpenID Connect, aplicando controles específicos tanto para el acceso de usuarios como para la comunicación entre sistemas y microservicios. En ese sentido, define claramente las configuraciones seguras para mecanismos de autenticación y políticas de control de acceso según roles o atributos.

En segundo lugar, proporciona una guía estructurada para el diseño e implementación de servicios REST, promoviendo buenas prácticas arquitectónicas, estandarización de estructuras de URI, uso adecuado de métodos HTTP, versionamiento de APIs y mecanismos de seguridad como autenticación por token, validación de entradas y cifrado en tránsito. También establece pautas para la documentación técnica de APIs, promoviendo la utilización de herramientas como OpenAPI/Swagger.

El documento también incorpora principios de desarrollo seguro basados en normativas como OWASP y NIST, abordando temas clave como el control de acceso al código fuente, la validación de datos de entrada, el manejo seguro de errores, la gestión de sesiones, las pruebas de seguridad y el uso de herramientas automatizadas para detección temprana de vulnerabilidades. Estas prácticas se deben aplicar de forma transversal durante todo el ciclo de vida del desarrollo de software.

Asimismo, se incluyen lineamientos para asegurar una adecuada trazabilidad y gestión de logs, estableciendo requerimientos técnicos sobre qué eventos deben registrarse, cómo deben almacenarse, los tiempos mínimos de retención, y los formatos y herramientas compatibles para su recolección y visualización. Esto garantiza una auditoría técnica confiable, especialmente para entornos críticos y sujetos a normativas regulatorias.

Otro componente esencial del estándar es la definición de los controles criptográficos que deben ser aplicados para proteger los datos en tránsito, en reposo y en uso. Se detallan los algoritmos aceptados para cifrado simétrico (AES-256), cifrado asimétrico (RSA 2048), funciones hash (SHA-512), técnicas de enmascaramiento de datos irreversibles y algoritmos de sustitución para visibilidad controlada de información sensible. Estos mecanismos criptográficos están alineados con recomendaciones internacionales como ISO/IEC 27001, NIST SP 800-57 y OWASP.

También se abordan las medidas para el tratamiento responsable de datos personales, mediante la clasificación de la información, la aplicación de principios de privacidad, técnicas de anonimización y minimización de datos, así como la obligatoriedad de realizar Evaluaciones de Impacto a la Privacidad (PIA) para nuevos sistemas o integraciones.

Por último, este documento establece criterios de cumplimiento normativo, identificando las leyes nacionales que regulan el manejo de datos y servicios digitales, así como los estándares internacionales de referencia que deben aplicarse en el diseño e implementación de los sistemas del SIIRC. También define los requisitos de auditoría, trazabilidad y conformidad técnica que deberán ser verificados de forma periódica.

En conjunto, este estándar tiene como finalidad consolidar una plataforma robusta, segura, moderna y ética, que garantice la confianza de los ciudadanos y la integridad de los procesos misionales que gestiona RENIEC.

1.2 Alcance

Este documento aplica a todos los elementos tecnológicos que componen o interactúan con la plataforma SIIRC, sin importar su origen, proveedor o tecnología subyacente. Se incluyen en su ámbito los módulos de autenticación y autorización de usuarios y servicios, las interfaces de comunicación basadas en servicios REST, los mecanismos de validación y protección integrados en las prácticas de desarrollo seguro, así como los procesos técnicos de generación, almacenamiento y análisis de trazabilidad a través de registros de eventos o logs.

Asimismo, el estándar comprende el uso de algoritmos y configuraciones criptográficas para el cifrado de datos en tránsito y en reposo, la aplicación de funciones hash para validación e integridad, el uso de técnicas de enmascaramiento y sustitución para la protección visual o lógica de datos confidenciales, y todos los mecanismos orientados a resguardar la confidencialidad, integridad y disponibilidad de la información gestionada por la plataforma. En paralelo, se incluyen las directrices relativas al tratamiento técnico y normativo de los datos personales, desde su clasificación y minimización hasta la evaluación de impacto a la privacidad, en cumplimiento de las disposiciones legales vigentes.

El estándar también cubre los lineamientos de cumplimiento normativo a nivel nacional e internacional, estableciendo los marcos de referencia que deben observarse para garantizar la conformidad legal, técnica y de auditoría de todos los desarrollos tecnológicos vinculados al SIIRC. El documento resulta obligatorio para todos los actores técnicos involucrados, incluyendo equipos internos de RENIEC, contratistas, integradores, proveedores de servicios, responsables de producto y auditores.

Quedan fuera del alcance de este documento aquellos sistemas que no interactúan directa ni indirectamente con SIIRC, los entornos físicos de infraestructura cuya gestión esté regulada por normativas propias de TI, y los procesos administrativos que no involucren tratamiento de información digital en los términos establecidos por este estándar. La aplicación de estas directrices es condición indispensable para asegurar la calidad técnica, la sostenibilidad operativa y la seguridad jurídica de la plataforma tecnológica que soporta las funciones misionales de RENIEC.

Los estándares descritos en este documento son agnósticos a la tecnología. Su propósito es establecer lineamientos conceptuales, metodológicos y de calidad que puedan implementarse en cualquier stack tecnológico, garantizando la interoperabilidad y consistencia del desarrollo.

1.3 Glosario, definiciones, Acrónimos y abreviaciones

Tabla 1. Definiciones y acrónimos

Término / Acrónimo	Definición
2FA / MFA	Autenticación de dos o más factores (contraseña + token, biometría, etc.) para aumentar la seguridad.
ABAC	Attribute-Based Access Control: acceso basado en atributos (cargo, área, ubicación, etc.).
Access Token	Token de acceso utilizado para autorizar peticiones a recursos protegidos.
API REST	Interfaz de programación basada en operaciones HTTP (GET, POST, PUT), orientada a recursos.
CSRF / XSS / SQLi	Tipos de vulnerabilidades comunes: falsificación de petición cruzada, scripts maliciosos, inyecciones SQL.
DevSecOps	Práctica que integra seguridad en los flujos de integración y entrega continua desde etapas tempranas.
DREAD	Modelo de evaluación de riesgos basado en: Daño, Reproducibilidad, Explotabilidad, Afectación a usuarios y Detección.
HSM (Hardware Security Module)	Dispositivo físico especializado en generar, almacenar y proteger claves criptográficas.
ID Token	Token emitido por el IdP que representa la identidad del usuario autenticado.
ISO/IEC 27001	Norma internacional para la gestión de la seguridad de la información.
ISO/IEC 27034	Norma que especifica directrices para la seguridad en el ciclo de vida del software.
ISO/IEC 27701	Extensión de la ISO 27001 enfocada en la gestión de la privacidad y protección de datos personales.
JWT (JSON Web Token)	Estructura segura en formato JSON que contiene datos firmados digitalmente, comúnmente usada para tokens.
NIST SP 800-218	Guía de desarrollo seguro de software publicada por el Instituto Nacional de Estándares y Tecnología de EE. UU.
OAuth 2.0	Protocolo de autorización (RFC 6749) que permite acceso delegado a recursos sin compartir credenciales.

OpenID Connect (OIDC)	Protocolo de autenticación federada construido sobre OAuth 2.0. Proporciona id_token para validar la identidad del **usuario**.
OpenTelemetry	Marco abierto para observabilidad distribuida (logs, métricas y trazabilidad en sistemas complejos).
OWASP	Open Web Application Security Project, organización que promueve buenas prácticas de seguridad web.
OWASP Top 10	Lista de las principales vulnerabilidades comunes en aplicaciones web.
PIA (Privacy Impact Assessment)	Evaluación de impacto en la privacidad que debe realizarse antes del tratamiento de datos sensibles.
RBAC	Role-Based Access Control: control de acceso basado en roles asignados a los usuarios.
Refresh Token	Token que permite renovar un Access Token sin requerir reautenticación.
RENIEC	Registro Nacional de Identificación y Estado Civil, entidad responsable de la identificación de los ciudadanos peruanos.
SIIRC	Sistema Integral de Identificación, Registro y Certificación de RENIEC, que articula los procesos misionales clave.
STRIDE	Modelo de amenazas que clasifica riesgos en: Suplantación, Falsificación, Repudio, Divulgación, Denegación y Elevación de privilegios.
Syslog / RFC 5424	Estándar para estructuración de logs de eventos del sistema, utilizado en auditoría técnica.
TLS (Transport Layer Security)	Protocolo criptográfico que asegura la confidencialidad e integridad de las comunicaciones.

2 Estándar de Logs

Los registros de logs digitales automáticos permiten documentar las acciones de transacción y eventos que se realizan en los sistemas informáticos. Funcionan como un libro de actas digital que registra lo que sucede en el sistema, incluyendo quién accedió, qué acciones realizó, cuándo y desde dónde las ejecutó, así como el resultado de dichas operaciones.

Considerando la criticidad de los servicios de identificación y registro civil que administra el RENIEC, la implementación de un sistema robusto de administración de logs resulta esencial. Este sistema permitirá garantizar la observabilidad, la seguridad y el cumplimiento normativo de los procesos, asegurando la trazabilidad de eventos dentro de los sistemas informáticos institucionales.

La información administrada por el RENIEC posee características particulares que hacen imprescindible un sistema de registro confiable y centralizado, ya que se trata de datos:

- **Sensibles**, al incluir información personal, biométrica y documentos de identidad de más de 33 millones de ciudadanos.
- **Legalmente protegidos**, por encontrarse bajo el marco de la Ley de Protección de Datos Personales y normativas internacionales.
- **Operacionalmente críticos**, debido a que los servicios soportados por RENIEC garantizan derechos fundamentales como la identificación, el voto y el acceso a servicios públicos.

En este contexto, la implementación de un sistema de logging centralizado constituye un pilar técnico para el SIIRC del RENIEC, estableciendo los lineamientos necesarios para la gestión integral de eventos y transacciones en sus plataformas tecnológicas.

Centralización de logs:

El sistema debe contar con una solución de centralización, agregación y visualización de registros de eventos (logs), que permita recolectar, procesar, almacenar y consultar la información generada por los distintos componentes de la infraestructura y las aplicaciones.

Esta solución debe contemplar capacidades para:

- **Ingesta y normalización de logs** provenientes de diversas fuentes (aplicaciones, bases de datos, servicios, sistemas operativos, etc.).
- **Almacenamiento indexado y búsqueda eficiente** para facilitar el análisis y trazabilidad.
- **Visualización y monitoreo en tiempo real** de eventos relevantes para la operación y la seguridad.

2.1 Cumplimiento Normativo

El establecimiento de un estándar institucional de logs responde a la necesidad de garantizar el cumplimiento de marcos normativos internacionales, nacionales y sectoriales que regulan la gestión de información, la seguridad de los sistemas y la trazabilidad de las operaciones realizadas en el entorno digital administrado por el RENIEC.

En el ámbito internacional, la norma ISO/IEC 27001, bajo la cual el RENIEC mantiene certificación vigente en su versión ISO 27001:2013, constituye el principal referente para la gestión de la seguridad de la información. Esta norma establece los requisitos para preservar la confidencialidad, integridad y disponibilidad de los activos de información mediante un enfoque basado en riesgos. En su actualización ISO/IEC 27001:2022, el control 8.15 – “Logging” refuerza la necesidad de implementar mecanismos de registro que aseguren la integridad, disponibilidad y confidencialidad de los eventos generados por los sistemas de información, incluyendo la identificación de usuarios, fechas, dispositivos involucrados, tipo de acción ejecutada y resultado obtenido, garantizando su protección frente a alteraciones o eliminaciones no autorizadas.

Complementariamente, la guía NIST SP 800-92 del Instituto Nacional de Estándares y Tecnología de los Estados Unidos (NIST) constituye una referencia técnica reconocida por la industria para la gestión de logs. Este documento recomienda políticas de retención, correlación, monitoreo y respuesta ante incidentes, asegurando que los registros sean útiles tanto para la operación como para la detección y análisis de eventos de seguridad. De manera similar, el EBA Standard on Logging and Monitoring (Autoridad Bancaria Europea) exige la trazabilidad completa de las actividades críticas y la capacidad de reconstruir acciones a partir de registros verificables y auditables, alineándose con las mejores prácticas de cumplimiento y auditoría.

A nivel nacional, el Marco de Seguridad Digital del Estado Peruano, establecido mediante la Directiva de Seguridad Digital del Estado, define los principios, políticas, normas y roles que las entidades públicas deben adoptar para preservar la confidencialidad, integridad y disponibilidad de la información. Asimismo, la Ley N.º 29733 – Ley de Protección de Datos Personales establece las obligaciones de las instituciones públicas y privadas respecto al tratamiento de datos personales, asegurando su manejo conforme a los principios de responsabilidad, proporcionalidad y transparencia.

En este contexto, la adopción de un estándar unificado de logs dentro del RENIEC no solo responde a exigencias normativas de seguridad y protección de datos, sino que fortalece la trazabilidad institucional, la capacidad de auditoría y la evidencia digital ante posibles incidentes, disputas o procesos de fiscalización. Este marco contribuye a consolidar la gestión segura de la información y a demostrar cumplimiento frente a auditorías internas, externas y organismos de control.

2.2 Importancia específica para RENIEC

Para el RENIEC, los logs representan un mecanismo esencial de control, evidencia y legitimidad institucional. Su función trasciende lo técnico, ya que garantizan la trazabilidad completa de las operaciones, permitiendo determinar quién ejecutó una acción, cuándo y con qué resultado, lo cual es vital para proteger la integridad y autenticidad de la información.

La entidad gestiona datos personales y biométricos sensibles del Sistema Nacional de Identificación, por lo que mantener un registro confiable de eventos es indispensable para asegurar la confianza ciudadana y la seguridad del ecosistema digital del Estado Peruano. En procesos misionales como la emisión de documentos de identidad, la verificación biométrica o la interoperabilidad con otras instituciones, los logs constituyen la única evidencia técnica verificable de las transacciones realizadas.

Además, su implementación permite cumplir con los marcos legales y normativos nacionales e internacionales, entre ellos la Ley N.º 29733, la Directiva de Seguridad Digital del Estado, y la ISO/IEC 27001:2013, garantizando la confidencialidad, integridad y disponibilidad de la información. Este cumplimiento fortalece la rendición de cuentas institucional y reduce riesgos legales y reputacionales.

En el ámbito de la Ciberseguridad nacional, los logs también actúan como una fuente clave para la detección temprana de incidentes y la respuesta coordinada ante amenazas, asegurando la resiliencia operativa de los servicios críticos de identificación ciudadana.

En síntesis, el sistema de logs tiene para el RENIEC una importancia estratégica, operativa y jurídica:

- Estratégica, porque sustenta su rol como garante de la identidad digital del país.
- Operativa, porque proporciona control y trazabilidad sobre los procesos críticos.
- Jurídica, porque genera la evidencia probatoria y auditable necesaria para la defensa institucional.

La estandarización de logs es, por tanto, una condición indispensable para la transparencia, seguridad y legitimidad del servicio público de identificación en el Perú.

2.3 Razones técnicas

Desde una perspectiva técnica, la implementación de un estándar unificado de logs en el RENIEC responde a la necesidad de fortalecer la observabilidad, seguridad y eficiencia operativa de los sistemas institucionales. En entornos modernos distribuidos —basados en microservicios, APIs y nube privada, los registros deben ser estructurados, consistentes e interoperables para garantizar trazabilidad y respuesta efectiva ante incidentes.

Principales razones técnicas

- **Interoperabilidad y consistencia:** Los formatos estandarizados (como Syslog RFC 5424, JSON Logging u OpenTelemetry Logs) permiten la integración fluida entre diferentes plataformas y herramientas de monitoreo, asegurando una captura uniforme de eventos.
- **Observabilidad integral:** Los logs proporcionan visibilidad en tiempo real del comportamiento de los sistemas, evitando que operen como “cajas negras” y facilitando el seguimiento preciso de fallos o anomalías.
- **Detección y prevención de amenazas:** Permiten la identificación automática de actividades sospechosas, como intentos de acceso no autorizados, ataques por fuerza bruta o comportamientos atípicos de red.
- **Diagnóstico y resolución de incidentes:** Facilitan la reconstrucción detallada de eventos, acelerando la detección de la causa raíz y reduciendo los tiempos de recuperación (MTTR).

- **Optimización del rendimiento:** Proveen información real sobre el uso del sistema, ayudando a identificar cuellos de botella y a planificar la capacidad de infraestructura de manera eficiente.
- **Cumplimiento normativo y trazabilidad:** Garantizan que cada evento sea verificable y auditable, reforzando la transparencia y el cumplimiento de las normas ISO/IEC 27001:2013, la Ley N.º 29733 y la Directiva de Seguridad Digital del Estado.

En conjunto, estas razones consolidan al estándar de logs como un requisito técnico fundamental para asegurar la confiabilidad, seguridad y eficiencia de los servicios digitales que RENIEC ofrece a la ciudadanía.

2.4 Formatos de Logging

2.4.1 Syslog RFC 5424(Formato Moderno)

Es un estándar internacional establecido por el Internet Engineering Task Force (IETF) que define cómo deben estructurarse y transmitirse los mensajes de logs en sistemas informáticos modernos. Es la versión más actual y avanzada del protocolo Syslog

Cada mensaje de log bajo RFC 5424 tiene una estructura estandarizada y precisa:

<PRIORIDAD>VERSIÓN TIMESTAMP HOSTNAME APLICACIÓN PROCESO MENSAJE-ID [DATOS-ESTRUCTURADOS] MENSAJE

2.4.1.1 Estructura técnica

- **PRIORITY:**

  Combina la "facilidad" (qué sistema generó el log) con la "severidad" (qué tan crítico es) por ejemplo:

  Un error de seguridad tiene mayor prioridad que una operación rutinaria. Esto Permite filtrar automáticamente qué logs requieren atención inmediata.

- **VERSION**

  Indica que se está usando el formato RFC 5424 (siempre es "1") esto garantiza compatibilidad y procesamiento correcto por diferentes sistemas.

- **TIMESTAMP**

  Es la fecha y hora exacta con precisión de microsegundos y zona horaria con Formato:

  2025-06-10T14:30:25.123456-05:00.

  Esto permite la sincronización precisa entre múltiples sistemas y ubicaciones geográficas

- **HOSTNAME**

  Es la identificación del servidor o equipo que generó el log por ejemplo:

  reniec-servidor-01.gob.pe

  Esto permite la localización exacta del origen de cada evento

- **APP-NAME**

  Es el nombre del programa o servicio que generó el evento, por ejemplo:

  sistema-consulta-dni, base-datos-principal.

  Esto permite la identificación rápida de qué componente específico generó el log

- **PROCID**

Es el identificador único del proceso en ejecución que permite rastrear eventos específicos dentro de una aplicación

- **MSGID**

  Es el identificador del tipo de evento, por ejemplo:

  LOGIN_SUCCESS, DATA_ACCESS_DENIED, SYSTEM_BACKUP_COMPLETE.

  Esto permite la categorización automática y búsqueda eficiente de tipos de eventos

- **ESTRUCTURED DATA**

  Es información adicional organizada en pares clave-valor, por ejemplo:

  [user@12345 id="juan.perez" ip="192.168.1.100" action="consulta_dni"].

  Esto permite tener la información contextual rica para análisis automatizado

MESSAGE

Es la descripción legible del evento en lenguaje natural, por ejemplo:

"Usuario consultó exitosamente DNI número 12345678"

Ejemplo práctico:

<134>1 2025-06-10T14:30:25.123456-05:00 reniec-servidor-01.gob.pe sistema-consulta-dni 8740 CONSULTA_EXITOSA [user@12345 id="funcionario.lima" dni_consultado="12345678" oficina="lima_centro" tiempo_respuesta="0.245s"] Usuario funcionario.lima consultó exitosamente DNI 12345678 desde oficina Lima Centro

Interpretación de este ejemplo:

- **Priority 134**: Evento informativo del sistema de aplicaciones
- **Timestamp**: 10 de junio 2025, 2:30:25 PM con microsegundos, zona horaria Perú
- **Server**: reniec-servidor-01.gob.pe
- **App-Name**: sistema-consulta-dni
- **Event-Type:** CONSULTA_EXITOSA
- **Estructured-Data:** Usuario, DNI consultado, oficina, tiempo de respuesta
- **Message**: Descripción clara del evento

2.4.2 Structured Logging(JSON)

Es una metodología moderna de registro de eventos que utiliza el formato JSON (JavaScript Object Notation) para estructurar la información de logs de manera que sea tanto legible para humanos como procesable por máquinas. A diferencia de los logs tradicionales de texto plano, cada entrada de log es un objeto JSON válido que contiene campos estructurados con datos específicos del evento.

Este enfoque permite análisis automatizado avanzado, búsquedas eficientes, correlación de eventos e integración directa con herramientas de monitoreo y análisis de datos modernos.

2.4.2.1 Estructura técnica

TIMESTAMP
Fecha y hora del evento en formato ISO 8601 con precisión de milisegundos y zona horaria. Ejemplo:

- "2025-06-10T14:30:25.123-05:00".

Esto garantiza ordenamiento cronológico preciso y sincronización entre sistemas distribuidos.

LEVEL
Indica la criticidad del evento usando niveles estándar como: "DEBUG", "INFO", "WARN", "ERROR", "FATAL". Esto permite filtrado automático por criticidad y alertas basadas en severidad.

LOGGER
Nombre del componente, clase o módulo que generó el log, por ejemplo:

- "reniec.consulta.dni.service", "reniec.auth.security".

Esto facilita la trazabilidad del origen exacto del evento dentro del código.

MESSAGE
Descripción legible del evento en lenguaje natural que resume lo ocurrido. Ejemplo:

- "Consulta de DNI procesada exitosamente".

Esto proporciona contexto inmediato para operadores humanos.

HOSTNAME/SERVICE
Nombre del servidor, contenedor o servicio que ejecuta la aplicación. Ejemplo:

- "reniec-api-prod-01", "/api/v1/registrocivil/MsBkDatosSolicitudes/{solicitudId}".

Esto permite identificar el origen físico o lógico del evento.

THREAD/PROCESS
ID del hilo de ejecución o proceso que generó el evento. Ejemplo:

- "http-nio-8080-exec-5".

Esto facilita el seguimiento de operaciones concurrentes y debugging.

REQUEST_ID/TRACE_ID
UUID único que correlaciona múltiples logs de una misma operación o transacción. Ejemplo:

- "550e8400-e29b-41d4-a716-446655440000".

Esto permite seguir el flujo completo de una operación a través de múltiples servicios.

USER_CONTEXT (Contexto del Usuario)
Información sobre el usuario que ejecutó la acción, incluyendo ID, roles, ubicación. Esto proporciona contexto de seguridad y auditoría.

TECHNICAL_DETAILS
Información específica del dominio como tiempos de respuesta, códigos de error, parámetros de entrada, resultados de operaciones.

METADATA
Información contextual adicional como versión de la aplicación, ambiente de ejecución, configuraciones relevantes.

Ejemplo técnico:

```
{
  "timestamp": "2025-06-10T14:30:25.123-05:00",
  "level": "INFO",
  "logger": "reniec.consulta.dni.service.ConsultaService",
  "message": "Consulta de DNI procesada exitosamente",
  "hostname": "reniec-api-prod-01.gob.pe",
  "service": "sistema-consulta-dni",
```

```
  "thread": "http-nio-8080-exec-12",
  "request_id": "550e8400-e29b-41d4-a716-446655440000",
  "user_context": {
    "user_id": "funcionario.lima",
    "rol": "operador_consultas",
    "oficina": "lima_centro",
    "ip_address": "192.168.100.45"
  },
  "technical_details": {
    "dni_consultado": "12345678",
    "tiempo_respuesta_ms": 245,
    "base_datos": "reniec_principal",
    "metodo_http": "POST",
    "endpoint": "/api/v1/registrocivil/MsBkDatosSolicitudes/{solicitudId}"
  },
  "metadata": {
    "app_version": "2.3.1",
    "environment": "production",
    "datacenter": "lima_primary"
  }
}
```


Interpretación de este ejemplo:

- **Event**: Consulta exitosa de DNI realizada el 10 de junio 2025 a las 2:30:25 PM
- **Source**: Servicio ConsultaService ejecutándose en servidor reniec-api-prod-01.gob.pe
- **User**: funcionario de la oficina Lima Centro con rol de operador
- **Operation**: Consulta POST al endpoint /api/v1/registrocivil/MsBkDatosSolicitudes para DNI 12345678
- **Performance**: Respuesta procesada en 245 milisegundos
- **Traceability**: Request ID único para correlacionar con otros logs relacionados
- **Technical Context**: Aplicación versión 2.3.1 en ambiente de producción

2.4.3 JSON Logging

JSON Logging es un formato de salida específico para sistemas de registro que utiliza exclusivamente la notación JSON (JavaScript Object Notation) como mecanismo de serialización y almacenamiento de eventos de log. A diferencia del concepto más amplio de Structured Logging, JSON Logging se enfoca específicamente en la implementación técnica del formato JSON como contenedor de datos de logging, independientemente de la metodología de estructuración subyacente.

Este enfoque prioriza la compatibilidad nativa con ecosistemas de herramientas que procesan JSON, garantizando parseo directo sin transformaciones intermedias y optimizando el rendimiento de ingestión en pipelines de datos modernos.

Diferencias Conceptuales con Structured Logging (JSON)

Structured Logging (JSON)

- Enfoque: Metodología de organización y estructuración de datos de logging
- Alcance: Estrategia completa de logging que incluye formato, esquemas y estándares
- Flexibilidad: Puede usar múltiples formatos de salida (JSON, XML, YAML, etc.)
- Objetivo: Crear consistencia y estructura en los datos de log

JSON Logging

- Enfoque: Formato de salida y serialización específica
- Alcance: Implementación técnica del formato JSON como contenedor
- Flexibilidad: Exclusivamente JSON como formato de salida
- Objetivo: Maximizar compatibilidad con herramientas que procesan JSON nativamente

Características Técnicas

FORMATO NATIVO
JSON como formato único de salida sin conversiones intermedias. Cada evento se serializa directamente como objeto JSON válido, garantizando compatibilidad inmediata con parsers JSON estándar y eliminando sobrecargas de transformación.

ESQUEMA FLEXIBLE
Estructura de campos dinámica que permite adaptación a diferentes contextos sin requerir redefinición de esquemas rígidos. Los objetos JSON pueden contener campos opcionales y anidados según las necesidades específicas del evento.

COMPATIBILIDAD UNIVERSAL
Soporte nativo en prácticamente todos los lenguajes de programación modernos y herramientas de procesamiento de datos. JSON es el formato estándar para APIs REST, bases de datos NoSQL y sistemas de mensajería.

SERIALIZACIÓN EFICIENTE
Parseo y generación optimizada aprovechando librerías JSON maduras y altamente optimizadas disponibles en cada plataforma tecnológica.

2.4.3.1 Estructura técnica

OBJETO RAÍZ
Cada entrada de log es un objeto JSON válido autónomo que contiene toda la información del evento en una estructura de clave-valor.

CAMPOS PRIMITIVOS
Utilización de tipos de datos JSON nativos: string, number, boolean, null para representar información básica del evento.

OBJETOS ANIDADOS
Agrupación lógica de información relacionada mediante objetos JSON anidados, permitiendo jerarquías de datos sin limitaciones de profundidad.

ARRAYS DE DATOS
Representación de colecciones de información mediante arrays JSON para datos como stack traces, parámetros múltiples o listas de validaciones.

ESCAPE Y ENCODING
Manejo automático de caracteres especiales y encoding UTF-8 garantizando integridad de datos y compatibilidad internacional.

Ejemplo técnico

```
{
  "timestamp": "2025-06-10T14:30:25.123-05:00",
  "level": "ERROR",
  "source": {
    "application": "reniec-api",
    "module": "dni-validation",
    "function": "validateDniFormat",
    "line": 245
  },
  "event": {
    "type": "validation_error",
    "description": "Formato de DNI inválido detectado",
    "error_code": "DNI_INVALID_FORMAT"
  },
```

```
  "context": {
    "request": {
      "id": "req_550e8400e29b41d4",
      "method": "POST",
      "endpoint": "/api/v1/registrocivil/MsBkDatosSolicitudes",
      "user_agent": "ReniecClient/2.1.0"
    },
    "session": {
      "user_id": "func_12345",
      "role": "validator",
      "office_code": "LIM001"
    }
  },
  "payload": {
    "input_dni": "1234567A",
    "validation_rules": ["format", "checksum", "blacklist"],
    "failed_rules": ["format"]
  },
  "metrics": {
    "processing_time_ms": 12,
    "memory_usage_mb": 45.2,
    "cpu_usage_percent": 8.5
  },
  "trace": {
    "correlation_id": "trace_abc123def456",
    "span_id": "span_789xyz012",
    "parent_span": "span_456abc789"
  },
  "environment": {
    "hostname": "reniec-prod-03",
    "container_id": "docker_789abc123",
    "version": "2.3.1",
    "datacenter": "lima_primary"
  }
}
```


Interpretación del Ejemplo

- **Event**: Error de validación de formato de DNI detectado el 10 de junio 2025 a las 2:30:25 PM
- **Technical Origin**: Función validateDniFormat en línea 245 del módulo dni-validation
- **Operation Context**: Request POST con ID específico ejecutado por usuario validador
- **Input Data**: DNI "1234567A" que falló la regla de validación de formato
- **Performance Metrics**: Procesamiento en 12ms con uso de memoria de 45.2MB
- **Traceability:** IDs de correlación y span para seguimiento distribuido
- **Environment:** Contenedor Docker en servidor de producción lima_primary

2.4.3.2 Ventajas Técnicas

PROCESAMIENTO DIRECTO
Ingestión inmediata en herramientas de observabilidad sin necesidad de parsers customizados o transformaciones de formato.

COMPATIBILIDAD API
Integración directa con APIs REST y servicios web que consumen JSON nativamente, facilitando el envio automático de logs.

ESCALABILIDAD HORIZONTAL
Distribución eficiente en sistemas distribuidos aprovechando la naturaleza ligera y portable del formato JSON.

INDEXACIÓN AUTOMÁTICA
Creación automática de índices en bases de datos NoSQL basados en la estructura de campos JSON.

2.4.3.3 Limitaciones Técnicas

SOBRECARGA DE METADATOS
Mayor uso de espacio por la inclusión de nombres de campos en cada registro, comparado con formatos binarios o delimitados.

PARSING OBLIGATORIO
Requerimiento de parsing JSON para lectura humana directa, a diferencia de logs de texto plano inmediatamente legibles.

LIMITACIONES DE TIPOS
Restricción a tipos de datos JSON nativos, requiriendo serialización especial para tipos complejos como fechas o números decimales de alta precisión.

FRAGMENTACIÓN DE OBJETOS
Posible fragmentación en sistemas que no manejan objetos JSON grandes eficientemente, afectando el rendimiento en eventos con gran cantidad de datos.

2.4.3.4 Implementación Recomendada

CONFIGURACIÓN DE SALIDA
Configurar loggers para generar exclusivamente JSON sin formatos alternativos, garantizando consistencia en toda la aplicación.

VALIDACIÓN DE ESQUEMA
Implementar validación de esquema JSON opcional para garantizar consistencia estructural sin impactar rendimiento de producción.

COMPRESIÓN AUTOMÁTICA
Utilizar algún software para comprimir ya que se reduce la sobrecarga de almacenamiento y transmisión de archivos JSON de log.

ROTACIÓN INTELIGENTE
Implementar rotación de archivos basada en tamaño de objeto JSON para evitar archivos parcialmente parseables.

BUFFERING OPTIMIZADO
Configurar buffering de escritura optimizado para objetos JSON completos, evitando cortes en medio de objetos durante una alta concurrencia.

2.4.4 Structured Logging (Logfmt)

Structured Logging (Logfmt) es un formato de registro estructurado que utiliza la notación de pares clave-valor separados por espacios como mecanismo principal de serialización y almacenamiento de eventos de log. Este formato, originalmente desarrollado por Heroku y ampliamente adoptado en ecosistemas de sistemas distribuidos, prioriza la legibilidad humana directa mientras mantiene la capacidad de parsing automático eficiente.

A diferencia de otros formatos de logging estructurado, Logfmt se enfoca en el equilibrio entre la simplicidad de lectura para humanos y la eficiencia de procesamiento automatizado,

eliminando la necesidad de parsers complejos mientras conserva la estructura de datos necesaria para análisis avanzado.

Diferencias Conceptuales con Structured Logging (JSON)

Structured Logging (JSON)

- **Enfoque**: Formato de objeto estructurado con jerarquías anidadas
- **Alcance**: Soporte completo para tipos de datos complejos y estructuras profundas
- **Flexibilidad**: Objetos anidados, arrays y tipos de datos nativos JSON
- **Objetivo**: Máxima expresividad estructural y compatibilidad con APIs

Structured Logging (Logfmt)

- **Enfoque**: Formato plano de pares clave-valor con legibilidad directa
- **Alcance**: Estructura simple y eficiente para datos de logging lineales
- **Flexibilidad**: Pares clave-valor planos con encoding especial para valores complejos
- **Objetivo**: Optimizar legibilidad humana y eficiencia de parsing

2.4.4.1 Estructura Técnica

PARES CLAVE-VALOR
Cada campo se representa como clave=valor donde la clave es un identificador alfanumérico y el valor puede ser string, número o boolean.

SEPARACIÓN POR ESPACIOS
Los pares clave-valor se separan por espacios simples, permitiendo parsing eficiente mediante división de strings.

ESCAPE DE VALORES
Valores que contienen espacios se encierran entre comillas dobles. Caracteres especiales se escapan usando backslash según estándares de strings.

ORDEN LÓGICO
Campos se organizan por importancia y frecuencia de consulta, típicamente: timestamp, level, mensaje, contexto, métricas.

CODIFICACIÓN UTF-8
Soporte completo para caracteres internacionales mediante encoding UTF-8 estándar.

Ejemplo técnico

```
timestamp=2025-06-10T14:30:25.123-05:00 level=ERROR application=reniec-api
module=dni-validation function=validateDniFormat line=245 event_type=validation_error
description="Formato de DNI inválido detectado" error_code=DNI_INVALID_FORMAT
request_id=req_550e8400e29b41d4 method=POST
endpoint="/api/v1/registrocivil/MsBkDatosSolicitudes" user_agent="ReniecClient/2.1.0"
user_id=func_12345 role=validator office_code=LIM001 input_dni=1234567A
validation_rules="format,checksum,blacklist" failed_rules=format processing_time_ms=12
memory_usage_mb=45.2 cpu_usage_percent=8.5 correlation_id=trace_abc123def456
span_id=span_789xyz012 parent_span=span_456abc789 hostname=reniec-prod-03
container_id=docker_789abc123 version=2.3.1 datacenter=lima_primary
```

Interpretación del Ejemplo

- **Event**: Error de validación de formato de DNI detectado el 10 de junio de 2025 a las 2:30:25 PM
- **Technical Source**: Función validateDniFormat en línea 245 del módulo dni-validation
- **Operation Context:** Request POST con ID específico ejecutado por usuario validador
- **Input Data:** DNI "1234567A" que falló la regla de validación de formato
- **Performance Metrics:** Procesamiento en 12 ms con uso de memoria de 45.2 MB

- **Traceability:** IDs de correlación y span para seguimiento distribuido
- **Environment**: Contenedor Docker en servidor de producción lima_primary

2.4.4.2 Ventajas Técnicas

LEGIBILIDAD DIRECTA
Lectura inmediata sin herramientas adicionales. Los logs son comprensibles directamente en consola, archivos de texto o interfaces de terminal.

PARSING SIMPLE
Procesamiento eficiente mediante separación básica de strings. Implementación trivial en cualquier lenguaje de programación sin librerías especializadas.

COMPACIDAD
Menor sobrecarga de metadatos comparado con formatos de objeto. Eliminación de llaves, corchetes y sintaxis de estructura jerárquica.

STREAMING OPTIMIZADO
Procesamiento línea por línea ideal para streaming de logs en tiempo real y pipelines de datos de alta velocidad.

GREP COMPATIBLE
Búsqueda directa usando herramientas estándar de texto sin necesidad de parsers especializados.

2.4.4.3 Limitaciones Técnicas

ESTRUCTURA PLANA
Imposibilidad de representar jerarquías complejas u objetos anidados sin serialización adicional como strings escapados.

LIMITACIONES DE TIPOS
Restricción a tipos primitivos (string, number, boolean). Tipos complejos requieren serialización manual a string.

ORDEN FIJO
Dependencia del orden de campos para optimización de lectura. Cambios en estructura pueden afectar herramientas de procesamiento.

ESCAPE COMPLEJO
Manejo manual de caracteres especiales y comillas en valores que contienen espacios o caracteres de control.

BÚSQUEDA LIMITADA
Consultas complejas requieren parsing previo, a diferencia de formatos que soportan queries estructuradas nativas.

2.4.4.4 Ventajas de la Implementación

FORMATO LEGIBLE
Estructura de pares clave-valor separados por espacios que permite lectura directa sin herramientas de parsing. Cada evento se representa como una línea única con información estructurada inmediatamente comprensible.

PARSING EFICIENTE
Procesamiento lineal simple que requiere únicamente separación por espacios y división por símbolo de asignación. No requiere parsers complejos ni análisis sintáctico profundo.

COMPATIBILIDAD UNIVERSAL
Soporte nativo en herramientas de observabilidad modernas y sistemas de logging distribuido. Compatible con pipelines de procesamiento de texto estándar.

SERIALIZACIÓN OPTIMIZADA
Generación directa de strings sin overhead de serialización de objetos complejos. Ideal para sistemas de alto rendimiento donde la latencia de logging es crítica.

2.4.4.5 Implementación Recomendada

CONFIGURACIÓN DE FORMATO
Configurar loggers para generar exclusivamente formato logfmt con esquema de campos consistente en toda la aplicación.

ESTÁNDARES DE NOMENCLATURA
Establecer convenciones de nombres de campos usando snake_case y prefijos descriptivos para agrupación lógica.

VALIDACIÓN DE FORMATO
Implementar validación de formato logfmt para garantizar parseo correcto y detectar errores de serialización.

BUFFERING LINEAL
Configurar buffering de escritura optimizado para líneas completas, evitando cortes parciales durante escritura concurrente.

HERRAMIENTAS DE ANÁLISIS
Integrar con herramientas de observabilidad que soportan logfmt nativamente y sistemas de métricas distribuidas.

2.4.5 OpenTelemetry Logs

OpenTelemetry Logs es el componente de logging del framework de observabilidad OpenTelemetry que proporciona un modelo de datos estandarizado y APIs unificadas para la captura, correlación y exportación de logs en sistemas distribuidos. Como parte integral del ecosistema OpenTelemetry, permite la correlación automática entre logs, traces y métricas, estableciendo un modelo unificado de observabilidad para arquitecturas de microservicios complejas.

OpenTelemetry Logs se diseñó específicamente para abordar los desafíos de visibilidad en sistemas distribuidos modernos, proporcionando contexto automático de trazabilidad, instrumentación no intrusiva y compatibilidad con infraestructuras existentes de logging. Su enfoque se centra en la correlación automática de eventos a través de múltiples servicios mediante trace context propagation y la estandarización de metadatos críticos para análisis de observabilidad.

2.4.5.1 Características Técnicas

MODELO DE DATOS ESTANDARIZADO
Estructura unificada de Log Record que incluye metadatos esenciales como timestamp, severity level, body, attributes, trace context y resource information. Garantiza consistencia en la representación de eventos a través de diferentes servicios y tecnologías.

CORRELACIÓN AUTOMÁTICA
Propagación automática de trace context que permite correlación directa entre logs, spans y métricas sin instrumentación manual adicional. Los logs heredan automáticamente trace_id y span_id del contexto activo de ejecución.

INSTRUMENTACIÓN NO INTRUSIVA
Integración transparente con bibliotecas de logging existentes mediante bridges y autoinstrumentación. Permite adopción gradual sin modificaciones significativas en código de aplicación existente.

PROTOCOLO OTLP
Exportación eficiente mediante OpenTelemetry Protocol (OTLP) que soporta compresión, batching y retry logic automático. Optimizado para transmisión de alta frecuencia en entornos de producción.

ATRIBUCION DE RECURSOS
Asociación automática de metadatos de recurso (service.name, service.version, deployment.environment) que proporciona contexto operacional completo para cada evento de log.

2.4.5.2 Estructura Técnica

CAMPOS FUNDAMENTALES

- Timestamp: Tiempo exacto de generación del evento con precisión de nanosegundos y soporte para timezone.
- Severity Level: Nivel de severidad estandarizado (TRACE, DEBUG, INFO, WARN, ERROR, FATAL) con valores numéricos consistentes.
- Body: Contenido principal del mensaje de log que puede ser string estructurado u objeto complejo.

METADATOS DE CONTEXTO

- Trace Context: trace_id y span_id que permiten correlación directa con traces distribuidos.
- Attributes: Mapa clave-valor de metadatos específicos del evento con soporte para tipos de datos primitivos y arrays.
- Resource: Información del recurso que genera el log incluyendo servicio, versión, ambiente y metadatos de infraestructura.

INFORMACIÓN DE INSTRUMENTACIÓN

- Instrumentation Scope: Identificación de la biblioteca o componente que genera el log con versión y schema_url. Observed
- Timestamp: Timestamp de captura por el sistema de colección cuando difiere del timestamp original.
- Flags: Indicadores binarios para metadatos adicionales como sampling decisions o trace context validity.

Ejemplo técnico

```
{
  "timestamp": "2025-06-10T19:30:25.123456789Z",
  "observedTimestamp": "2025-06-10T19:30:25.125000000Z",
  "severityLevel": "ERROR",
  "severityText": "ERROR",
  "body": {
    "stringValue": "Validación de DNI falló: formato inválido detectado"
  },
  "attributes": {
    "application.component": "dni-validation-service",
    "application.function": "validateDniFormat",
    "application.version": "2.3.1",
```

```
    "error.code": "DNI_INVALID_FORMAT",
    "error.type": "ValidationError",
    "request.method": "POST",
    "request.endpoint": "/api/v1/registrocivil/MsBkDatosSolicitudes",
    "request.id": "req_550e8400e29b41d4",
    "user.id": "func_12345",
    "user.role": "validator",
    "reniec.office_code": "LIM001",
    "input.dni": "1234567A",
    "validation.rules": ["format", "checksum", "blacklist"],
    "validation.failed_rules": ["format"],
    "performance.processing_time_ms": 12,
    "performance.memory_usage_mb": 45.2,
    "performance.cpu_usage_percent": 8.5
  },
  "traceId": "abc123def4567890123456789012345678 90abcd",
  "spanId": "123456789abcdef0",
  "traceFlags": 1,
  "resource": {
    "attributes": {
      "service.name": "reniec-dni-validation-api",
      "service.version": "2.3.1",
      "service.namespace": "reniec.validation",
      "deployment.environment": "production",
      "k8s.cluster.name": "reniec-prod-cluster",
      "k8s.namespace.name": "validation-services",
      "k8s.pod.name": "dni-validation-pod-789abc",
      "k8s.container.name": "dni-validation-container",
      "cloud.provider": "aws",

"cloud.region": "sa-east-1",
      "cloud.availability_zone": "sa-east-1a",
      "host.name": "reniec-prod-worker-03",
      "process.pid": 1247
    }
  },
  "instrumentationScope": {
    "name": "com.reniec.validation.logging",
    "version": "1.0.0",
    "schemaUrl": "https://opentelemetry.io/schemas/1.24.0"
  }
}
```


Interpretación del Ejemplo

- **Main Event:** Error de validación de formato de DNI registrado el 10 de junio 2025 con correlación automática a trace distribuido.
- **Traceability Context:** Log correlacionado automáticamente con trace_id y span_id, permitiendo seguimiento completo de la request a través de múltiples servicios.
- **Operational Metadata:** Información completa del contexto de ejecución incluyendo usuario, oficina RENIEC, método de request y endpoint específico.
- **Validation Data:** DNI procesado y reglas de validación aplicadas con identificación específica de reglas fallidas.
- **Performance Metrics:** Métricas automáticas de procesamiento, memoria y CPU capturadas sin instrumentación manual adicional.
- **Infrastructure Context:** Resource attributes que proporcionan información completa del ambiente de ejecución incluyendo Kubernetes, proveedor de nube y host específico.
- **Instrumentation:** Scope information que identifica la biblioteca específica de logging y versión del esquema utilizado.

2.4.5.3 Ventajas Técnicas

OBSERVABILIDAD UNIFICADA
Correlación automática entre logs, traces y métricas que proporciona visibilidad completa de transacciones distribuidas sin configuración manual adicional.

ESTANDARIZACIÓN INDUSTRIAL
Adopción de estándares abiertos que garantizan interoperabilidad entre vendors y herramientas de observabilidad, eliminando vendor lock-in.

CONTEXTO AUTOMÁTICO
Propagación transparente de contexto de trazabilidad que permite seguimiento de requests complejas a través de múltiples servicios y componentes.

INSTRUMENTACIÓN EFICIENTE
Sobrecarga mínima de rendimiento mediante batching inteligente, sampling configurable y exportación asíncrona optimizada para entornos de alta demanda.

FLEXIBILIDAD DE BACKEND
Compatibilidad con múltiples backend de observabilidad mediante protocolo OTLP estándar, permitiendo migración entre herramientas sin cambios en instrumentación.

ATRIBUCION DE RECURSOS
Enriquecimiento automático con metadatos de infraestructura que facilita troubleshooting y análisis de capacidad sin configuración manual.

2.4.5.4 Consideraciones de Implementación

ADOPCIÓN GRADUAL
OpenTelemetry permite implementación incremental mediante bridges con bibliotecas de logging existentes, facilitando migración sin disrupciones operacionales.

SOBRECARGA DE RENDIMIENTO
Consideración del impacto en latencia y throughput especialmente en servicios de alta frecuencia que requieren configuración específica de sampling y batching.

GESTIÓN DE VOLUMEN
Planificación de estrategias de sampling y filtrado para controlar volumen de logs en sistemas de alta escala como los de RENIEC.

CONFIGURACIÓN DE CONTEXTO
Necesidad de configuración adecuada de trace context propagation para garantizar correlación efectiva en arquitecturas complejas de microservicios.

COMPATIBILIDAD CON SISTEMAS LEGACY
Evaluación de compatibilidad con sistemas de logging existentes y planificación de estrategias de migración para infraestructura heredada (legacy)

2.4.5.5 Implementación Recomendada

CONFIGURACIÓN DE SDK
Configurar OpenTelemetry SDK con resource attributes específicos para cada servicio incluyendo service.name, service.version y deployment.environment consistentes con la nomenclatura organizacional.

INSTRUMENTACIÓN AUTOMÁTICA
Implementar auto-instrumentación para frameworks web y bibliotecas de acceso a datos para captura automática de logs con contexto completo sin modificaciones de código.

GESTIÓN DE SAMPLING
Configurar sampling strategies apropiadas para diferentes tipos de logs basado en nivel de severidad y volumen esperado, priorizando ERROR y WARN levels para captura completa.

CONFIGURACION DE EXPORTACION
Establecer configuración de exporters OTLP con batching optimizado, retry logic y timeouts apropiados para garantizar delivery confiable de telemetría.

ESTRATEGIA DE CORRELACION
Implementar propagación consistente de trace context a través de boundaries de servicio usando context propagators estandarizados para HTTP y messaging.

SEMANTICA DE RECURSOS
Definir semantic conventions consistentes para resource attributes y log attributes específicos del dominio RENIEC para facilitar queries y análisis posteriores.

MONITOIREO Y ALERTAMIENTO
Establecer monitoreo del pipeline de observabilidad misma para detectar pérdidas de logs, latencias de export y problemas de conectividad con backends.

CONSIDERACIONES DE SEGURIDAD
Implementar filtrado de información sensible mediante attribute processors y configurar encryption en transit para proteger datos de logs durante transmisión.

2.5 Logs de Auditoría

Los logs de auditoría tienen como finalidad registrar los eventos relacionados con las acciones de los usuarios, los sistemas y las aplicaciones, garantizando la responsabilidad, la integridad y el no repudio. Desde una perspectiva arquitectónica, estos logs deben estar integrados en la capa de seguridad y cumplimiento del ecosistema tecnológico de RENIEC, sirviendo como insumo para los procesos de auditoría interna, investigación forense digital y control de acceso a los sistemas.

Los registros de auditoría deben capturar información clave como:

- Identidad del usuario y del sistema que ejecuta la acción.
- Fecha, hora y origen de la operación.
- Resultado del evento (éxito o error).
- Elementos afectados (bases de datos, documentos, registros de identidad).
- Firma digital o hash de integridad.

2.5.1 Mecanismos de integridad, cifrado y retención

Cada evento de auditoría deberá firmarse digitalmente mediante algoritmos hash (SHA-256 o superiores), y almacenarse en repositorios inmutables o con control de versiones (WORM – Write Once Read Many).

Se recomienda cifrado en tránsito (TLS 1.3) y en reposo (AES-256) y políticas de retención diferenciadas:

- Eventos críticos: mínimo 5 años.
- Eventos operacionales: mínimo 2 años.

2.5.2 Formatos recomendados

Los formatos más adecuados para logs de auditoría dentro de RENIEC son:

- **Syslog (RFC 5424):** formato clásico de registro estructurado, ampliamente soportado por sistemas SIEM (Security Information and Event Management). Permite incluir campos como prioridad, identificador de host, timestamp y aplicación generadora.

- **Structured Logging (JSON):** ofrece una estructura flexible con campos definidos para usuario, evento y nivel de severidad, facilitando la interoperabilidad entre sistemas y APIs modernas.

Estos formatos garantizan compatibilidad con herramientas de análisis, estandarización en la semántica de eventos y evidencia digital verificable para procesos legales o de control.

2.6 Logs de Trazabilidad

Los logs de trazabilidad permiten reconstruir el flujo completo de una transacción o solicitud a través de múltiples sistemas, garantizando la transparencia y la coherencia de los procesos institucionales. Desde una visión arquitectónica, estos logs se ubican en la capa intermedia de integración, capturando la interacción entre servicios, bases de datos y componentes de red.

La trazabilidad digital es esencial para RENIEC al permitir:

- Seguimiento de solicitudes de identidad o trámites en línea.
- Correlación de eventos entre microservicios.
- Validación de secuencias lógicas de procesamiento.
- Reconstrucción del flujo transaccional para auditorías o reclamaciones.

2.6.1 Control de flujo y aseguramiento de datos críticos

Los eventos trazables deberán incorporar campos de relación jerárquica (parent_id y child_id) y conservar una referencia temporal estandarizada en formato ISO 8601 (UTC), asegurando la correcta secuencia y correlación entre acciones. Para preservar la integridad de la cadena de eventos, se recomienda implementar hash chaining, de modo que cada registro contenga el hash del evento anterior, impidiendo alteraciones o inserciones no autorizadas. Complementariamente, pueden emplearse tecnologías de certificación criptográfica, como blockchain interno o árboles de Merkle, para reforzar la autenticidad, inmutabilidad y validación cronológica de los eventos críticos dentro del sistema de logs institucional.

2.6.2 Formatos recomendados

Los formatos más apropiados para la implementación de trazabilidad son:

- **Structured Logging (JSON):** debido a su capacidad para almacenar identificadores únicos (trace_id, request_id, session_id), permite correlacionar eventos entre componentes distribuidos.
- **JSON Logging:** implementación técnica del anterior, optimizada para herramientas de procesamiento automatizado y análisis masivo

Estos formatos soportan el seguimiento secuencial de eventos, la detección de inconsistencias y la construcción de trazas distribuidas.

2.7 Logs de Observabilidad

La observabilidad constituye la capacidad de comprender el estado interno de un sistema complejo a partir de los datos que emite: logs, métricas y trazas. En una arquitectura de microservicios, los logs de observabilidad son el eje de la operación continua, permitiendo a los equipos detectar anomalías, medir rendimiento y anticipar fallos.

En RENIEC, estos logs son esenciales para:

- Monitoreo del rendimiento de servicios de identificación y autenticación.
- Detección de errores en tiempo real.
- Correlación entre métricas de infraestructura y eventos de aplicación.
- Mejora continua del SLA y resiliencia operativa.

2.7.1 Estándares y frameworks de observabilidad adoptados globalmente

- **OpenTelemetry (CNCF):** estándar abierto que unifica la recolección de métricas, traces y logs, permitiendo instrumentación consistente entre componentes.
- **W3C Trace Context:** define encabezados HTTP estandarizados para el trazo de solicitudes distribuidas.
- **Google SRE Guidelines:** lineamientos de ingeniería de confiabilidad para observabilidad basada en indicadores SLI/SLO.
- **CNCF Best Practices:** define patrones de agregación, etiquetado y contextualización de logs.

2.7.2 Formatos recomendados

Los formatos más eficaces para observabilidad en RENIEC son:

- **Logfmt:** formato plano clave=valor, legible y eficiente, ideal para ingestión masiva en sistemas de monitoreo.
- **OpenTelemetry Logs:** modelo moderno de telemetría estructurada, que permite correlacionar trace_id, span_id y atributos de recurso para análisis integral.

Estos formatos deben integrarse con pipelines de procesamiento como **Prometheus, Loki, Grafana o ELK**, garantizando visualización centralizada, retención eficiente y alertas automatizadas.

2.8 Consideraciones sobre Bases de Datos No Relacionales y Backup de Logs

El sistema de logging debe estar diseñado para gestionar grandes volúmenes de datos de manera eficiente, garantizando **escalabilidad horizontal, alta disponibilidad y retención controlada** según la criticidad de los eventos. En este contexto, las bases de datos no relacionales ofrecen una solución óptima por su capacidad para manejar **datos semiestructurados** y **consultas dinámicas** sin comprometer el rendimiento. Este enfoque brinda flexibilidad tecnológica, permitiendo a la organización seleccionar la plataforma más adecuada conforme a sus necesidades operativas, sin depender de un proveedor o motor específico.

El modelo de almacenamiento se basa en una estructura JSON estandarizada que facilita la trazabilidad, la auditoría y el análisis correlativo de eventos. Cada registro conserva la relación jerárquica entre operaciones (padre-hijo), una marca temporal normalizada en formato **ISO 8601 (UTC)** y mecanismos de integridad criptográfica para garantizar la autenticidad de los datos.

Ejemplo de estructura de log:


```
{
  "event_id": "a12f3e9b-45b7-4f0d-83d1-9afc782b16a5",
  "timestamp": "2025-11-08T13:45:23Z",
  "nivel": "INFO",
  "servicio": "autenticacion_ciudadana",
  "origen": {
    "ip": "192.168.20.5",
    "entidad": "RENIEC",
    "modulo": "api-autenticacion"
  },
  "usuario": {
```

```
    "id": "DNI-45678901",
    "rol": "validador",
    "accion": "inicio_sesion"
  },
  "evento": {
    "descripcion": "Inicio de sesión exitoso",
    "resultado": "autenticado",
    "duracion_ms": 243
  },
  "relacion": {
    "parent_id": "d93bf561-11a3-4b5a-8c8b-df6b2cc84f40",
    "child_id": null
  },
  "integridad": {
    "hash_actual": "b79a0dce12ffbb8a13a...",
    "hash_anterior": "f9ac18d7b1c3ef44d6c...",
    "metodo": "SHA-256"
  },
  "metadata": {
    "version": "1.0",
    "entorno": "produccion",
    "retencion_dias": 365
  }
}
```


Esta estructura representa un modelo estandarizado para el registro de eventos técnicos dentro de un sistema institucional. Su propósito es capturar información clave de manera estructurada, auditable y procesable por máquinas, permitiendo realizar análisis, auditorías y correlaciones entre servicios o usuarios.

A continuación, se explica cada componente de la estructura:

- **event_id:** identificador único y universal del evento. Permite distinguirlo dentro del sistema y enlazarlo con otros registros relacionados.
- **timestamp:** fecha y hora exacta en formato **ISO 8601 (UTC)**, lo que asegura sincronización temporal entre sistemas distribuidos.
- **nivel:** severidad o tipo del evento (por ejemplo, *INFO*, *WARN*, *ERROR*), útil para priorizar alertas o análisis de incidentes.
- **servicio:** nombre del componente o módulo que generó el evento (por ejemplo, autenticación, validación, registro).
- **origen:** conjunto de metadatos que describe el contexto técnico donde ocurrió el evento, incluyendo dirección IP, entidad responsable y módulo específico.
- **usuario:** información asociada al actor que originó o fue afectado por el evento. Incluye su identificador, rol y la acción ejecutada.
- **evento:** descripción concreta de lo ocurrido, el resultado obtenido y parámetros de rendimiento (por ejemplo, duración de la operación en milisegundos).
- **relación:** define la jerarquía o dependencia entre eventos. Los campos *parent_id* y *child_id* permiten reconstruir cadenas de procesos o flujos de transacciones.
- **integridad:** sección dedicada a garantizar la autenticidad e inmutabilidad del evento. Contiene el hash del registro actual, el hash del evento anterior y el algoritmo criptográfico utilizado (como SHA-256).
- **metadata:** campos complementarios sobre el entorno operativo (por ejemplo, versión del esquema, ambiente de despliegue o política de retención del registro).

Ventajas técnicas clave:

- **Escalabilidad horizontal sin interrupciones**, permitiendo el crecimiento del sistema conforme aumente la demanda.

- **Eficiencia en búsquedas** por clave, usuario, nivel o servicio gracias a la indexación automática.
- **Compatibilidad con estructuras JSON y datos anidados**, que facilita la integración con motores analíticos y herramientas de observabilidad.
- **Particionamiento inteligente** por fecha, aplicación o criticidad, optimizando almacenamiento y retención.
- **Integridad y trazabilidad reforzada**, mediante encadenamiento de hashes (*hash chaining*), lo que evita manipulaciones intermedias y preserva la confiabilidad de los registros.

Estrategias de respaldo y retención:

- **Snapshots automatizados diarios** y replicación en entornos seguros.
- **Almacenamiento inmutable (WORM)** para garantizar que los logs críticos no puedan ser modificados.
- **Validación periódica de integridad** mediante hash de bloques y pruebas de recuperación.
- **Retención segmentada por nivel de criticidad**, manteniendo logs de auditoría y seguridad por un año y los operativos por períodos más cortos.

Este enfoque proporciona a **RENIEC** una infraestructura de registro resiliente, auditable y escalable, fundamental para la trazabilidad de eventos, la protección de datos sensibles y la continuidad de las operaciones en su ecosistema digital.

2.9 Desafíos de Logging para microservicios

Tabla 2. Desafíos de Logging

Desafío	Impacto	Requisito de Logging
Trazabilidad Distribuida	Alto	Correlation IDs, Distributed Tracing
Debugging Complejo	Crítico	Logs estructurados, contexto completo
Múltiples Tecnologías	Medio	Estándar agnóstico al lenguaje
Escalabilidad	Alto	Performance, agregación eficiente
Observabilidad	Crítico	Logs + Métricas + Trazas unificadas
Troubleshooting	Crítico	Búsqueda rápida, correlación temporal

Syslog: no tiene correlation IDs nativos. En microservicios, un simple user request toca 5-10 servicios. Sin correlation, debugging toma horas en lugar de minutos.

JSON sin OpenTelemetry: es como tener partes de un puzzle sin la imagen completa. Se requiere un mecanismo de distributed tracing para ver el full request flow.

Solución personalizada: Reinventar la observabilidad es como escribir tu propio database. OpenTelemetry es el estándar que usan Google, Microsoft, Amazon - battle-tested a escala."

2.10 Retención de Logs por nivel de Criticidad

La retención de logs es un componente crítico para garantizar la trazabilidad, auditoría, investigación forense, control de calidad y monitoreo de incidentes dentro de los sistemas del RENIEC.

La duración de almacenamiento debe estar alineada a la criticidad del evento registrado, la naturaleza del sistema, los requisitos regulatorios y la necesidad de mantener evidencia verificable para procesos administrativos o legales.

A continuación, se definen los tiempos mínimos institucionales que deben cumplir todos los sistemas:

2.10.1 Tiempos mínimos de retención según nivel de criticidad

2.10.1.1 Nivel INFO

Tiempo mínimo: 6 meses

Registra eventos informativos relacionados con el funcionamiento normal del sistema, como métricas operativas, accesos generales y estados rutinarios de los servicios. Estos datos permiten monitorear rendimiento y continuidad sin asociarse a fallas o actividades anómalas. Su retención es menor, dado que no representan riesgos operativos ni de seguridad y tienen un valor limitado para auditorías o análisis forense.

Ejemplos:

- Inicio y finalización de procesos rutinarios.
- Logs de acceso general no sensibles.
- Ejecuciones de tareas automatizadas sin errores.

2.10.1.2 Nivel WARNING

Tiempo Mínimo: 1 año

Registra condiciones inusuales o comportamientos atípicos que, aunque no afectan la operación inmediata, podrían evolucionar en fallas o incidentes. Su retención extendida permite identificar patrones recurrentes, anticipar riesgos y aplicar medidas preventivas antes de que escalen.

Ejemplos:

- Retrasos en procesamiento.
- Fallas recuperables.
- Datos incompletos o inconsistentes detectados pero corregidos automáticamente.

2.10.1.3 Nivel ERROR

Tiempo Mínimo:3 años

Incluye fallas que afectan la continuidad del servicio, requieren intervención humana o pueden generar consecuencias operativas, técnicas o legales. Este nivel registra errores que comprometen la correcta ejecución de procesos críticos o la calidad de los datos institucionales. Su retención prolongada permite realizar auditorías, trazabilidad histórica y análisis forense detallado ante incidentes relevantes.

Ejemplos:

- Errores en validación biométrica.
- Fallas en integraciones o API críticas.
- Interrupciones de servicio.

2.10.1.4 Nivel CRITICAL / SECURITY

Tiempo Mínimo: 5 años

Los eventos que comprometen la integridad, disponibilidad o confidencialidad de los sistemas incluyen accesos no autorizados, intentos de intrusión, fallas críticas o alteración de datos sensibles. Debido a su alto impacto operativo y de seguridad, requieren una retención prolongada que permita asegurar la trazabilidad completa del incidente, así como el soporte a auditorías internas o investigaciones forenses. Esta conservación extendida garantiza que la institución pueda reconstruir hechos, validar responsabilidades y mantener la continuidad de las operaciones bajo estándares de seguridad.

Ejemplos:

- Intentos fallidos de autenticación repetidos.
- Detección de actividad sospechosa o malware.
- Accesos no autorizados o violaciones de seguridad.
- Alteración de datos sensibles.

2.11 Finalidad de los Logs

Los logs no solo registran lo que ocurre, sino que son un **mecanismo de trazabilidad, análisis y respuesta** ante eventos del sistema. Su finalidad se clasifica en:

- **Seguridad**: Detección de accesos sospechosos, intentos fallidos de autenticación, actividad fuera de horario, o desde ubicaciones anómalas.
- **Auditoría**: Generación de evidencias para cumplimiento normativo (ISO 27001, Ley 29733).
- **Soporte y resolución de fallos**: Análisis post-mortem para encontrar la causa raíz de incidentes o caídas de servicio.
- **Optimización de procesos**: Detectar cuellos de botella o ineficiencias en consultas, tiempos de respuesta, o uso de recursos.
- **Machine Learning:** Entrenamiento de modelos con datos históricos para predecir anomalías o anticipar fallas.

3 Estándar de Timestamp y Zonas Horarias

La gestión uniforme del tiempo en los logs es esencial para garantizar la trazabilidad, auditoría y seguridad de los sistemas del RENIEC. Cada evento registrado como biométrico, transaccional, de autenticación o de interoperabilidad requiere un timestamp exacto y consistente para permitir correlación entre plataformas distribuidas y validar secuencias de operación.

Dado que el RENIEC opera con múltiples sistemas, oficinas descentralizadas y servicios integrados con entidades del Estado, cualquier diferencia temporal puede generar inconsistencias en auditorías, fallas en análisis forense y riesgos en procesos críticos de identificación. Por ello, este capítulo define el estándar obligatorio de timestamps y zona horaria para asegurar un punto temporal único, verificable y coherente en todos los registros de logs institucionales.

3.1 Principios Generales

Los siguientes principios establecen las bases que deben cumplir todos los sistemas del RENIEC para asegurar una gestión temporal coherente, confiable y alineada con las necesidades operativas, legales y de auditoría de la institución:

- **Unificación temporal institucional:** Todos los sistemas deben utilizar un único estándar temporal para mantener consistencia en la trazabilidad de eventos.
- **Eliminación de ambigüedades:** Los timestamps deben registrarse con un formato claro y sin variaciones locales que puedan afectar auditorías o análisis de incidentes.

- **Correlación precisa entre plataformas:** El tiempo registrado debe permitir unir eventos generados en microservicios, sistemas biométricos, bases de datos y plataformas de interoperabilidad.
- **Independencia de ubicación geográfica:** Todas las oficinas y nodos deben mantener una referencia temporal uniforme sin depender de configuraciones locales.
- **Soporte a procesos críticos:** Los procesos sensibles del RENIEC —identificación, biometría, autenticación y trazabilidad del SIIRC— deben operar sobre un tiempo confiable.
- **Compatibilidad con estándares internacionales:** El manejo del tiempo debe alinearse con Syslog moderno, OpenTelemetry, JSON Logging y lineamientos ISO/NIST para garantizar interoperabilidad y buenas prácticas

3.2 Zona Horaria Institucional (UTC)

El RENIEC adopta **UTC (Coordinated Universal Time)** como la única zona horaria permitida para el registro de eventos en todos los sistemas que generan logs. Esta estandarización garantiza que la información temporal sea coherente en plataformas distribuidas y evita discrepancias que afecten auditorías, trazabilidad y análisis forense.

Reglas institucionales:

- Todos los timestamps deben registrarse exclusivamente en UTC, sin conversión previa a la hora local (UTC−5).
- No se permite el uso de zonas horarias locales, configuraciones regionales ni ajustes automáticos por horario de verano.
- La zona horaria UTC debe aplicarse de manera uniforme en microservicios, sistemas biométricos, bases de datos, aplicativos web, infraestructura de red, dispositivos y plataformas de interoperabilidad.
- Los sistemas heredados que operen con hora local deberán ser ajustados para emitir logs en UTC o convertirlos automáticamente antes de su almacenamiento centralizado.

Justificación para RENIEC:

- Facilita la correlación exacta entre eventos provenientes de oficinas descentralizadas, servicios biométricos, SIIRC, módulos de autenticación y sistemas de terceros del Estado.
- Evita inconsistencias temporales en procesos críticos como emisión de DNI, validación documental, biometría, auditoría de transacciones y firma digital.
- Permite análisis precisos dentro de herramientas SIEM, plataformas de observabilidad y soluciones de monitoreo unificado.

3.3 Formato de Timestamps (ISO 8601)

El RENIEC adopta el estándar **ISO 8601** como formato obligatorio para todos los timestamps generados por sistemas internos, microservicios, plataformas biométricas, bases de datos, infraestructura de red y aplicaciones integradas. Este formato garantiza uniformidad, evita ambigüedades y asegura compatibilidad con herramientas de observabilidad, auditoría y análisis forense.

3.3.1 Especificación técnica del formato

El formato ISO 8601 define una estructura clara, no ambigua y universal para la representación de fecha y hora. El formato institucional obligatorio es:

YYYY-MM-DDThh:mm:ss.sssZ

Componentes:

- **YYYY:** año en cuatro dígitos.
- **MM:** mes (01–12) con cero inicial obligatorio.
- **DD:** día del mes (01–31) también con cero inicial.
- **T:** carácter separador normativo entre fecha y hora.
- **hh:** hora (00–23).
- **mm:** minutos (00–59).
- **ss:** segundos (00–59).
- **sss:** milisegundos (precisión mínima exigida).
- **Z:** indica que el tiempo está expresado explícitamente en **UTC**.

Ejemplo correcto

2025-11-19T15:27:54.128Z

3.3.2 Reglas de formato obligatorias

- **Uso obligatorio de milisegundos:** se requieren tres dígitos (.sss) para garantizar precisión en correlación de eventos.
- **Prohibido el uso de formatos abreviados, locales o con separadores no normativos** (p. ej., “19/11/2025 3:27 PM”).
- **Prohibido el uso de zonas horarias numéricas** (+00:00, -05:00, etc.); solo se permite **Z** (UTC).
- **Prohibido omitir el símbolo “T”** entre fecha y hora.
- **Debe mantenerse estrictamente la representación en formato expandido**, sin variaciones compactas (“20251119T152754Z”).
- **La codificación debe ser compatible con Unicode o UTF-8** en todas las plataformas.

3.3.3 Razones institucionales para adoptar ISO 8601

El RENIEC maneja datos sensibles y servicios distribuidos de alta criticidad, por lo que la precisión temporal es clave. ISO 8601 se adopta por las siguientes razones:

- **Evita ambigüedades interpretativas** entre sistemas heredados con formatos distintos.
- **Facilita la correlación entre eventos** en SIEM, OpenTelemetry, Syslog moderno y plataformas de trazabilidad.
- **Soporta análisis forense digital**, donde la reconstrucción de la línea de tiempo debe ser inequívoca.
- **Se integra sin ajustes** en servicios biométricos (AFIS, reconocimiento facial), módulos de autenticación, bases de datos transaccionales y servicios de emisión del DNI.
- **Permite la interoperabilidad directa** con entidades externas del Estado que emplean estándares internacionales.
- **Reduce errores de conversión** en pipelines de logs, ETLs y sistemas centralizados como el SIIRC.

3.3.4 Consideraciones para sistemas heredados

Sistemas antiguos que generen timestamps en formatos locales deberán:

- Convertir automáticamente su fecha/hora al formato ISO 8601 + UTC antes de almacenar o transmitir el log.
- Ajustar la configuración de sus librerías de fecha para garantizar el cumplimiento del estándar.
- Documentar cualquier limitación técnica en un plan de adecuación institucional

3.4 Sincronización del Tiempo (NTP)

La sincronización de tiempo mediante Network Time Protocol (NTP) es un requisito fundamental para garantizar que todos los sistemas del RENIEC operen bajo una referencia temporal uniforme y confiable. Dado que la institución maneja procesos sensibles como validación biométrica, emisión de documentos de identidad, trazabilidad de transacciones y auditoría legal; incluso una mínima divergencia en los relojes de los sistemas puede afectar la correlación de eventos o la validez de un registro.

La sincronización precisa evita discrepancias entre plataformas que operan de forma distribuida (oficinas, centros de datos, nube, interoperabilidad), permitiendo reconstruir líneas de tiempo de forma exacta durante análisis forense, investigación de incidentes o auditorías internas y externas.

3.4.1 Requisitos técnicos de la sincronización NTP

3.4.1.1 Fuentes de tiempo institucionales

- El RENIEC debe mantener servidores NTP internos que actúen como autoridad temporal para todos los sistemas.
- Dichos servidores deben sincronizarse, a su vez, con fuentes externas confiables (“stratum 1” o “stratum 2”) aprobadas institucionalmente.
- Los sistemas críticos (biometría, SIIRC, autenticación, servicios de auditoría) deben apuntar siempre a los servidores NTP institucionales para reducir dependencia externa.

3.4.1.2 Redundancia y continuidad operativa

- Se deben configurar mínimo dos servidores NTP internos para asegurar continuidad en caso de fallas.
- Los clientes deben tener configurado un conjunto de servidores NTP, nunca uno solo, para minimizar riesgo de desincronización.
- Si una instancia pierde conectividad temporal, debe mantener su local clock con tolerancias controladas hasta restablecer sincronización.

3.4.1.3 Control del clock drift

- El límite de clock drift permitido debe definirse institucionalmente (por ejemplo ±50 ms) para sistemas de alta sensibilidad temporal.
- Los servicios deben monitorear periódicamente su desviación de tiempo e iniciar alertas si el desfase supera los umbrales definidos.
- Servicios biométricos y sistemas de autenticación deben operar con desfase mínimo, ya que pequeñas diferencias pueden impedir correlación precisa de eventos biométricos capturados.

3.4.1.4 Aplicación en infraestructura

- Servidores físicos y virtuales: deben usar NTP institucional directamente.
- Contenedores: deben heredar sincronización del host; no deben implementar sincronización independiente dentro del contenedor.
- Nodos en la nube: deben utilizar sincronización mediante el servicio oficial del proveedor y, cuando sea posible, vincularse al NTP institucional.
- Dispositivos biométricos: deben configurarse para sincronizar tiempo automáticamente con servidores institucionales, evitando configuraciones manuales en campo.

3.4.1.5 Monitoreo y verificación

- La plataforma de observabilidad o SIEM debe incluir verificaciones periódicas del estado de sincronización de todos los sistemas.
- Los fallos de sincronización deben registrarse como eventos críticos, dado que pueden afectar auditorías y la integridad del flujo de logs.
- El estado NTP de cada host debe formar parte de los chequeos de cumplimiento para sistemas críticos.

3.5 Reglas de uso y Conversión del Tiempo

Las siguientes reglas garantizan que los timestamps registrados por los sistemas del RENIEC mantengan coherencia desde su origen hasta su visualización final, resguardando la integridad temporal en procesos críticos, auditorías y análisis forense.

3.5.1 Reglas Institucionales Ampliadas

3.5.1.1 Registro obligatorio en UTC en el origen

Todos los sistemas deben generar sus timestamps directamente en UTC, sin depender de configuraciones locales del sistema operativo, contenedores, aplicaciones o bases de datos. Esto asegura que los registros sean uniformes y comparables independientemente de la ubicación física del servicio o del equipo desde el cual se origina el evento.

3.5.1.2 Almacenamiento único en UTC

Todo log debe conservar su timestamp original en UTC en:

- Archivos de log locales
- Bases de datos relacionales y no relacionales
- Sistemas de auditoría
- Brokers de mensajería
- Sistemas de observabilidad y SIEM
- Repositorios históricos o de respaldo

Esto garantiza que no existan variaciones temporales al momento de consolidar o exportar eventos.

3.5.1.3 Conversión solo en la capa de visualización

Cualquier conversión a hora local (UTC−5) debe realizarse únicamente en:

- Dashboards de monitoreo
- Consolas operativas
- Reportes de gestión
- Interfaces de usuario
- Herramientas ad-hoc de análisis

Nunca se debe modificar el timestamp original almacenado, ya que constituye evidencia técnica y legal de lo ocurrido.

3.5.1.4 Prohibición de conversión previa al almacenamiento

Queda explícitamente prohibido convertir timestamps a hora local antes de guardarlos en:

- Logs de microservicios
- Registros biométricos
- Eventos del SIIRC
- Auditorías de transacciones
- Logs generados en aplicaciones web o móviles

Cualquier conversión previa introduce ambigüedad y afecta procesos de investigación o correlación de eventos.

3.5.1.5 Conservación obligatoria del valor en UTC en exportaciones

Al extraer, integrar o compartir logs con entidades externas del Estado, proveedores o plataformas federadas:

- Se debe mantener el timestamp original en UTC.
- Si se requiere mostrar la hora local, esta debe agregarse como campo adicional, nunca reemplazar la hora UTC.

3.5.1.6 Conversión inversa para sistemas legados

En caso de sistemas heredados que aún operan en hora local:

- Deben convertir sus timestamps a UTC **antes de enviarlos al sistema centralizado de logs**.
- La conversión debe ser automática y documentada.
- No se deben almacenar valores mixtos (UTC + local), ya que dificultan la trazabilidad.

3.5.1.7 No manipulación de timestamps en procesos internos

Tareas internas como:

- Pipelines ETL,
- Normalización de datos,
- Transformaciones logísticas,
- Migraciones,
- Replicaciones

No deben modificar el timestamp original. Cualquier transformación debe preservarlo como campo inmutable.

4 Estándar de Controles Criptográficos

Los Establecer los lineamientos técnicos se emplean para la implementación de controles criptográficos en los sistemas de información que componen la plataforma SIIRC de RENIEC, con el fin de proteger la confidencialidad, integridad y autenticidad de la información. Este estándar adopta un enfoque basado en riesgo y cumplimiento normativo, conforme a buenas prácticas internacionales definidas por ISO/IEC 27001, ISO/IEC 27002, ISO/IEC 27005, NIST SP 800-57, y las recomendaciones del OWASP Cryptographic Storage Cheat Sheet.

4.1 Principios Generales

Los controles criptográficos del SIIRC se aplican para garantizar la **confidencialidad, integridad, autenticidad y no repudio** de la información tratada por los sistemas de RENIEC. Su implementación debe regirse por los siguientes principios:

- **Actualización y solidez técnica:** usar únicamente algoritmos y configuraciones criptográficas seguras y vigentes, aprobadas por organismos internacionales (ISO/IEC, NIST, OWASP).
- **Proporcionalidad y clasificación:** aplicar mecanismos de protección acordes al nivel de sensibilidad y riesgo de la información (pública, interna, confidencial o restringida).
- **Gestión integral de claves:** garantizar la generación, almacenamiento, rotación y eliminación segura de claves, mediante procesos auditables y controlados por el área de seguridad.
- **Aplicación transversal:** asegurar la protección criptográfica de datos en tránsito, en reposo y, cuando sea necesario, en uso, cubriendo toda la arquitectura tecnológica del SIIRC.

- **Evitar algoritmos obsoletos:** prohibir el uso de tecnologías vulnerables como DES, 3DES, RC4, MD5 o SHA-1 en cualquier entorno o sistema.
- **Auditoría y trazabilidad:** mantener evidencias de las operaciones criptográficas realizadas y permitir su monitoreo dentro del modelo de cumplimiento de RENIEC.
- **Cumplimiento normativo:** alinear la aplicación de los mecanismos criptográficos con la Ley de Protección de Datos Personales, los estándares ISO/IEC 27001, 27002, 27005, y las buenas prácticas OWASP.

4.1.1 Practicas Criptográficas Recomendadas

Se deben aplicar prácticas criptográficas robustas para proteger la confidencialidad e integridad de la información, tanto en tránsito como en reposo, siguiendo estándares actualizados y evitando tecnologías obsoletas.

Se implementan mecanismos de seguridad que incluyen cifrado simétrico, asimétrico y funciones hash. Para el caso específico del enmascaramiento de datos, RENIEC requiere establecer lineamientos claros que definan su aplicación según la criticidad y nivel de sensibilidad de la información.

Uso de protocolos seguros de comunicación:

- Emplear TLS 1.2 o superior, siendo TLS 1.3 altamente recomendado por sus mejoras en seguridad, rendimiento y reducción del número de intercambios en el protocolo. Se deben deshabilitar todos los protocolos inseguros u obsoletos, como SSL y TLS 1.0/1.1.

Cifrado de datos sensibles:

- En tránsito: Utilizar canales cifrados a través de HTTPS con certificados digitales válidos, emitidos por autoridades de certificación confiables y con mecanismos automatizados de renovación.
- En reposo: Aplicar cifrado a nivel de columna, archivo o disco completo en sistemas de almacenamiento, incluyendo bases de datos y copias de seguridad.

Algoritmos criptográficos recomendados:

- Cifrado simétrico: Utilizar algoritmos seguros como AES con claves de al menos 256 bits.
- Cifrado asimétrico: Utilizar algoritmos como RSA con claves de 2048 bits o superiores, o curvas elípticas (ECC) que provean seguridad equivalente con menor consumo computacional.

Gestión segura de claves criptográficas:

Las claves deben almacenarse utilizando mecanismos seguros y aislados, como módulos de hardware o servicios dedicados de gestión de claves. Además, se deben aplicar:

- Rotación periódica de claves.
- Segregación por entorno (desarrollo, pruebas, producción).
- Controles de acceso basados en roles y auditoría de uso.

4.2 Cifrado

El cifrado transforma información legible (texto plano) en un formato ilegible (texto cifrado), que solo puede ser interpretado nuevamente por entidades autorizadas mediante el uso de claves.

Tipos de cifrado:

- Reversible (con clave) → Cifrado simétrico y asimétrico.
- No reversible → Hash y enmascaramiento.

Todo dato clasificado como confidencial o restringido debe cifrarse cuando:

- Se almacena en bases de datos, archivos, volúmenes o respaldos.
- Se transmite a través de redes públicas o no confiables.
- Se comparte con terceros mediante APIs, brokers o sistemas de mensajería.

4.2.1 Cifrado Simétrico

El cifrado simétrico usa una misma clave secreta para cifrar y descifrar datos. Es la opción por rendimiento para cifrado en reposo y en tránsito una vez establecida la sesión. Sin embargo, la seguridad real depende críticamente de: elección de algoritmo y modo, gestión de claves, uso correcto de IV/nonce, protección contra padding/oracle attacks y autenticación de los datos.

Tabla 3. Parámetros obligatorios cifrado simétrico

Elemento	Valor Requerido	Implicación práctica
Algoritmo	AES (Advanced Encryption Standard)	AES es estándar mundial (FIPS 197). Implementaciones seguras y aceleración hardware (AES-NI) disponibles.
Longitud de clave	256 bits	AES-256 ofrece margen de seguridad. Recomendado para datos *confidenciales/restringidos*.
Modo de operación	CBC (Cipher Block Chaining)	CBC es clásico; correcto si se aplican contra-medidas (IV aleatorio por mensaje + autenticación). No obstante, se recomienda preferir AEAD cuando sea posible.
Relleno (padding)	PKCS#7	Necesario para CBC; conlleva riesgo de padding oracle si no se maneja bien. Preferible usar AEAD sin padding.

Uso sugerido: Cifrado de bases de datos, archivos, almacenamiento en la nube, tokens de sesión y datos sensibles en servicios internos.

4.2.2 Cifrado Asimétrico

El cifrado asimétrico es un método criptográfico que utiliza un par de claves matemáticamente relacionadas; una clave pública para cifrar la información y una clave privada para descifrarla, garantizando así la confidencialidad, autenticidad e integridad en las comunicaciones.

A diferencia del cifrado simétrico, no requiere compartir una misma clave entre las partes, lo que reduce significativamente los riesgos de exposición. Aunque es computacionalmente más costoso, su aplicación resulta fundamental en entornos donde se manejan datos altamente sensibles o intercambios entre sistemas distribuidos, como en las infraestructuras de RENIEC.

El algoritmo RSA (Rivest–Shamir–Adleman), con una longitud mínima de 2048 bits según las recomendaciones del NIST SP 800-131A, se utiliza como estándar para el intercambio seguro de claves simétricas, autenticación mutua, cifrado de tokens y firma digital, aplicando esquemas de relleno como PKCS#1 v1.5 o RSA-OAEP para reforzar la resistencia frente a ataques criptográficos.

Tabla 4. Parámetros obligatorios cifrado simétrico

Elemento	Valor requerido
Algoritmo	RSA (Rivest–Shamir–Adleman)
Longitud de clave	2048 bits (mínimo, conforme a NIST SP 800-131A)
Relleno (Padding)	PKCS#1 v1.5 o RSA-OAEP

Uso sugerido: Intercambio de claves simétricas, autenticación mutua entre sistemas, cifrado de tokens, firma digital.

4.3 Funciones Hash

Una función hash transforma un mensaje de longitud variable en una huella digital (digest) de longitud fija. No es posible recuperar el mensaje original desde su hash (no reversible).

enmascaramiento de dato

Tabla 5. Parámetros obligatorios funciones Hash

Elemento	Valor requerido
Algoritmo	SHA-512 (Secure Hash Algorithm, RFC 6234)

Aplicaciones: Verificación de integridad, hash de contraseñas (combinado con salt y mecanismos como bcrypt/scrypt/Argon2), almacenamiento seguro de identificadores anonimizados, verificación de mensajes firmados digitalmente.

Funciones Hash – Aplicación en SIIRC

Transformación irreversible de datos en huellas digitales únicas.
Algoritmo Estándar: SHA-512

Usos Específicos en SIIRC:

Almacenamiento Seguro de Contraseñas
- Combinado con salt + bcrypt/Argon2
- Ejemplo: contraseña + salt → hash almacena

Verificación de Integridad de Datos
- Documentos, certificados, actas digitales
- Ejemplo: Hash de archivo PDF vs. hash almacenado

Identificadores Anonimizados
- Para análisis y reporting sin exponer datos reales
- Ejemplo: DNI + salt → identificador único anónimo

Firmas Digitales y Validación
- Hash del mensaje + firma criptográfica
- Verificación de autenticidad en transacciones

Reduplicación de Registros
- Identificación de registros duplicados sin comparar datos sensibles
- Ejemplo: Hash de conjunto de datos para comparación

4.3.1 Gestión segura de contraseñas

Almacenar contraseñas usando algoritmos hash seguros:

- Utilizar funciones de hash con sal (salt) como:
  - bcrypt (recomendado por OWASP).
  - scrypt o Argon2 para mayor resistencia a ataques por hardware (GPU/FPGA).

Aplicar políticas de complejidad, caducidad y reuso:

- Longitud mínima (por ejemplo, 12 caracteres).
- Inclusión de letras mayúsculas, minúsculas, números y caracteres especiales.
- Evitar contraseñas comunes o filtradas (mediante listas negras como haveibeenpwned).
- Prohibir reúso reciente mediante almacenamiento de hashes antiguos.

4.4 Blockchain

El estándar de controles criptográficos del RENIEC al referirse a Blockchain, se aborda como una tecnología de registro distribuido de criptografía para garantizar la inmutabilidad y la confianza en el registro de la información.

Blockchain, o cadena de bloques, se describe como un registro digital compartido (libro mayor distribuido) que emplea técnicas criptográficas avanzadas para asegurar que las transacciones o datos, una vez registrados, no puedan ser modificados ni eliminados.

El estándar del RENIEC reconoce a la tecnología Blockchain como una implementación práctica de principios criptográficos avanzados para garantizar la integridad, no repudio y trazabilidad auditiva de la información. Su aplicación no sustituye, sino que se basa en y extiende, los controles criptográficos definidos en este documento.

4.4.1 Aplicación en RENIEC (Implícita)

El interés de RENIEC en Blockchain se relaciona con:

- Trazabilidad de la Identidad: Crear un historial inmutable de eventos clave en la vida de un ciudadano (nacimientos, cambios de estado civil, emisión de documentos).
- Emisión de Credenciales Verificables: Utilizar la cadena de bloques para emitir certificados digitales o credenciales que puedan ser verificados por cualquier tercero sin necesidad de consultar directamente la base de datos central de RENIEC.
- Credenciales Consumibles: Implementar autorizaciones digitales de uso limitado que permitan a ciudadanos o funcionarios ejecutar acciones específicas dentro del sistema. Cada credencial se consume una vez utilizada o al expirar, registrando su uso de forma inmutable para asegurar trazabilidad, control de acceso y prevención de operaciones no autorizadas.
- Mecanismo de Consenso y Gobernanza: Se definirá y documentará formalmente el protocolo de consenso (ej: Proof of Authority - PoA, o Blockchain Privada con nodos autorizados) que garantice la validez de las transacciones y la gobernanza de la red, priorizando la eficiencia y el control sobre modelos de incentivos.
- Criptografía de Aplicación: Toda transacción o dato registrado en la cadena debe firmarse digitalmente utilizando algoritmos asimétricos aprobados (ej: ECDSA con curvas elípticas seguras), asegurando el no repudio y la autenticidad del origen.
- Integridad de la Cadena: La inmutabilidad se garantiza mediante el uso de funciones hash criptográficas resistentes a colisiones (ej: SHA-256 o superiores) para enlazar los bloques. Cualquier alteración en un bloque invalidaría todos los bloques subsiguientes.
- Gestión de Claves: Las claves privadas utilizadas para firmar transacciones en la Blockchain estarán sujetas a los mismos controles rigurosos definidos en este estándar, incluyendo su custodia en HSMs o KMS y los procesos de rotación y revocación.

Aplicación Específica como Estándar de Seguridad

El RENIEC aplicará Blockchain como un componente de seguridad en los siguientes escenarios, siempre que se cumpla;

- Registro de Eventos Inmutables (Logging de Seguridad): Crear un registro distribuido e inmutable de eventos críticos, como accesos a bases de datos sensibles o cambios en el estado civil, proporcionando una traza de auditoría inviolable.
- Emisión de Credenciales Verificables: Implementar esquemas donde la validez de un documento digital pueda ser verificada de forma descentralizada y confiable contra un hash registrado en la Blockchain, reduciendo la dependencia de consultas a bases de datos centrales y previniendo la falsificación.

4.5 Enmascaramiento de los Datos

Se basará en una clasificación previa de la sensibilidad de la información. Este marco establecerá que el enmascaramiento será aplicado de manera obligatoria en todos los entornos que no sean de producción (ej. desarrollo, testing, calidad) para los datos catalogados como sensibles o de alto impacto, tales como números de documento nacional de identidad, información financiera o datos personales protegidos por ley. La técnica de enmascaramiento a utilizar (como el enmascaramiento parcial, la sustitución por valores anónimos o la aleatorización) será determinada por el nivel de confidencialidad asignado a cada campo de dato, garantizando que la información enmascarada conserve su utilidad funcional para las pruebas o desarrollos, pero sin revelar el valor real, mitigando así los riesgos de exposición indebida.

4.6 Consideraciones adicionales

- Las claves criptográficas, tanto para sistemas tradicionales como para aplicaciones Blockchain, deben gestionarse mediante HSMs o servicios de KMS.
- Es mandatorio implementar un proceso de rotación periódica de claves y revocación anticipada en caso de incidentes.
- Todos los algoritmos criptográficos (hash, firma digital) utilizados en cualquier implementación, incluyendo Blockchain, deben ser auditables y verificables por el equipo de seguridad.
- Se prohíbe terminantemente el uso de algoritmos obsoletos o vulnerables como DES, 3DES, RC4, MD5 o SHA-1 en cualquier capa del sistema, ya sea centralizada o descentralizada.

5 Estándar de Tratamiento de Datos Confidenciales

El Estándar de Tratamiento de Datos Confidenciales en el contexto de RENIEC establece las directrices para proteger la información sensible que maneja la entidad, asegurando que los datos personales y de identificación ciudadana sean tratados de manera segura, íntegra y confidencial durante todo su ciclo de vida.

Basado en los principios de la ISO 27001, este estándar busca fortalecer el Sistema de Gestión de Seguridad de la Información (SGSI) de RENIEC, garantizando que los procesos asociados a la recolección, almacenamiento, transmisión y eliminación de datos se realicen bajo controles técnicos y organizativos adecuados.

Su aplicación permite a RENIEC preservar la confianza ciudadana, cumplir con las exigencias de la Ley N.º 29733 Ley de Protección de Datos Personales, y mantener la disponibilidad y exactitud de la información crítica utilizada en los servicios de identificación, registro civil y certificación digital.

Asimismo, contribuye a la prevención de incidentes de seguridad, evitando accesos no autorizados, filtraciones o manipulaciones indebidas de la información bajo su custodia.

5.1 Principios Generales

Los principios generales del Estándar de Tratamiento de Datos Confidenciales en RENIEC garantizan que la información personal sea gestionada con seguridad, confidencialidad e integridad, promoviendo el uso responsable y limitado de los datos y el cumplimiento normativo, con el fin de proteger la privacidad ciudadana y fortalecer la confianza en los servicios de identificación.

Para materializar este compromiso, se han definido los siguientes principios generales:

- **Confidencialidad de la información:**
  Todo dato personal, biométrico o de identificación administrado por RENIEC debe ser accesible únicamente por personal autorizado y bajo condiciones de uso legítimas, evitando su divulgación o uso indebido.

- **Integridad de los datos:**
  Los sistemas y procesos deben garantizar que la información almacenada o transmitida no sea alterada, destruida o manipulada sin autorización, asegurando la exactitud y consistencia de los registros ciudadanos.

- **Disponibilidad controlada:**
  Los datos deben estar disponibles para su uso cuando sean requeridos por procesos institucionales o legales, manteniendo mecanismos de redundancia y continuidad operativa sin comprometer la seguridad.

- **Legalidad y transparencia en el tratamiento:**
  RENIEC debe asegurar que toda recolección y tratamiento de datos personales se realice con base en el consentimiento informado y con fines explícitos, legítimos y previamente definidos.

- **Minimización y proporcionalidad:**
  Solo se deben recopilar y conservar los datos estrictamente necesarios para cumplir con las funciones institucionales, evitando el almacenamiento o tratamiento excesivo.

- **Responsabilidad y trazabilidad:**
  RENIEC debe implementar mecanismos de auditoría y control que permitan evidenciar el cumplimiento de las políticas de seguridad, garantizando trazabilidad en el acceso y modificación de los datos.

- **Cumplimiento normativo y mejora continua:**
  El tratamiento de datos debe alinearse con la Ley N.º 29733, la ISO 27001 y su extensión ISO 27701, promoviendo revisiones periódicas, evaluación de riesgos y actualización de controles ante nuevas amenazas o cambios regulatorios.

5.2 Importancia específica para RENIEC

El RENIEC gestiona uno de los repositorios más sensibles del Estado, al concentrar datos personales y biométricos de la población peruana utilizados para la identificación oficial de los ciudadanos. Por ello, la aplicación rigurosa de los estándares de seguridad y privacidad resulta esencial para garantizar la confidencialidad, integridad y disponibilidad de esta información.

El cumplimiento de la norma ISO 27001 junto con la Ley N.º 29733, permite que RENIEC asegure un tratamiento de datos conforme a principios internacionales y obligaciones legales, reduciendo riesgos de acceso indebido, suplantación de identidad o filtración de información.

Asimismo, la adopción de controles criptográficos, mecanismos de enmascaramiento y gestión segura de claves refuerza la protección de los registros ciudadanos, asegurando que los procesos de almacenamiento, transmisión y consulta de datos se realicen bajo un marco de seguridad verificable y trazable.

En este sentido, el fortalecimiento del tratamiento de datos personales dentro de RENIEC no solo garantiza el cumplimiento normativo, sino que también preserva la confianza pública en los servicios de identificación y certificación digital del Estado.

5.3 Aspectos clave de la ISO 27001

Esta norma es fundamental para entidades públicas y privadas que gestionan información sensible, ya que promueve un enfoque sistemático y basado en riesgos, adaptable a las necesidades de cada institución. Además, su extensión hacia la **ISO/IEC 27701** refuerza los controles de privacidad, alineando la gestión de datos personales con los estándares internacionales de cumplimiento y protección de la información.

Aspectos Clave de la ISO 27001:

- **Extensión de la ISO 27001:** Se basa en los principios de seguridad de la información, pero añade requisitos específicos para la privacidad.
- **Sistema de Gestión de Privacidad de la Información (PIMS):** Define controles para la protección de datos personales dentro de una organización.
- **Roles específicos:** Establece requisitos tanto para controladores como para procesadores de datos personales.
- **Cumplimiento normativo:** Ayuda a las empresas a alinearse con regulaciones internacionales de privacidad.
- **Beneficios:** Mejora la confianza de clientes y socios, reduce riesgos de sanciones y refuerza la seguridad de la información.

5.4 Protección de Datos Personales, Ley 29733

Fue promulgada en Perú el 21 de junio de 2011 y publicada el 3 de julio del mismo año. Su objetivo principal es garantizar el derecho fundamental a la protección de los datos personales, conforme al artículo 2, numeral 6, de la Constitución Política del Perú.

Puntos clave de la Ley 29733:

- **Principios rectores:** Establece normas para el tratamiento adecuado de los datos personales.
- **Derechos del titular:** Los ciudadanos tienen derecho a acceder, rectificar, cancelar y oponerse al uso de sus datos.
- **Obligaciones de quienes manejan datos**: Tanto los titulares como los encargados de bancos de datos personales deben cumplir con medidas de seguridad y confidencialidad.
- **Autoridad Nacional de Protección de Datos Personales:** Se encarga de supervisar el cumplimiento de la ley.
- **Sanciones**: Se establecen infracciones y sanciones administrativas para quienes incumplan la normativa.

La Ley 29733 reconoce varios derechos fundamentales para los titulares de datos personales en Perú. Aquí se muestran los más importantes:

- **Derecho de información:** El titular tiene derecho a conocer quién recopila sus datos, con qué propósito y cómo serán tratados.
- **Derecho de acceso:** Puede solicitar información sobre sus datos almacenados en cualquier banco de datos personales.
- **Derecho de rectificación:** Si sus datos son incorrectos, incompletos o desactualizados, puede pedir que sean corregidos.

- **Derecho de cancelación:** Puede solicitar la eliminación de sus datos cuando ya no sean necesarios o cuando se haya vulnerado la normativa.
- **Derecho de oposición**: Puede negarse al tratamiento de sus datos en ciertos casos, especialmente si afectan sus derechos fundamentales.
- **Derecho a la revocación del consentimiento**: Si previamente autorizó el uso de sus datos, puede retirar su consentimiento en cualquier momento.

Estos derechos buscan garantizar la privacidad y el control sobre la información personal.

La Ley 29733 establece varias obligaciones para quienes manejan datos personales en Perú y estas son:

- **Consentimiento informado**: Antes de recopilar datos personales, deben obtener el consentimiento del titular, salvo excepciones legales.
- **Finalidad específica:** Los datos solo pueden ser usados para el propósito declarado al momento de la recolección.
- **Seguridad y confidencialidad:** Deben implementar medidas de seguridad para proteger los datos contra accesos no autorizados, pérdida o alteración.
- **Acceso y actualización:** Deben permitir que los titulares accedan a sus datos y los actualicen cuando sea necesario.
- **Eliminación de datos:** Si los datos ya no son necesarios o el titular lo solicita, deben ser eliminados conforme a la normativa.
- **Registro de bancos de datos:** En algunos casos, deben inscribir sus bases de datos en el Registro Nacional de Protección de Datos Personales.

En la solicitud del DNI en Perú, el documento contiene datos personales como nombre, dirección y fotografía, etc. Que resultan ser de alta sensibilidad, por ende, la aplicación de la ley debería cumplir con lo siguiente:

- **Consentimiento y finalidad:** La información personal se recopila con el propósito exclusivo de identificación ciudadana y no puede ser usada para otros fines sin autorización.
- **Protección de datos:** El RENIEC (Registro Nacional de Identificación y Estado Civil) debe garantizar la seguridad y confidencialidad de los datos personales.
- **Derecho de acceso y rectificación**: Los ciudadanos pueden solicitar acceso a sus datos y pedir correcciones si hay errores en su información.
- **Uso restringido:** Los datos del DNI no pueden ser compartidos con terceros sin consentimiento, salvo excepciones legales.

6 Estándar de Desarrollo Seguro

6.1 Principios de Desarrollo Seguro

Los principios que guían este estándar incluyen, pero no se limitan a:

- **Validación y sanitización de entradas**: Evitar inyecciones, ataques XSS, CSRF, etc.
- **Control de acceso basado en roles (RBAC) y principio de mínimo privilegio**.
- **Gestión segura de secretos y credenciales**.
- **Registro y monitoreo de eventos de seguridad**.
- **Manejo adecuado de errores y excepciones**.
- **Actualización periódica de dependencias y parches de seguridad**.
- **Pruebas de seguridad automatizadas y manuales**: SAST, DAST, IAST, pentesting.
- **Passkey:** Forma de autenticación digital que permite a los usuarios iniciar sesión en sitios web, aplicaciones y servicios sin necesidad de contraseñas, utilizando en su lugar credenciales criptográficas seguras vinculadas al dispositivo del usuario.

6.2 Requisitos de Desarrollo Seguro

El desarrollo seguro de software es una práctica fundamental para garantizar la integridad, confidencialidad y disponibilidad de los sistemas y datos gestionados por una aplicación. A medida que las amenazas cibernéticas se vuelven más sofisticadas y frecuentes, es imperativo que las organizaciones incorporen principios de seguridad desde las primeras fases del ciclo de vida del software (SDLC), siguiendo el enfoque "Security by Design".

Este apartado establece una serie de requisitos técnicos y organizacionales clave que deben adoptarse para reducir la superficie de ataque, prevenir vulnerabilidades comunes como inyecciones, accesos no autorizados, fuga de información y otras formas de explotación. Estos requisitos cubren aspectos como la validación de entradas, autenticación y autorización robustas, gestión de contraseñas, uso de criptografía fuerte, manejo de errores, registros y monitoreo, así como una configuración segura por defecto.

Implementar estos controles no solo mejora la seguridad de las aplicaciones, sino que también contribuye al cumplimiento de estándares y normativas legales vigentes, promoviendo la confianza de los usuarios y la sostenibilidad de las soluciones en entornos críticos como el sector público, financiero o de salud.

6.2.1 Validación y sanitización de entradas

Validar del lado del cliente y del servidor:

- La validación en el cliente mejora la experiencia del usuario, pero no debe considerarse una medida de seguridad.
- La validación en el servidor es obligatoria para proteger la lógica de negocio y la integridad de los datos.

Validaciones comunes incluyen:

- Tipo de dato, longitud, formato y rango permitido.
- Listas blancas para campos críticos.

Prevenir ataques como inyección SQL, XSS y comandos del sistema:

- **SQL Injection:** Usar parámetros con prepared statements o usar ORMs seguros para evitar concatenación de strings SQL.
- **Command Injection:** Evitar ejecutar comandos del sistema con entradas no controladas. Validar exhaustivamente cualquier argumento y usar funciones seguras como execFile (en Node.js) o subprocess.run con listas de argumentos (en Python).
- **Usar ORM para acceso a base de datos:** Un ORM reduce el riesgo de inyecciones al gestionar la interacción con la base de datos mediante objetos. Un ORM reduce el riesgo de inyecciones SQL al gestionar la interacción con la base de datos mediante objetos y consultas parametrizadas, añadiendo una capa de seguridad.

Aplicación en Arquitecturas Modernas (Microservicios):

En una arquitectura de microservicios, el uso de procedimientos almacenados evoluciona respecto al patrón legacy. Ya no se utilizan para contener lógica de negocio central, sino como mecanismos de seguridad y rendimiento para operaciones específicas de datos:

- **Contratos de Datos Seguros:** Actúan como la única interfaz permitida para operaciones críticas (ej: sp_RegistrarCiudadano). El microservicio invoca el procedimiento, pero no tiene permisos de escritura directos sobre las tablas, previniendo así modificaciones no controladas y ataques de inyección SQL.
- **Operaciones Atómicas Complejas:** Son ideales para transacciones que requieren múltiples pasos a nivel de base de datos (ej: actualizaciones masivas, cálculos

agregados), donde su ejecución dentro del motor de base de datos es significativamente más eficiente.
- **Aislamiento y Eficiencia:** El microservicio mantiene su lógica de negocio y autonomía. Los procedimientos se utilizan de forma estratégica para descargar operaciones intensivas de datos, mejorando el rendimiento sin acoplar la lógica de negocio a la base de datos.

6.2.2 Registro y monitoreo

Registro de eventos relevantes

Incluir eventos como:

- Inicios y cierres de sesión.
- Cambios en configuración o privilegios.
- Errores del sistema y excepciones no controladas.
- Intentos de acceso fallidos o actividades sospechosas.

Asimismo, se debe garantizar la integridad de los registros, mediante mecanismos como firmas digitales, almacenamiento inmutable (WORM) o técnicas de auditoría forense, con el fin de asegurar que los logs no sean alterados y puedan ser utilizados como evidencia confiable.

Monitoreo de integridad y alertas:

Se debe implementar una solución de monitoreo continuo del estado e integridad de los sistemas, con capacidad para generar alertas automáticas ante eventos anómalos, posibles brechas de seguridad o comportamientos no autorizados.

La arquitectura de monitoreo debe incluir funcionalidades como:

- Supervisión activa de infraestructura, servicios y aplicaciones, mediante métricas, registros y eventos.
- Detección de alteraciones no autorizadas en archivos, configuraciones críticas o componentes del sistema.
- Correlación de eventos y generación de alertas en tiempo real, que permitan una respuesta temprana ante incidentes de seguridad o fallas operativas.

Las herramientas seleccionadas deben permitir una integración flexible con sistemas existentes, soporte para múltiples fuentes de datos, definición de umbrales personalizados y mecanismos de notificación multicanal (correo, mensajería, dashboards, entre otros).

6.2.3 Manejo seguro de errores

No exponer trazas de error al usuario final, evitar mostrar información sensible como rutas, nombres de variables o estructuras de base de datos. Mostrar mensajes genéricos, registrar el detalle internamente.

Capturar y gestionar excepciones adecuadamente:

- Uso de try-catch o controladores globales de errores.
- Clasificar errores (funcionales, técnicos, seguridad).
- Fallbacks seguros ante fallos inesperados.

6.2.4 Configuración segura por defecto

Deshabilitar servicios innecesarios:
Minimizar la superficie de ataque desactivando módulos, puertos, APIs o plugins no requeridos.

Uso de entornos separados:

- Aislar desarrollo, pruebas y producción con diferentes configuraciones y credenciales.
- Aplicar control de acceso según entorno.

No usar configuraciones de fábrica:

- Cambiar usuarios, claves y configuraciones por defecto (especialmente en bases de datos, routers y aplicaciones).
- Eliminar código y archivos temporales no usados antes del despliegue.

6.3 Principios de Seguridad en el Ciclo de Vida del SIIRC

Los principios de seguridad constituyen un conjunto de buenas prácticas orientadas a proteger los sistemas informáticos contra amenazas internas y externas. Su aplicación rigurosa durante el ciclo de vida del desarrollo de software garantiza la confidencialidad, integridad y disponibilidad de la información, así como la protección de los activos digitales frente a accesos no autorizados y ataques maliciosos.

6.3.1 Seguridad por diseño

- **Aplicar el principio de "Privacidad desde el diseño" (Privacy by Design):** La privacidad y protección de datos deben ser consideradas desde la concepción del sistema, no como un añadido posterior. Este principio implica incorporar mecanismos que aseguren que los datos personales sean recolectados, procesados, almacenados y eliminados de manera segura, con consentimiento explícito, y cumpliendo marcos regulatorios como el GDPR o leyes locales de protección de datos.
- **Incluir requisitos de seguridad desde la fase de análisis y diseño del sistema:** Es fundamental que los requisitos de seguridad no se incorporen de forma reactiva, sino que se integren desde las etapas iniciales del proyecto. Esto incluye definir políticas de acceso, requerimientos de cifrado, validación de datos y controles de auditoría como parte de la especificación técnica y funcional del sistema.
- **Identificar amenazas mediante análisis de riesgo y modelado de amenazas (por ejemplo, STRIDE o DREAD):**
  Aplicar metodologías estructuradas para la identificación de riesgos y amenazas potenciales.
- **STRIDE** Analiza amenazas desde seis categorías: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service y Elevation of Privilege.
- **DREAD** permite cuantificar el riesgo asignando puntuaciones a Damage, Reproducibility, Exploitability, Affected Users y Discoverability.
- Estos modelos ayudan a priorizar mitigaciones y diseñar controles efectivos para minimizar vulnerabilidades desde el inicio del desarrollo.

6.3.2 Principio de menor privilegio

Todo componente del sistema debe operar con el nivel mínimo de privilegios necesario para el cumplimiento de sus funciones. Esto implica que procesos, servicios, usuarios o cualquier otro elemento del sistema solo deben contar con los permisos estrictamente requeridos para su funcionamiento, lo cual reduce significativamente el riesgo ante posibles vulnerabilidades o accesos no autorizados.

Por ejemplo, una cuenta de acceso a la base de datos utilizada por una aplicación web no debe poseer privilegios de administración si únicamente requiere permisos de lectura.

Las credenciales, tales como contraseñas, claves de acceso, tokens u otros secretos, no deben estar embebidas en el código fuente ni almacenarse en texto plano. Estos elementos deben ser gestionados y resguardados mediante mecanismos seguros de gestión de secretos, que contemplen almacenamiento cifrado, control de acceso basado en roles y trazabilidad de accesos.

Adicionalmente, se deben aplicar prácticas como configuración externa segura, inyección de dependencias controlada y encriptación de datos en reposo, para garantizar la integridad y confidencialidad de los secretos y configuraciones sensibles.

El código fuente debe ser sometido de manera regular a análisis de exposición de secretos mediante herramientas especializadas de escaneo, con el fin de detectar posibles fugas accidentales en los repositorios de desarrollo.

6.3.3 Seguridad en capas (defensa en profundidad)

La estrategia de defensa en profundidad implica proteger el sistema mediante un conjunto de controles distribuidos en diferentes niveles, de modo que, si una capa falla, otra pueda contener la amenaza. Las principales capas incluyen:

- **Validación de entradas:** para prevenir ataques como *inyección SQL, XSS y CSRF*. Toda entrada del usuario debe ser validada, normalizada y sanitizada.
- **Autenticación:** uso de mecanismos robustos como *OAuth2, OpenID Connect, MFA* (autenticación multifactor) y estándares seguros para la gestión de sesiones.
- **Autorización:** control estricto de acceso a recursos según roles, permisos y políticas definidas. Implementar principios como *RBAC* (Role-Based Access Control) o *ABAC* (Attribute-Based Access Control).
- **Encriptación:** cifrado de datos en tránsito (TLS/SSL) y en reposo (AES-256, por ejemplo). Además, se deben aplicar buenas prácticas en el manejo de claves criptográficas.
- **Registros de auditoría (logging):** generación y preservación segura de logs detallados sobre accesos, errores y acciones sensibles, con mecanismos para detectar y responder a eventos sospechosos.
- **Monitoreo y detección de intrusos:** implementación de sistemas de detección (IDS), prevención de intrusos (IPS), y análisis continuo de eventos de seguridad.

6.4 Controles Organizacionales Complementarios

Capacitación continua:
Se debe promover la formación periódica del personal técnico y funcional en temas clave como el OWASP Top 10, prácticas de ingeniería de software segura, gestión de incidentes y cumplimiento normativo y legal en materia de seguridad de la información.

Revisión y auditoría de código:

- **Manual:** Implementar revisiones de código entre pares (peer review) para identificar defectos de diseño, lógica insegura y malas prácticas.
- **Automatizada:** Emplear herramientas de análisis estático de código que permitan detectar automáticamente vulnerabilidades, errores de seguridad y violaciones de estándares de codificación.

Pruebas de seguridad:

- **Escaneo de vulnerabilidades:** Realizar análisis estático y dinámico con herramientas especializadas, siguiendo las metodologías definidas por OWASP, para identificar posibles fallos explotables en entornos de prueba o preproducción. Esto aplica tanto a componentes analizados mediante análisis estático de seguridad (SAST - Static Application Security Testing) como a aquellos evaluados con análisis dinámico de seguridad (DAST - Dynamic Application Security Testing).
- **Pentesting (pruebas de penetración):** Ejecutar pruebas controladas, internas o por parte de terceros especializados, para evaluar la resiliencia del sistema ante ataques reales, identificando debilidades técnicas y organizacionales. Se recomienda realizar mínimo cada 6 meses.

6.5 Puntos Claves para el Desarrollo

Para fortalecer el desarrollo seguro, se recomienda el uso de herramientas especializadas en cada etapa del ciclo de vida del software:

Tabla 6 Puntos claves para el desarrollo

Categoría	Herramientas
Gestión de código	Sistemas de control de versiones distribuidos que permitan trazabilidad, control de cambios, integración con pipelines y colaboración entre equipos.
Análisis de seguridad	Herramientas de análisis estático y dinámico del código fuente para identificar vulnerabilidades de seguridad durante el desarrollo.
Gestión de vulnerabilidades	Plataformas de escaneo de infraestructura y aplicaciones para identificar, clasificar y gestionar vulnerabilidades conocidas en componentes desplegados.
CI/CD con DevSecOps	Integraciones automatizadas que incorporan prácticas de seguridad desde el pipeline de entrega continua, incluyendo validaciones, escaneos y revisiones.
Seguridad de dependencias	OWASP y Herramientas que verifican componentes externos (librerías, paquetes, módulos) en busca de vulnerabilidades o versiones inseguras durante el ciclo de build.
Gestión de secretos	Soluciones para almacenar, cifrar y acceder de forma segura a credenciales, claves API y otros secretos sensibles de configuración mediante control de acceso.

6.6 DevSecOps + Seguridad (DevSecOps)

El presente apartado establece el estándar de **DevSecOps y Seguridad (DevSecOps)** para el ciclo de vida de desarrollo de software (SDLC) del Sistema Integrado de Identificación y Registro Civil (SIIRC). La adopción de este marco representa un pilar estratégico para la modernización de RENIEC, fusionando las prácticas de desarrollo ágil (Dev), operaciones de TI (Ops) y seguridad (Sec) en un proceso unificado y automatizado. El objetivo es construir una capacidad institucional para entregar software de alta calidad de manera rápida, frecuente y, sobre todo, segura.

En el contexto de una entidad de misión crítica como RENIEC, donde la confidencialidad, integridad y disponibilidad de los datos de identidad nacional son primordiales, el enfoque tradicional de aplicar la seguridad como un control reactivo al final del ciclo de desarrollo es insuficiente y riesgoso. Por ello, este estándar formaliza la implementación de **DevSecOps**, una filosofía donde la seguridad es una responsabilidad compartida e integrada desde el inicio. A través de la automatización y la práctica de "Seguridad desde la Izquierda" (Shift-Left Security), se busca identificar y mitigar vulnerabilidades en las fases más tempranas, cuando el costo de remediación es significativamente menor.

Este documento no solo define los controles de seguridad, sino que también estandariza las prácticas DevSecOps que los sustentan, como la integración continua, la entrega continua (CI/CD), la infraestructura como código (IaC) y las estrategias de despliegue progresivo. La adopción de este marco es indispensable para acelerar la entrega de valor a la ciudadanía, fortalecer la resiliencia de los sistemas y garantizar el cumplimiento normativo sin comprometer la postura de seguridad de la institución.

6.6.1 Alcance del Estándar DevSecOps

- **Automatizar los Controles de Seguridad:** Integrar herramientas de seguridad directamente en el pipeline de Integración Continua y Entrega Continua (CI/CD) para una validación automática y temprana.
- **Reducir la Superficie de Ataque:** Detectar y remediar vulnerabilidades en las fases más tempranas del desarrollo, cuando el costo de corrección es menor.
- **Acelerar la Entrega Segura:** Permitir que los equipos de desarrollo entreguen funcionalidades de manera rápida y frecuente, con la confianza de que se cumplen los umbrales de seguridad definidos.
- **Fomentar una Cultura de Seguridad Compartida:** Establecer que la seguridad es una responsabilidad de todos los roles involucrados (desarrollo, operaciones, seguridad), no únicamente de un equipo especializado.

6.6.2 Componentes de Apoyo del Ecosistema DevSecOps

Un pipeline DevSecOps eficaz se apoya en un conjunto de herramientas y plataformas que deben estar estandarizadas para garantizar la coherencia, seguridad y trazabilidad del proceso.

Repositorio de Control de Versiones:

- **Función:** Es la fuente única de verdad (Single Source of Truth) para todo el código fuente y las configuraciones declarativas (como la Infraestructura como Código). Debe utilizar un sistema de control de versiones distribuido.
- **Estándar:** Se debe seguir una estrategia de ramificación definida para gestionar el ciclo de vida del código, con políticas de protección de ramas (main, develop) que exijan revisiones por pares (Pull/Merge Requests) antes de la integración.

Servidor de Integración Continua (CI):

- **Función:** Es el motor que orquesta la ejecución del pipeline. Se activa automáticamente ante cambios en el repositorio de código para compilar, probar y empaquetar la aplicación.
- **Estándar:** Debe operar con agentes de construcción efímeros o aislados para garantizar que cada ejecución sea limpia y reproducible, evitando la contaminación entre builds.

Repositorio de Artefactos:

- **Función:** Es un almacén centralizado y versionado para los artefactos binarios generados por el pipeline de CI. No se deben almacenar artefactos en el repositorio de código fuente.

- **Estándar:** El repositorio debe soportar múltiples formatos de paquetes, integrarse con herramientas de escaneo de seguridad (SCA) para analizar los artefactos almacenados, y gestionar políticas de limpieza para artefactos antiguos o inseguros. Actúa como la fuente de verdad para los despliegues

Gestor de Secretos:

- **Función:** Es una bóveda centralizada y segura para almacenar, gestionar y acceder a secretos como contraseñas, claves de API, tokens y certificados.
- **Estándar:** El pipeline y las aplicaciones deben obtener los secretos de este gestor en tiempo de ejecución. Queda estrictamente prohibido almacenar secretos en texto plano en repositorios de código, archivos de configuración o variables de entorno del servidor de CI.

6.6.3 El Pipeline DevSecOps: Fases, Prácticas y Controles

El pipeline DevSecOps descrito en esta sección aplica de forma transversal a todos los ambientes mínimos establecidos (QA, PRE y PROD). Sin embargo, cada etapa del flujo incorpora controles y validaciones diferenciadas según el nivel de criticidad del ambiente.

Dentro del pipeline, los ambientes se abordan de manera secuencial y controlada: QA funciona como entorno de validación inicial con pruebas estáticas y dinámicas obligatorias; PRE actúa como entorno previo a producción donde se ejecutan validaciones reforzadas, revisión de compliance y pruebas de seguridad avanzadas; y PROD recibe únicamente artefactos firmados y aprobados, aplicando controles estrictos de despliegue, auditoría y monitoreo continuo.

La explicación del flujo detalla cómo cada fase (desde el commit hasta el despliegue) aplica políticas de seguridad y gobernanza adaptadas al ambiente correspondiente, garantizando una cadena de entrega consistente, segura y trazable en todos los niveles.

El pipeline de CI/CD es el corazón de la estrategia DevSecOps. Cada fase debe incorporar tanto prácticas DevSecOps estandarizadas como puertas de seguridad (Security Gates) automatizadas.

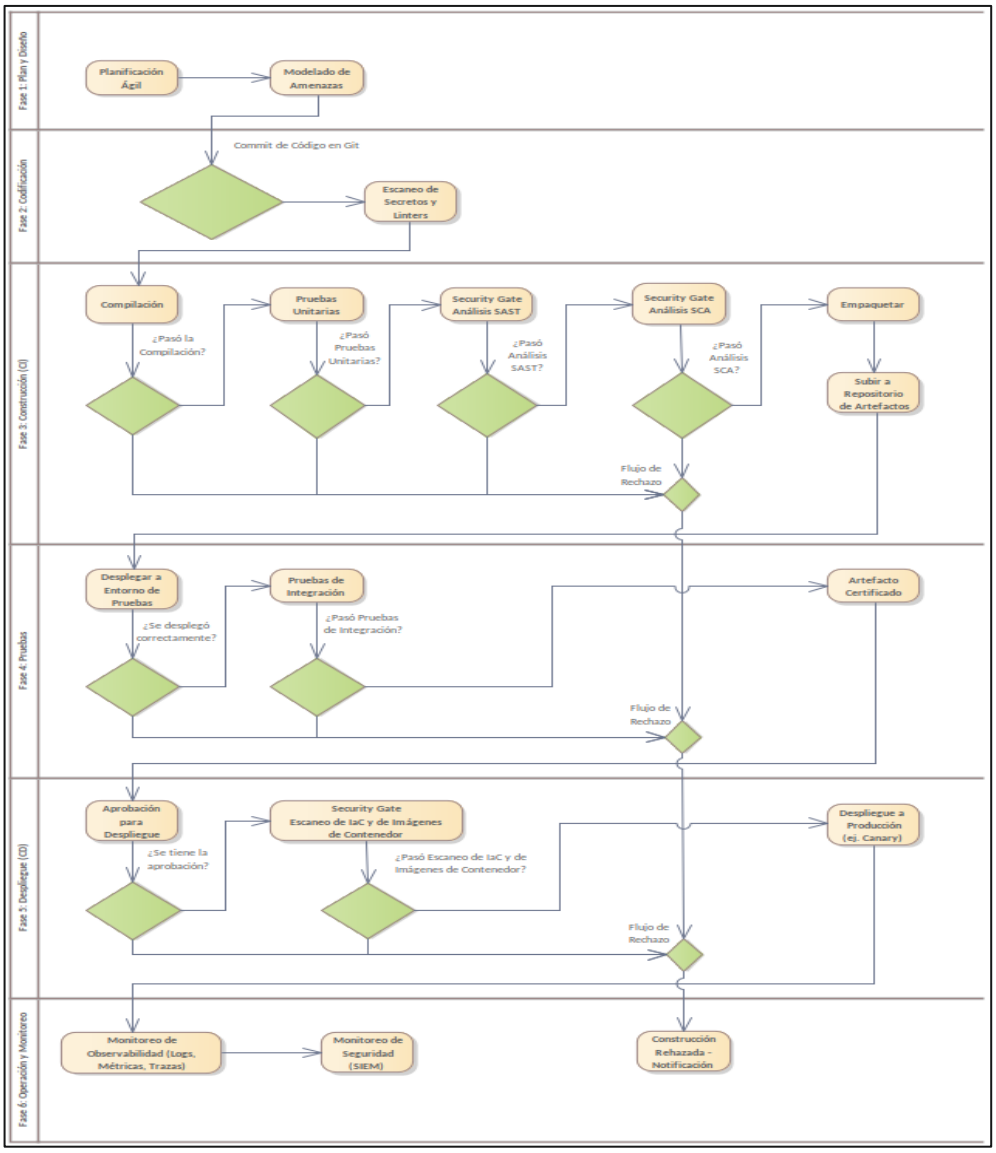


Ilustración 1 Diagrama de flujo que ilustra el pipeline de CI/CD con las fases, prácticas DevSecOps y las puertas de seguridad integradas; ubicado en la herramienta Enterprise Architect en la ruta: Model / 4. Arquitectura Tecnología / 4.2 Servicio de tecnología / Diagramas / UML – Pipeline DevSecOps / UML – Pipeline DevSecOps.

Fase de Planificación y Diseño (Plan & Design):

- **Práctica DevSecOps :** Adopción de metodologías ágiles (Scrum/Kanban) para la gestión de requisitos.
- **Control de Seguridad (Sec):** Modelado de Amenazas (Threat Modeling) utilizando metodologías como STRIDE para identificar vectores de ataque en la arquitectura.

Fase de Codificación (Code):

- **Práctica DevSecOps :** Uso de un sistema de control de versiones distribuido con una estrategia de ramificación estandarizada.
- **Control de Seguridad (Sec):** Escaneo de Secretos y Linters de Seguridad integrados en el entorno de desarrollo local y en el repositorio para impedir la subida de credenciales.

Fase de Construcción (Build):

- **Práctica DevSecOps :** El Servidor de CI compila el código, ejecuta pruebas unitarias y empaqueta la aplicación. El artefacto resultante, versionado semánticamente, es subido al Repositorio de Artefactos.
- **Control de Seguridad 1 (Sec):** Análisis Estático de Seguridad (SAST) del código fuente.
- **Control de Seguridad 2 (Sec):** Análisis de Composición de Software (SCA) de las dependencias.

Fase de Pruebas (Test):

- **Práctica DevSecOps :** El pipeline descarga el artefacto desde el Repositorio de Artefactos y lo despliega en un entorno de pruebas para la ejecución de pruebas de integración automatizadas.
- **Control de Seguridad (Sec):** Análisis Dinámico de Seguridad (DAST) de la aplicación en ejecución.

Fase de Despliegue (Deploy/Release):

- **Práctica DevSecOps :** Uso de Infraestructura como Código (IaC) para provisionar entornos y Gestión de Configuración para aplicar configuraciones. El pipeline orquesta el despliegue del artefacto (obtenido del repositorio) al entorno de producción.
- **Control de Seguridad 1 (Sec):** Escaneo de la configuración de IaC para detectar políticas inseguras.
- **Control de Seguridad 2 (Sec):** Si se usan contenedores, se realiza el escaneo de seguridad de las imágenes antes del despliegue.

Fase de Operación y Monitoreo (Operate & Monitor):

- **Práctica DevSecOps :** Monitoreo continuo mediante una plataforma de observabilidad (logs, métricas, trazas) y configuración de mecanismos de rollback automático.
- **Control de Seguridad (Sec):** Monitoreo Continuo de Seguridad mediante la ingesta de datos en un sistema SIEM para la detección y correlación de incidentes.

6.6.4 Estrategias de Despliegue Seguro: Despliegues Canary

El Despliegue Canary es una técnica de liberación de software diseñada para reducir el riesgo inherente a la puesta en producción de nuevas versiones. Esta estrategia consiste en desplegar la nueva versión de la aplicación en un subconjunto controlado de la infraestructura, permitiendo que coexista temporalmente con la versión estable actual.

A diferencia de un despliegue "Big Bang" (donde se actualizan todas las instancias simultáneamente), el modelo Canary permite validar la estabilidad y el rendimiento del nuevo artefacto bajo condiciones de tráfico real, pero limitando el impacto a un porcentaje reducido de usuarios.

Flujo de Ejecución Estandarizado

Para garantizar la integridad del servicio durante la actualización, el proceso debe seguir estrictamente las siguientes fases:

Despliegue en Paralelo (Stage): Se despliega la nueva versión (v2) en el entorno de producción, pero sin recibir tráfico público inicialmente. Esto permite verificar que la aplicación arranca y conecta correctamente con sus dependencias (Bases de datos, servicios externos).

Segmentación de Tráfico (Routing): Mediante el balanceador de carga, se configura una regla de enrutamiento para dirigir un porcentaje minoritario de las peticiones (por ejemplo, el 5% o usuarios internos) hacia la nueva versión (v2), manteniendo el grueso del tráfico en la versión estable (v1).

Validación mediante Observabilidad: Esta es la fase crítica de decisión. Se deben monitorear métricas clave (Tasa de Errores HTTP, Latencia y Saturación de recursos) comparando el comportamiento de la v2 frente a la v1. No se debe aumentar el tráfico hasta confirmar la salud del sistema.

Promoción o Reversión (Rollout/Rollback):

- Éxito: Si las métricas se mantienen dentro de los umbrales de aceptación (SLOs), se incrementa gradualmente el tráfico (20% -> 50% -> 100%) hasta que la nueva versión atiende la totalidad de la demanda.
- Fallo: Si se detectan anomalías, se ejecuta un Rollback inmediato, redirigiendo el 100% del tráfico a la versión estable (v1) y eliminando las instancias de la versión defectuosa.

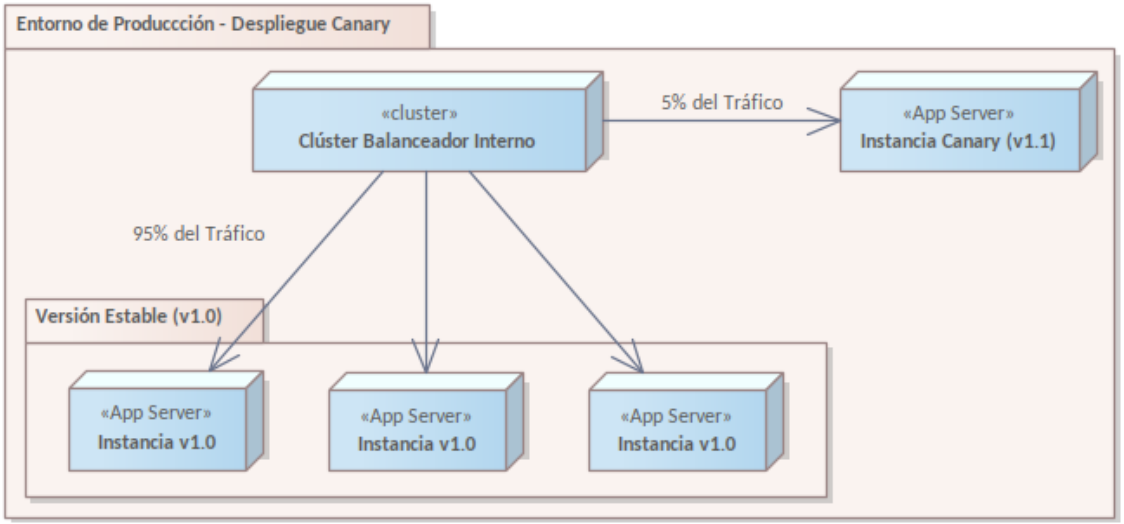


Ilustración 2 Diagrama de despliegue que ilustra la infraestructura con un despliegue Canary, mostrando el balanceador de carga dirigiendo 95% del tráfico a la versión v1 y 5% a la v2; ubicado en la herramienta Enterprise Architect en la ruta: Model / 4. Arquitectura Tecnología /4.2. Servicio de tecnología / Diagramas / UML – Despliegues Canary / UML – Despliegues Canary

Beneficios para RENIEC:

- **Reducción de Riesgo:** Limita el impacto de posibles bugs o problemas de rendimiento.
- **Pruebas en Producción Seguras:** Permite validar el comportamiento con tráfico real.
- **Rollback Instantáneo:** La reversión de un despliegue fallido es rápida y automatizada.

6.6.5 Versionado y CI/CD - Estrategia Integral

Versionado de Código Fuente

Estrategia de Versionado Semántico (SemVer)

MAJOR.MINOR.PATCH

└── Bug fixes (backward compatible)

```
│  └──────── Nuevas funcionalidades (backward compatible)
└──────────────── Breaking changes (NO backward compatible)
```

Ejemplos:

- v1.0.0 → v1.0.1 (patch: fix bug)
- v1.0.1 → v1.1.0 (minor: nueva funcionalidad)
- v1.1.0 → v2.0.0 (major: breaking change)

6.7 Estándares de pseudocódigo

El pseudocódigo es una forma de representar la lógica de un algoritmo utilizando lenguaje natural estructurado. Su objetivo es describir los pasos que debe seguir un sistema para resolver un problema, de manera comprensible para todos los perfiles del equipo, sin preocuparse por la sintaxis específica de un lenguaje de programación. Facilita la validación funcional, la comunicación entre áreas y la posterior implementación técnica.

Propósito en este proyecto

En este proyecto, el pseudocódigo se utilizará para representar la lógica de negocio de cada proceso definido, antes de su implementación técnica. Permitirá validar reglas, condiciones y flujos con todos los actores del equipo, incluidos desarrolladores, analistas, testers y usuarios no técnicos.

Enfoque de construcción

- A partir de cada criterio de aceptación (CA) se generará una función o bloque de pseudocódigo independiente.
- Cada función representará una parte específica del proceso.
- Habrá un artefacto principal (tipo main) que orquestará el flujo general, invocando las funciones individuales de cada CA.
- Todo estará alineado con los requerimientos funcionales y los diagramas BPM.

Estructura recomendada por función

Cada bloque de pseudocódigo debe contener:

- Definición de entradas y salidas.
- Validaciones y condiciones (SI, SINO, MIENTRAS, etc.).
- Acciones o resultados esperados.
- Comentarios breves alineados al criterio de aceptación.

Ejemplo:

```
Funcion ValidarIdentidad(CUI)
   Si identidadIncorrecta
      Retornar "OBSERVADO"
   Sino
      Retornar "IDENTIDAD_CORRECTA"
FinFuncion
```

Buenas prácticas

- Lenguaje claro y conciso: frases comprensibles, evitando tecnicismos innecesarios.
- Bloques organizados: usar sangrías para reflejar la jerarquía del flujo.
- Evitar sintaxis de lenguajes reales: el pseudocódigo debe ser neutral.
- Entradas y salidas bien definidas: al inicio y al final de cada bloque.
- Palabras clave estandarizadas:

Inicio / Fin
Leer, Mostrar
Si / Entonces / Sino / FinSi
Mientras / FinMientras
Para / Hasta / FinPara
Retornar

Comentarios claros: explicar únicamente las partes complejas o que necesitan contexto (ej. // CA008 –Validar duplicidad de trámite).
Una función, una responsabilidad: cada función debe alinearse con un único criterio funcional.

- Responsable de la actualización.

7 Estándar de Autenticación y Autorización

7.1 Autenticación y Autorización

Utilizar estándares como OAuth 2.0, OpenID Connect o SAML:

- **OAuth 2.0:** Ideal para delegar acceso a recursos sin compartir credenciales.
- **OpenID Connect:** Extiende OAuth 2.0 para incluir autenticación de usuarios.
- **SAML:** Recomendado para integraciones empresariales (por ejemplo, **SSO con Active Directory).**
- **Aplicar multifactor de autenticación (2FA):** Añadir una capa adicional de seguridad mediante SMS, TOTP (Aplicaciones de generación de códigos, Aplicaciones autenticadoras compatibles con algoritmos estándar) o autenticadores físicos (U2F, Dispositivos físicos de autenticación compatibles con FIDO/U2F). Fundamental para cuentas con privilegios administrativos.

Implementar control de sesiones seguro (expiración, invalidación, detección de anomalías):

- Cookies HttpOnly, Secure y SameSite.
- Renovación periódica de tokens (rotación).
- Mecanismos para cerrar sesiones desde el servidor.
- Detección de uso simultáneo desde ubicaciones sospechosas (detección de anomalías).

7.2 OpenID Connect

OpenID Connect (OIDC) es un protocolo de autenticación federada basado en el estándar OAuth 2.0, que permite a las aplicaciones verificar la identidad de los usuarios a partir de la autenticación realizada por un proveedor de identidad (IdP). Además, facilita la obtención de información básica del usuario denominada *claims* de manera segura y estandarizada.

Este protocolo es ampliamente adoptado por entidades gubernamentales, corporaciones y plataformas web modernas debido a su seguridad, interoperabilidad y facilidad de integración con diferentes entornos y tecnologías.

OIDC introduce un componente adicional denominado id_token, un JSON Web Token (JWT) que contiene los datos del usuario autenticado (por ejemplo, nombre, correo electrónico o número de documento), firmado digitalmente por el proveedor de identidad para garantizar su autenticidad e integridad.

7.2.1 Diagrama de Flujo de autenticación con OIDC

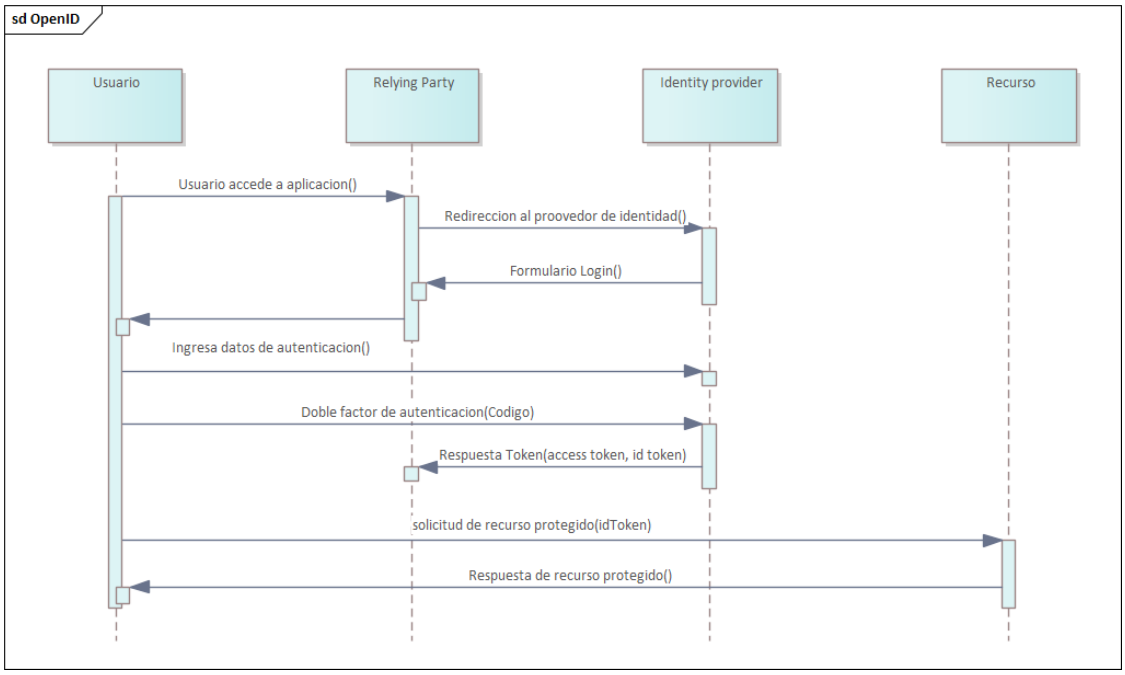


Ilustración 3 Flujo de autenticación OpenID Connect; ubicado en la herramienta Enterprise Architect en la ruta: Model / 5. Arquitectura Seguridad / 5.3. Modelo / Authentication and authorization / OpenID

Los pasos generales del flujo de autenticación son:

- El usuario accede a la aplicación
- La aplicación redirige al usuario al Proveedor de Identidad (IdP) de RENIEC y retorna el formulario de autenticación.
- El usuario ingresa sus credenciales en el IdP.
- El usuario ingresa el código del doble factor de autenticación
- Si son válidas, el IdP redirige al usuario de vuelta a la aplicación con un id_token y un access_token.
- El id_token contiene los datos del usuario autenticado.
- La aplicación usa ese id_token para realizar la solicitud de recurso protegido.

7.2.2 Ventajas de OpenID Connect para SIIRC

Tabla 7 Flujo de autenticación OpenID Connect

Ventaja	Descripción
Seguridad	Tokens firmados digitalmente, prevención de ataques de suplantación.
Interoperabilidad	Compatible con múltiples lenguajes, frameworks y sistemas.
Centralización de identidad	Usuarios autenticados pueden acceder a múltiples módulos.
Soporte para móviles y web	OIDC funciona bien con aplicaciones móviles, SPA y web tradicionales.

Escalabilidad y modularidad	Permite federar identidades y escalar autenticación sin acoplamiento.

7.2.3 Elementos Principales del protocolo OIDC

Tabla 8 Elementos principales OIDC

Elemento	Descripción
id_token	Token JWT que contiene la identidad del usuario autenticado.
access_token	Token que permite acceder a recursos protegidos vía OAuth 2.0.
userinfo	Endpoint del IdP que devuelve información adicional del usuario.
claims	Información sobre el usuario (ej. nombre, email, DNI) incluida en el id_token.
discovery	Mecanismo que permite a las apps obtener automáticamente la configuración del IdP.

7.3 OAuth 2.0

OAuth 2.0 es un protocolo abierto de autorización que permite a las aplicaciones obtener acceso limitado a recursos protegidos en nombre del usuario, sin necesidad de compartir sus credenciales. Este estándar se ha convertido en un pilar fundamental en arquitecturas modernas basadas en APIs y microservicios, al promover la seguridad, la interoperabilidad y la escalabilidad.

El funcionamiento de OAuth 2.0 se centra en el uso de tokens de acceso, los cuales son emitidos por un servidor de autorización (Authorization Server) y consumidos por un servidor de recursos (Resource Server). La aplicación cliente actúa como intermediaria entre el usuario y el sistema, y solo puede acceder a los recursos protegidos cuando ha sido debidamente autorizada.

7.3.1 Flujo Client Credentials (Servicios internos de SIIRC)

Este flujo es utilizado para la comunicación entre componentes sin interacción del usuario.

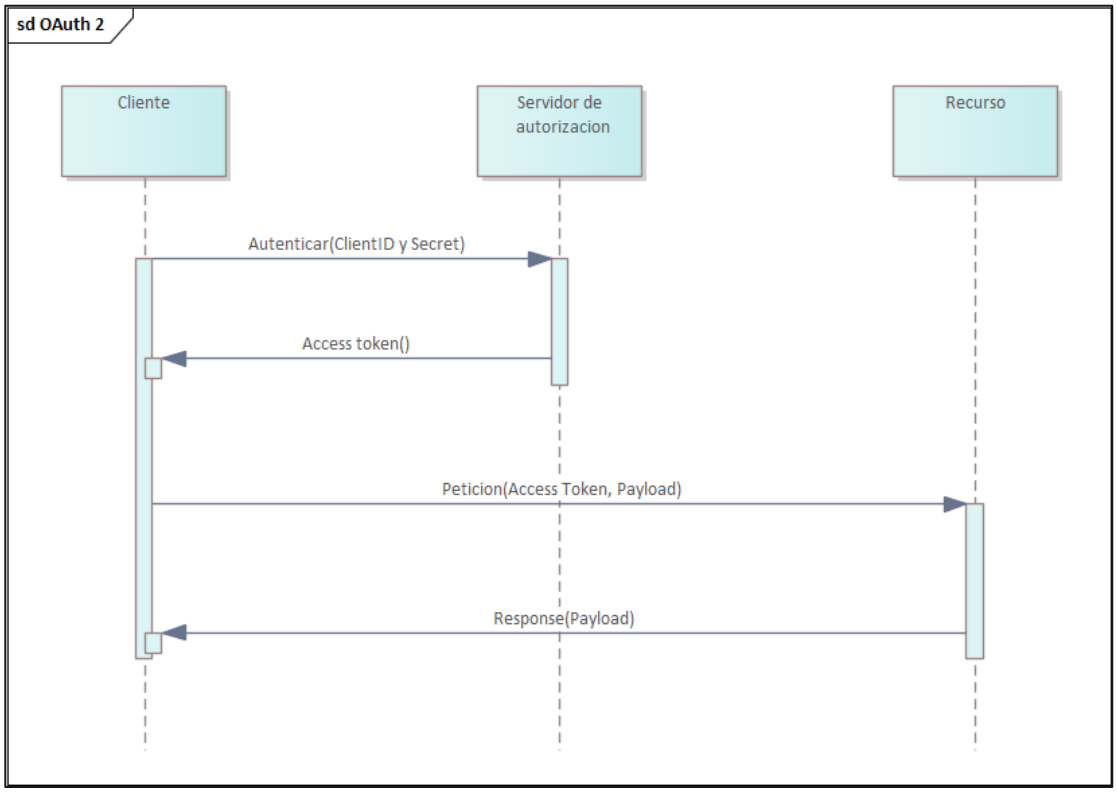


Ilustración 4 Flujo de Autorización servicios internos SIIRC; ubicado en la herramienta Enterprise Architect en la ruta: Model / 5. Arquitectura Seguridad / 5.3. Modelo / Authentication and authorization / OAuth 2.

Los pasos generales del flujo de autorización son:

- La aplicación cliente envía una petición al servidor de autenticación incluyendo las credenciales de aplicación client_id y client_secret.
- Si las credenciales son válidas el servidor de autenticación retorna el access_token.
- La aplicación usa el access_token en las cabeceras HTTP al llamar a APIs del recurso protegido.
- Si el access_token es válido el recurso retorna la información solicitada.

7.3.2 Parámetros y cabeceras utilizados en OAuth 2.0

En los distintos flujos OAuth 2.0, se utilizan parámetros y cabeceras estándar que permiten autenticar y autorizar correctamente a las aplicaciones y usuarios. A continuación, se describen los más importantes.

Tabla 9 Parámetros comunes OAuth2.0

Parámetro	Descripción
client_id	Identificador único de la aplicación cliente.

client_secret	Secreto asignado a la aplicación cliente para autenticar solicitudes.
grant_type	Tipo de flujo OAuth que se está utilizando (ej. authorization_code).
redirect_uri	URL a la que el servidor de autorización enviará la respuesta.
code	Código de autorización temporal que se intercambia por un token.
refresh_token	Token utilizado para obtener un nuevo token de acceso.
username / password	Credenciales del usuario (solo en password grant).
scope	Alcance o permisos solicitados (ej. read:persona write:registro).
state	Parámetro opcional para prevenir ataques CSRF.

Tabla 10 Cabeceras HTTP utilizadas

Cabecera	Descripción
Authorization: Basic	Se usa para enviar el client_id:client_secret codificados en Base64.
Authorization: Bearer	Se usa para enviar el token de acceso al consumir un recurso protegido.
Content-Type: application/x-www-form-urlencoded	Tipo de contenido común para enviar formularios.
Accept: application/json	Solicita una respuesta en formato JSON.

7.4 EIDas

eIDAS es el acrónimo de electronic IDentification, Authentication and trust Services (Identificación electrónica, autenticación y servicios de confianza).

Se refiere al Reglamento (UE) 910/2014 del Parlamento Europeo y del Consejo, que es una normativa de la Unión Europea sobre la identificación electrónica y los servicios de confianza para las transacciones electrónicas en el mercado interior europeo.

En esencia, el Reglamento eIDAS:

- Establece un marco legal común para garantizar la seguridad y la confianza en las interacciones electrónicas transfronterizas entre ciudadanos, empresas y administraciones públicas en la UE.
- Define y regula los "servicios de confianza", como las firmas electrónicas (simple, avanzada y cualificada, siendo esta última legalmente equivalente a una firma

manuscrita), los sellos electrónicos, los sellos de tiempo, los servicios de entrega electrónica certificada y los certificados de autenticación de sitios web.
- Asegura que los sistemas de identificación electrónica reconocidos por un Estado miembro sean aceptados y tengan validez legal en el resto de los Estados miembros.

El eIDAS es el Reglamento europeo para construir un mercado único digital con identificación y firmas electrónicas transfronterizas. El RENIEC es la autoridad peruana que cumple una función similar a nivel nacional, al emitir el DNIe y los certificados digitales que garantizan la identidad y la validez legal de las firmas electrónicas en el Perú.

7.5 Identidad Autosoberana

La Identidad Autosoberana (SSI) es un concepto de gestión de la identidad digital que busca devolver el control total de los datos personales al individuo, eliminando la dependencia de intermediarios centralizados como gobiernos, instituciones financieras o plataformas tecnológicas. En este modelo, el usuario es el propietario, administrador y único responsable de su identidad digital, decidiendo cuándo, cómo y con quién comparte su información.

La SSI se basa en tecnologías descentralizadas, especialmente blockchain, que permiten garantizar la inmutabilidad, trazabilidad y verificabilidad de los datos sin comprometer la privacidad. En lugar de depender de una autoridad emisora tradicional, las credenciales digitales (por ejemplo, un título universitario o una licencia de conducción) se almacenan en una cartera digital (digital wallet) en poder del usuario.

Estas credenciales pueden ser verificadas por terceros mediante identificadores descentralizados (DID) y credenciales verificables (VCs), sin necesidad de acceder directamente a las bases de datos del emisor, garantizando así autenticidad, confidencialidad y no repudio.

Entre las ventajas principales del enfoque SSI se destacan:

- Autonomía y control: el individuo gestiona sus propios datos sin intermediarios.
- Privacidad y seguridad: solo se comparte la información estrictamente necesaria, utilizando técnicas de zero-knowledge proof (prueba de conocimiento cero).
- Interoperabilidad: permite el uso de la identidad digital en distintos sistemas, países y plataformas, gracias a estándares abiertos (W3C).
- Reducción de fraudes: las credenciales verificables son resistentes a manipulaciones o suplantaciones.
- Simplificación de procesos: elimina la necesidad de repetir verificaciones de identidad en múltiples servicios.

7.5.1 Aplicación al SIIRC de RENIEC

Actualmente, el modelo del RENIEC es centralizado (el RENIEC es el emisor y administrador de la identidad), mientras que la SSI es descentralizada.

La aplicación de la SSI al SIIRC representaría una evolución del modelo actual, enfocándose en la distribución del control y la privacidad selectiva.

En resumen, el SIIRC pasaría de ser solo un "dueño de datos" a ser un "Emisor de Credenciales Verificables" de la identidad legal, empoderando a los ciudadanos con el control y la privacidad de su identidad en línea.

8 Estándar de REST Services

Considerando lo critico de los servicios de identificación y registro civil para el estado, se considera de especial consideración la implementación de los microservicios y garantizar la observabilidad, seguridad y cumplimiento normativo del sistema.

Los estándares de REST Services para aplicativos del RENIEC garantizan lo siguiente:

- **La Interoperabilidad y Ecosistema Unificado.** Los servicios requieren comunicación seamless (sin interrupciones) entre múltiples entidades. REST proporciona un protocolo estandarizado que permite que sistemas heterogéneos se comuniquen eficientemente.
- **Escalabilidad Horizontal:** Los microservicios REST permiten escalar componentes específicos según demanda (picos en emisión de documentos, consultas masivas), optimizando recursos y costos operacionales.
- **Mantenibilidad y Evolución:** Cada microservicio puede evolucionar independientemente, crucial para sistemas gubernamentales que requieren actualizaciones continuas sin afectar servicios críticos.
- **Arquitectura en Capas para Microservicios – SIIRC:** Establecer una estructura de capas estandarizada para todos los microservicios del SIIRC que garantice separación de responsabilidades, mantenibilidad y escalabilidad.

La solución se estructura en tres capas principales:

- **Capa de Exposición (API Management Layer):** API Manager como punto único de entrada con gestión centralizada de seguridad, throttling y versionado.
- **Capa de Representación (Microservices Layer)**: Microservicios de negocio con lógica específica de dominio y responsabilidad única.
- **Capa de Integración (Integration Layer):** Event Streaming para comunicación asíncrona y conectores a sistemas legados.

Este patrón garantiza que cada capa del SIIRC mantenga su independencia, exponga solo la información necesaria y pueda evolucionar sin impactar a las demás capas, cumpliendo con los principios de seguridad y separación de responsabilidades requeridos en sistemas de identificación ciudadana.

8.1 Cumplimiento Normativo y Marco Legal Peruano

Ley de Gobierno Digital (Ley N° 31364)

- Interoperabilidad obligatoria: Los estándares REST garantizan el cumplimiento del artículo 12° sobre intercambio de información entre entidades públicas
- Principio de una sola vez: REST permite arquitecturas que evitan solicitar al ciudadano información ya registrada en otras entidades
- Seguridad digital: Implementación de OAuth 2.0 y JWT conforme a los lineamientos de la PCM

Ley de Protección de Datos Personales (Ley N° 29733)

- Trazabilidad completa: Cada request REST debe ser auditado para cumplir con la obligación de registro de tratamiento de datos
- Minimización de datos: Endpoints específicos que solo expongan la información necesaria

8.2 Justificación Estratégica

La adopción de estándares REST para nuevos sistemas del SIIRC representa una inversión crítica en sostenibilidad tecnológica que garantiza:

Continuidad Operacional Sin Riesgos

- Desarrollo paralelo: Los nuevos sistemas REST operarán independientemente de los sistemas legacy SOAP existentes
- Cero interrupciones: Los servicios críticos de identificación ciudadana continuarán funcionando durante toda la transición
- Migración controlada: Cada componente se moderniza según cronograma planificado, sin afectar operaciones diarias

Preparación para Crecimiento Demográfico

- Escalabilidad automática: Los servicios REST pueden multiplicar su capacidad en minutos, no meses
- Atención simultánea: Capacidad para atender múltiples consultas ciudadanas sin degradación del servicio
- Respuesta a demanda: Ajuste dinámico de recursos durante picos de consultas (elecciones, censos, trámites masivos)

Optimización de Recursos Institucionales

- Reducción de costos operativos: Menor consumo de servidores y recursos de red
- Mantenimiento simplificado: Arquitectura más simple reduce horas-hombre de soporte técnico
- Reutilización de infraestructura: Aprovechamiento máximo de la inversión tecnológica actual

Evolución Tecnológica Gradual

- Riesgo controlado: Implementación por fases elimina el riesgo de "big-bang" tecnológico
- Aprendizaje continuo: El equipo técnico adquiere conocimientos sin presión de migración total
- Flexibilidad estratégica: Ajustes del plan según resultados de cada fase

8.3 Métodos HTTP - Especificación Completa

Tabla 11 Métodos HTTP

Método	Descripción	Idempotente	Seguro	Cacheable	Uso en SIIRC	Ejemplo
GET	Recuperar datos	✅	✅	✅	Consulta de personas, documentos	/api/v1/documentos/MsDatosRepositorioDocumental/documento/{id}/descargar
POST	Crear nuevos recursos	❌	❌	❌	Registro de nacimientos, nuevas personas	/api/v1/documentos/MsDatosRepositorioDocumental/documento
PUT	Actualizar/reemplazar recurso completo	✅	❌	❌	Actualización completa de datos personales	/api/v1/documentos/MsDatosRepositorioDocumental/documento/{id}

8.3.1 Implementación Técnica

A continuación, ejemplos de uso de los verbos HTTP

- GET - Consultas y Filtros

```
# Consulta básica
GET /api/v1/registrocivil/ MsDatosSolicitudes


# Consulta con filtros
GET /api/v1/registrocivil/MsDatosSolicitudes/{solicitudId}

# Paginación y ordenamiento
GET /api/v1/registrocivil/MsDatosSolicitudes/{solicitudId}?page=1&size=50&sort=asc
```

- POST - Creación de **Recursos**

```
POST /api/v1/util/MsDatosPlantillas/plantillas
Content-Type: application/json
{
  "nombrePlantilla": "string",
  "descripcion": "string",
  "version": "string",
  "plantillaHtml":"string",
  "idioma": "string"
  "variables": [
  {
    "nombreCampo":"string",
    "requerido":"Boolean",
    "descripcion":"String"
  }
 ]
}
```

8.3.2 Manejo de respuestas y Errores SIIRC

Tabla 12 Resumen por Categorías

Categoría	Rango	Significado	Uso en SIIRC	Ejemplos
Éxito (2xx)	200-299	Operación exitosa	Consultas y operaciones exitosas	200, 201, 202, 204
Redirección (3xx)	300-399	Acción adicional requerida	Migración de endpoints, balanceadores	301, 302, 304
Error Cliente (4xx)	400-499	Error en la petición	Validaciones, autenticación, autorización	400, 401, 403, 404, 409
Error Servidor (5xx)	500-599	Error interno del servidor	Fallos de sistema, mantenimiento	500, 502, 503, 504

Tabla 13 Códigos HTTP para SIIRC

Código	Nombre	Uso Específico en SIIRC	Implementación	Métrica de Éxito
200 OK	Éxito general	Consultas exitosas de personas	GET /api/v1/identificacion/ciudadano/12345678	> 95% de respuestas
201 Created	Recurso creado	Nuevo registro civil	POST /api/v1/registrocivil/acta	< 500ms tiempo respuesta

202 Accepted	Procesamiento asíncrono	Generación de certificados	POST /api/v1/documentos/generardocumento/documento	Queue confirmado
204 No Content	Éxito sin contenido	Actualizaciones exitosas	PUT /api/v1/identificacion/ciudadano/12345678	Confirmación operación
304 Not Modified	Recurso no modificado	Cache de documentos	Headers If-Modified-Since	Reducción ancho banda
400 Bad Request	Datos inválidos	DNI formato incorrecto	Validación de entrada	< 5% de peticiones
401 Unauthorized	Sin autenticación	Token JWT expirado/inválido	Middleware autenticación	100% de accesos no autorizados bloqueados
403 Forbidden	Sin permisos	Acceso datos restringidos	RBAC (Role-Based Access)	Auditoría completa
404 Not Found	Recurso inexistente	Persona no registrada	GET /api/v1/identificacion/ciudadano/00000000	< 10% de consultas
409 Conflict	Conflicto de datos	DNI duplicado en registro	Validación unicidad	0% duplicados permitidos
422 Unprocessable Entity	Error validación semántica	Fecha nacimiento futura	Validaciones de negocio	Validación 100% casos
429 Too Many Requests	Límite de velocidad	Rate limiting por IP/usuario	API Gateway configurado	Protección DDoS
500 Internal Server Error	Error interno	Fallo base de datos	Logs detallados + alertas	< 0.1% de errores
502 Bad Gateway	Error proxy/gateway	Microservicio no disponible	API Gateway + Circuit Breaker	Failover automático
503 Service Unavailable	Servicio no disponible	Mantenimiento programado	Health check endpoints	Comunicación proactiva
504 Gateway Timeout	Timeout de gateway	Operación muy lenta	Timeout configurado	< 30s límite respuesta

8.3.3 Estructura Gestión de Errores

Se propone implementar una estructura de gestión de errores estandarizada que garantice consistencia across todos los microservicios. Esta estructura incorpora un esquema jerárquico que incluye un código de error específico para identificación única, un tipo categorizado para filtrado, y un objeto details que desglosa cada incidencia con identificadores técnicos (issue), descripciones legibles, valores recibidos y restricciones violadas. Adicionalmente, se integran metadatos de contexto como trace_id, path, service y timestamp para facilitar la trazabilidad completa del error en sistemas distribuidos. La estructura se complementa con un flag success para verificación inmediata del estado y enlaces a documentación específica, asegurando una experiencia de debugging eficiente y alineada con estándares RESTful.

8.3.3.1 Esquema Base de Respuesta de Error

Cada error generado debe adoptar una estructura jerárquica con los siguientes elementos obligatorios:

Tabla 14. Estructura base de respuesta de error estandarizada

Elemento	Tipo	Descripción
tipo	string	Identificador único del error bajo un formato estructurado
titulo	string	Categoría funcional del error
detalleError	string	Descripción legible y resumida del error orientada al usuario o consumidor.
estado	integer	indica el código de estado HTTP
details	string	Contiene el desglose técnico de la incidencia, incluyendo campos con errores, restricciones incumplidas y mensajes específicos.

8.3.3.2 Ejemplo de Implementación JSON

```
"error": {
  "tipo": "VALIDATION_ERROR_001",
  "titulo": " Formato inválido",
  "estado": 400,

  "errores": [
    {
      "detalleError": "El campo email no cumple con el formato esperado."
    }
  ]
}
```

8.4 Patrones de Diseño REST para Microservicios

8.4.1 Patrón API Gateway - Justificación Arquitectónica

El patrón API Gateway es un componente fundamental en arquitecturas de microservicios que actúa como punto de entrada único para todas las solicitudes de clientes, proporcionando funcionalidades como enrutamiento, autenticación, limitación de velocidad, y agregación de respuestas.

Arquitectura de Implementación

- API Client: Actúa como el punto de inicio de todas las solicitudes. Es la aplicación o sistema cliente que consume los servicios disponibles, generando las peticiones HTTP/REST que ingresan al ecosistema.
- API Gateway: Funciona como el único punto de entrada y fachada unificada para todos los clientes. Recibe todas las solicitudes del API Client y se encarga de: enrutamiento inicial, autenticación centralizada, limitación de tasa (rate limiting), terminación SSL y agregación de respuestas.
- Router de Versiones: Componente especializado que gestiona el versionado de APIs. Una vez que el API Gateway procesa la solicitud, el Router de Versiones determina qué versión específica del microservicio debe atender la petición, permitiendo coexistencia de múltiples versiones y facilitando actualizaciones sin breaking changes.
- Microservicios V1/V2/V3 con Bases de Datos: Capa de ejecución de negocio compuesta por microservicios independientes y especializados:
  - Cada microservicio (V1, V2, V3) representa un contexto delimitado con lógica de negocio específica.
  - Operan de manera autónoma y pueden ser desarrollados, desplegados y escalados independientemente.

- Cada microservicio gestiona su propia base de datos, aplicando el principio de polyglot persistence donde cada servicio elige la tecnología de almacenamiento más adecuada a sus necesidades.
- Las bases de datos son privadas por servicio, evitando acoplamiento directo y permitiendo esquemas de datos independientes.

Flujo Completo: El API Client envía una solicitud → API Gateway (control de acceso y gestión) → Router de Versiones (direccionamiento por versión) → Microservicio específico (procesamiento de negocio) → Base de datos correspondiente (persistencia especializada).

Beneficios Específicos para SIIRC

- Punto Único de Entrada: Simplifica integraciones para entidades cliente.
- Agregación de Respuestas: Una llamada obtiene datos de múltiples microservicios.
- Transformación Legacy → REST: Mantiene compatibilidad SOAP mientras migra a REST.
- Rate Limiting: Protección automática contra sobrecarga del sistema.

8.4.2 Patrón Saga para Transacciones Distribuidas

El Patrón Saga es una solución para manejar transacciones distribuidas en sistemas de microservicios, donde necesitas coordinar operaciones que abarcan múltiples servicios sin depender de transacciones ACID tradicionales.

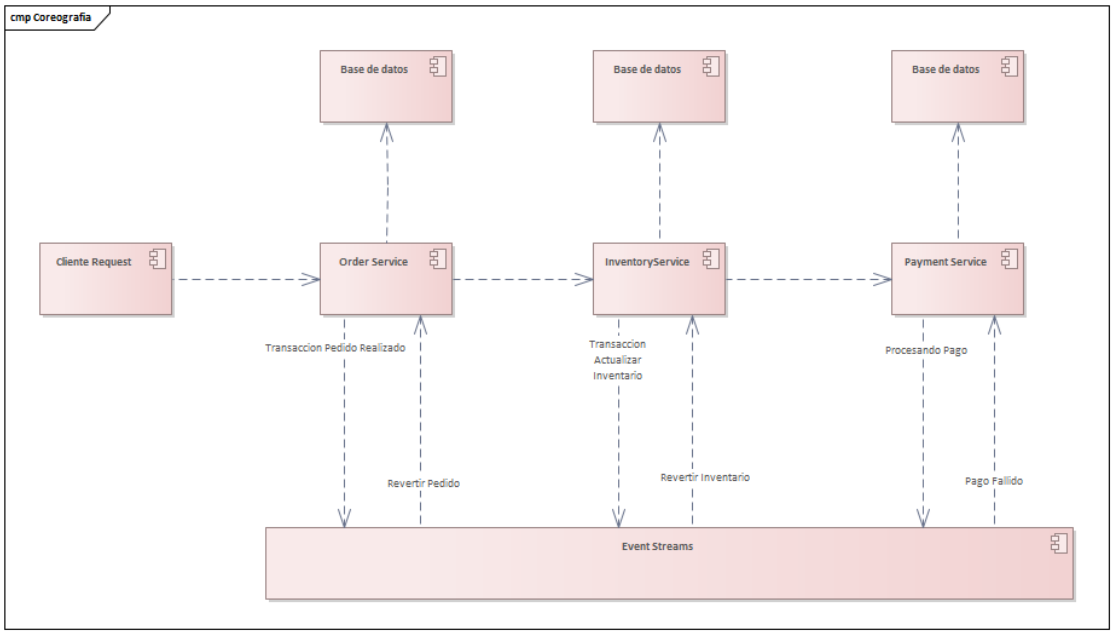


Ilustración 5 Patrón Saga - Coordinación de Transacciones Distribuidas en Microservicios; ubicado en la herramienta Enterprise Architect en la ruta: Model / 2. Arquitectura Aplicaciones / 2.10. Modelo / 2.10.6. Diagramas Documentacion / Coreografia / Coreografia.

Coreografía (Choreography): Los servicios se coordinan mediante eventos

Caso de Uso ejemplo: Registro de Nacimiento

Saga Coreografía - Eventos Distribuidos
1. POST /api/v1/identificacion/ciudadano
   → Response: 201 Created + event "Ciudadano.Creado"

2. Event "Ciudadano.Creado" → POST /api/v1/registrocivil/actas
   → Response: 201 Created + event "Acta.Creada"

3. Event "Acta.Creada" → POST /api/v1/documentos/certificadoNacimiento
   → Response: 201 Created + event "Documento.Generado"

4. Event "Documento.Generado" → PUT /api/v1/identificacion/ciudadano/{id}
   → Response: 200 OK + "Ciudadano.Actualizado"

Beneficios vs Transacciones Monolíticas

- Scalability: Cada paso puede ejecutarse en diferentes servidores
- Resilience: Fallo en un paso no bloquea todo el proceso
- Performance: Procesamiento asíncrono reduce tiempo de respuesta percibido
- Auditability: Cada paso queda registrado para trazabilidad completa

8.5 Principios REST Fundamentales

8.5.1 Stateless

Un servicio stateless es aquel que no guarda información del estado del usuario o de la sesión entre una petición y otra. Cada petición que llega al microservicio debe contener toda la información necesaria para ser procesada, sin depender de datos almacenados en memoria del servidor.

Justificación

- Escalabilidad: Como no hay datos de sesión en memoria, se pueden agregar o quitar instancias del microservicio fácilmente
- Tolerancia a fallos: Si una instancia se cae, otra puede responder la petición sin pérdida de datos de sesión.
- Simplicidad: No hay necesidad de replicar o sincronizar estados entre servidores.
- Facilita el balanceo de carga: Cualquier instancia puede atender cualquier petición sin preocuparse de mantener continuidad.

8.5.2 Uniform Interface

El principio de interfaz uniforme establece que todos los servicios deben comunicarse de una forma estándar y consistente, usando las mismas reglas, estructuras y convenciones para interactuar.

Beneficios Operacionales

- Reutilización de Código: Librerías cliente comunes para todos los servicios
- Testing Automatizado: Frameworks de pruebas reutilizables
- Documentación Automatizada: OpenAPI/Swagger generado automáticamente
- Consistencia: Todos los servicios se usan de la misma forma.
- Descubrimiento fácil: Los clientes pueden navegar las APIs sin documentación extra.
- Independencia: Los servicios pueden evolucionar sin romper la interacción.
- Mantenibilidad: Facilita entender, probar y consumir APIs de distintos equipos.

8.5.3 Resource-Based

El enfoque Resource-Based (Basado en Recursos) se define como un paradigma arquitectónico donde todos los componentes del sistema son modelados como recursos identificables y direccionables mediante URIs. En este modelo, cada recurso representa una entidad conceptual o física dentro del dominio de negocio, y todas las interacciones se realizan a través de operaciones estandarizadas sobre estos recursos. La esencia de este enfoque radica en la representación uniforme de estados y la transferencia de representaciones de recursos entre cliente y servidor, lo que establece una separación clara entre concerns y promueve la escalabilidad del sistema.

Definición de Métodos HTTP en Arquitectura REST

- GET: método para operaciones de consulta y recuperación de recursos, permitiendo acceder a representaciones de datos sin generar efectos secundarios en el servidor. Es idóneo para operaciones de lectura que deben ser seguras e idempotentes.
- POST: Conceptualiza como el mecanismo para la creación de nuevos recursos o la ejecución de procesos complejos que no se ajustan a las operaciones CRUD convencionales. Se utiliza cuando la URL del recurso resultante es desconocida al momento de la solicitud.
- PUT: Define como la operación de actualización de recursos existentes, donde el cliente envía una representación completa del recurso para reemplazar la existente en el servidor. Requiere que el cliente conozca previamente el identificador único del recurso.

8.5.4 Ventajas Arquitectónicas

- Cacheo Predictivo: URLs semánticas permiten estrategias de cache específicas
- Escalabilidad por Recurso: Cada tipo de recurso puede escalar independientemente
- Integración Simplificada: APIs autodescriptivas reducen documentación externa

8.6 Nombrado de microservicios

El estándar de nombramiento para los microservicios del SIIRC sigue una estructura unificada y continúa basada en la convención CamelCase. Esta estructura permite identificar rápidamente la naturaleza, el tipo y la función principal del componente con solo leer su nombre.

Este esquema asegura la consistencia y facilita el mantenimiento y la comprensión de la arquitectura.

8.6.1 Estructura del Nombramiento

Todos los nombres de los microservicios en el SIIRC deben ser una única palabra sin separadores (. o -), siguiendo el siguiente formato concatenado:

{Prefijo} + {Tipo de Microservicio} + {Nombre Funcional}

Toda la cadena resultante debe seguir la convención CamelCase (la primera letra es mayúscula y las palabras subsiguientes, incluyendo el Tipo de microservicio, comienzan con mayúscula).

8.6.2 Componentes del Nombre

Prefijo: Ms

Todos los nombres deben comenzar con el prefijo Ms.

Propósito: Identificar inequívocamente el componente como un Microservicio del SIIRC.

Tipo de Microservicio (Clasificación Funcional)

El segundo segmento clasifica el microservicio según su rol arquitectónico primario. Este clasificador comienza con una letra mayúscula y se concatena al prefijo Ms.

Datos, Interactúa exclusivamente con una Base de Datos dedicada (capa de persistencia CRUD), ej: MsDatosActas

Adaptador, Interactúa y traduce protocolos con sistemas y servicios externos al SIIRC (Interactúa con la API del BCP), ej: MsAdaptadorBcp

Coreog, Orquesta o coordina flujos complejos de negocio llamando a múltiples microservicios (sin usar patrón Saga), ej: MsCoreogTramite

Dominio, Atiende las funcionalidades propias del negocio del SIIRC, ej: MsDominioLogistica

Saga, Gestiona flujos transaccionales distribuidos utilizando el patrón Saga (Coordina la creación de registros con varios microservicios), ej: MsSagaAPD

Nombre Funcional

El tercer segmento, que sigue al Tipo de microservicio, define la funcionalidad específica, debe ser descriptivo, conciso y debe seguir la convención CamelCase.

8.7 Versionado de APIs - Estrategia de Evolución Continua

8.7.1 Estrategia de Versionado por URL Path

Este versionado es fácil de implementar y entender nos da una claridad absoluta sobre qué versión se está usando y facilita el routing y el cache.

Implementación Técnica

```
# Versión actual en producción /api/v1/registrocivil/MsDatosSolicitudes/12345678
# Versión futura en desarrollo /api/v2/registrocivil/MsDatosSolicitudes/{solicitudId}
```

Justificación

- Zero Downtime Migration: Si se realiza alguna migración, este se realizará sin interrupción de servicios críticos
- Risk Mitigation: De necesitarse realizar algun Rollback se puede realizar de inmediato si problemas en nueva versión
- Business Continuity: Los diferentes departamentos oficinas o entidades pueden migrar a su ritmo sin presión operacional
- Investment Protection: El ROI está protegido al mantener sistemas funcionando durante transición

8.7.2 Política de Deprecación Empresarial

Timeline Estratégico

```
Ciclo de Vida de Versiones:  v1_release: "2025-Q2"
# Producción inicial v2_development: "2025-Q3"
# Desarrollo paralelo v2_release: "2025-Q4"
# Release nueva versión v1_deprecation_notice: "2026-Q1"
# Notificación formal v1_sunset: "2027-Q1"
# Fin de soporte v1
```

Headers de Comunicación Proactiva

```
HTTP/1.1 200 OK
API-Version: v1
API-Deprecated: false
API-Sunset: 2027-01-01
```

Beneficios de Comunicación

- Transparencia Operacional: Entidades cliente conocen timeline de cambios
- Planificación de Presupuesto: 12 meses de notice permiten planificación financiera
- Reducción de Soporte: Documentación clara reduce tickets de soporte técnico

8.8 Métricas de Éxito y KPIs Ejecutivos

Tabla 15 SLAs Objetivo para Nuevos Sistemas REST

Métrica	Valor Objetivo	Medición	Impacto de Negocio
Disponibilidad	99.95%	Uptime mensual	Máximo 21.6 min downtime/mes
Latencia P95	< 200ms	Response time	Experiencia usuario óptima
Throughput	5,000+ TPS	Transactions/sec	Soporte picos de demanda
Error Rate	< 0.1%	Failed requests	999 de cada 1000 exitosas
MTTR	< 15 min	Recovery time	Restauración rápida de servicio

9 Estándar de base de datos

Existe un aspecto que no siempre se tiene en cuenta y que suele tener un gran impacto en el rendimiento de cualquier aplicación: el diseño de la base de datos.

A continuación, se analizarán las mejores prácticas de diseño de bases de datos para garantizar que el acceso a los datos no sea un cuello de botella que afecte negativamente al rendimiento de la aplicación.

9.1 Propósito de un buen diseño de la base de datos

Además de mejorar el rendimiento del acceso a los datos, un buen diseño consigue otros beneficios, como mantener la coherencia, precisión y fiabilidad de los datos y reducir el espacio de almacenamiento eliminando redundancias. Otro beneficio de un buen diseño es que la base de datos es más fácil de utilizar y mantener. Cualquiera que tenga que gestionarla sólo tendrá que echar un vistazo al diagrama entidad-relación (ERD) para comprender su estructura.

Los ERD son la herramienta fundamental del diseño de bases de datos. Pueden crearse y visualizarse en tres niveles de diseño: conceptual, lógico y físico.

El diseño conceptual muestra un diagrama muy resumido, con sólo los elementos necesarios para acordar criterios con las partes interesadas del proyecto, que no necesitan comprender los detalles técnicos de la base de datos. El diseño lógico muestra las entidades y sus relaciones en detalle, pero de forma agnóstica a la base de datos.

Por último, el diseño físico es el que añade al ERD todos los detalles necesarios para convertirlo en una base de datos utilizable en un DBMS concreto. A continuación, se presentan las mejores prácticas que hay que tener en cuenta al diseñar una ERD para que la base de datos resultante funcione al máximo.

9.2 Tipo de base de datos a diseñar

Se suelen distinguir dos tipos fundamentales de bases de datos: relacionales y dimensionales.

Las bases de datos relacionales se utilizan para aplicaciones tradicionales que ejecutan transacciones sobre los datos, es decir, obtienen información de la base de datos, la procesan y almacenan los resultados.

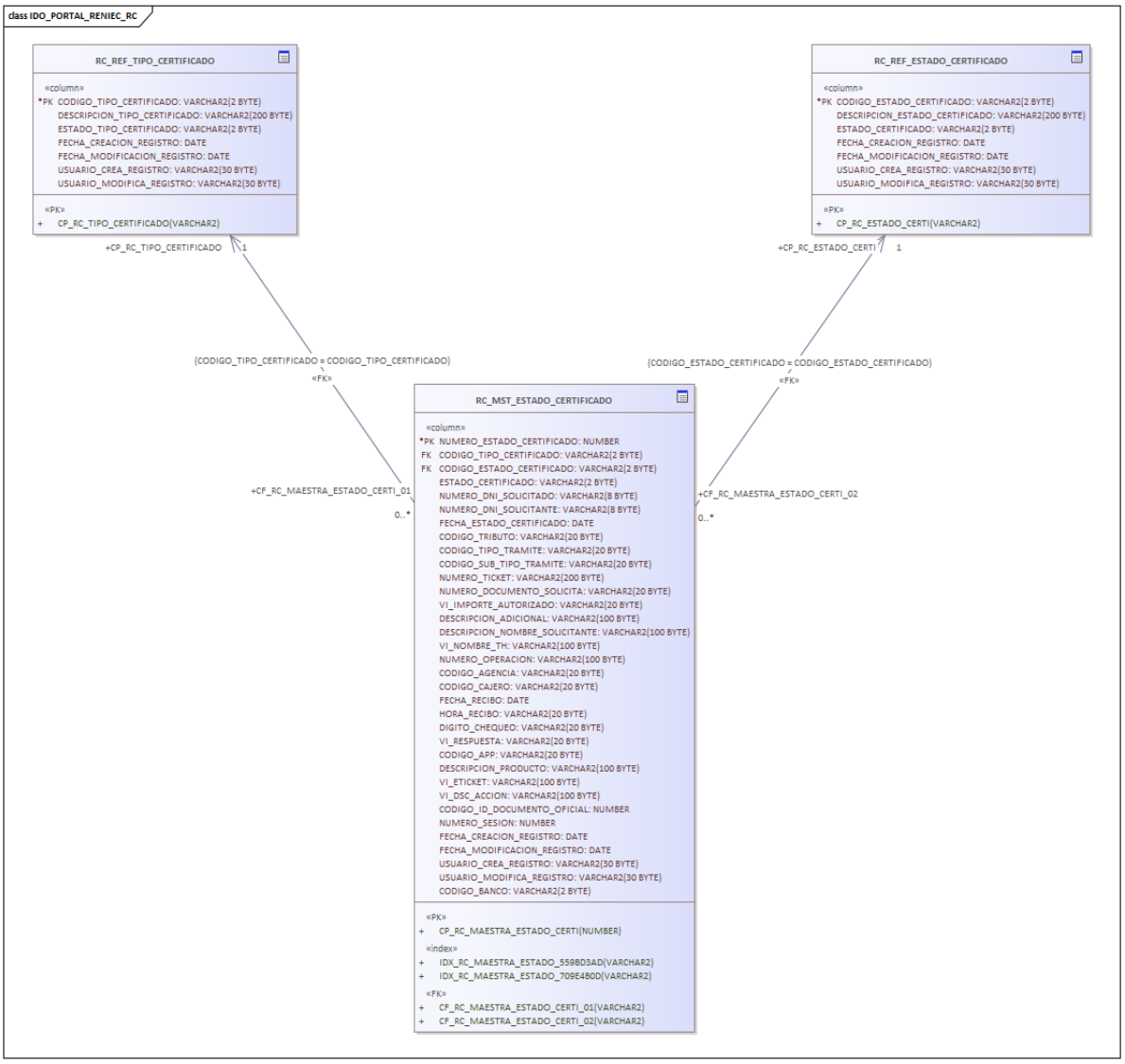


Ilustración 6 Diagrama de Relaciones entre Entidades, ubicado en la herramienta Enterprise Architect en la ruta: Model / 3. Arquitectura Datos/ 3.4. Modelo / 3.4.2. ToBe RENIEC (Aplicaciones) / 3.4.2.2 REGISTRO CIVIL / IDO_PORTAL_RENIEC_RC.

Por otro lado, las bases de datos dimensionales se utilizan para la creación de almacenes de datos: grandes depósitos de información para el análisis de datos y la minería de datos con el fin de obtener perspectivas.

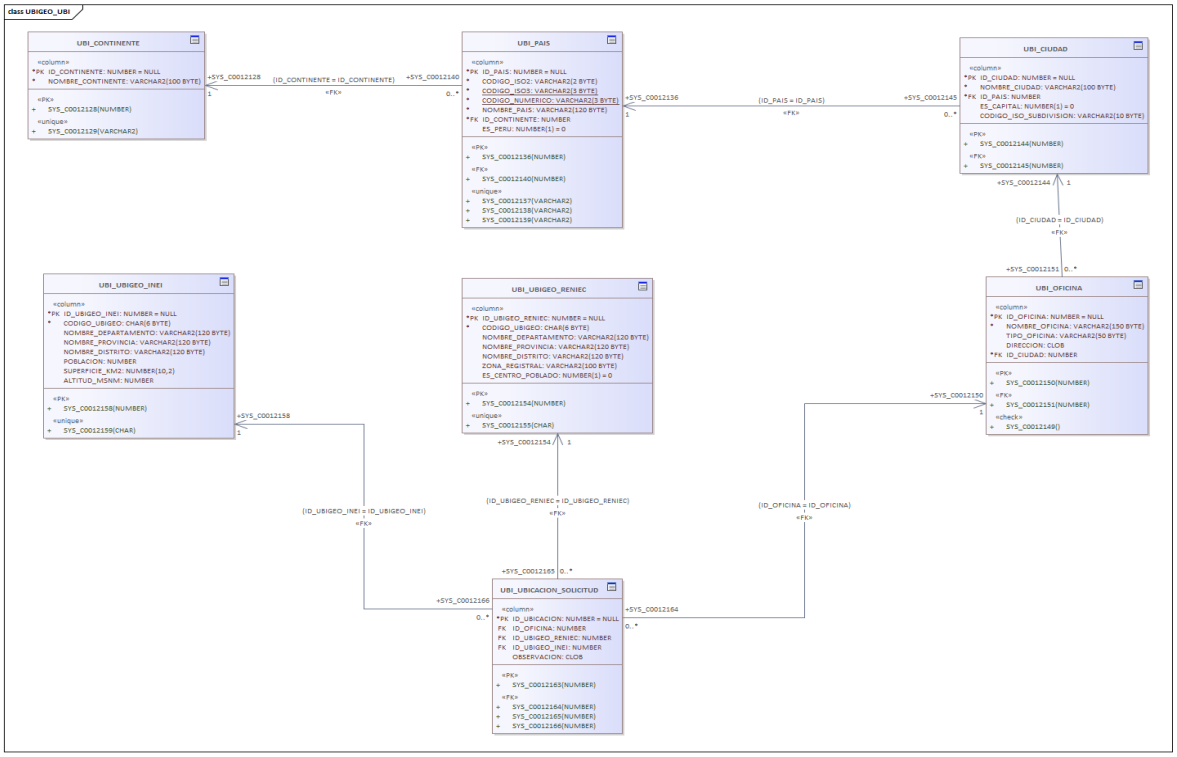


Ilustración 7 Ejemplo de Modelo Entidad-Relación, ubicado en la herramienta Enterprise Architect en la ruta: Model / 3. Arquitectura Datos/ 3.4. Modelo / 3.4.2. ToBe RENIEC (Aplicaciones) / 3.4.2.14. UBIGEO_UBI.

El primer paso en cualquier tarea de diseño de bases de datos es elegir uno de los dos tipos principales de bases de datos con los que se va a trabajar: relacionales o dimensionales. Es vital tener esto claro antes de empezar a diseñar. De lo contrario, puede caer fácilmente en errores de diseño que, a la larga, acarrearán muchos problemas y serán difíciles (o imposibles) de corregir.

9.3 Convención de nombres

Los nombres utilizados en el diseño de la base de datos son esenciales porque, una vez creado un objeto en una base de datos, cambiar su nombre puede ser fatal. Cambiar sólo una letra del nombre puede romper dependencias, relaciones e incluso sistemas enteros.

Por eso es fundamental trabajar con una convención de nomenclatura saludable: un conjunto de reglas que le ahorren la molestia de probar 50 posibilidades diferentes para encontrar el nombre de un objeto que no puede recordar.

No existe una guía universal sobre cómo debe ser una convención de nomenclatura para cumplir su función. Pero lo importante es establecer una convención de nomenclatura antes de nombrar cualquiera de los objetos de una base de datos y mantener esa convención para siempre. Una convención de nomenclatura establece directrices como si se debe utilizar un guion bajo para separar las palabras o unirlas directamente, si se deben utilizar todas las letras mayúsculas o las palabras en mayúsculas (estilo Camel Case), si se deben utilizar palabras en plural o en singular para nombrar los objetos, etc.

Empiece por el diseño conceptual, luego el diseño lógico y, por último, el diseño físico.

Ese es el orden natural de las cosas. Como diseñador, puede tener la tentación de empezar creando objetos directamente en el SGBD para saltarse pasos. Pero esto le impedirá disponer de una herramienta para debatir con las partes interesadas y asegurarse de que el diseño cumple los requisitos de la empresa.

Tras el diseño conceptual, debe pasar al diseño lógico para disponer de la documentación adecuada que ayude a los programadores a comprender la estructura de la base de datos. Es vital mantener actualizado el diseño lógico para que sea independiente del motor de base de datos que se vaya a utilizar. De este modo, si con el tiempo migra la base de datos a un motor diferente, el diseño lógico seguirá siendo útil.

Por último, el diseño físico puede ser creado por los propios programadores o por un DBA, tomando el diseño lógico y añadiendo todos los detalles de implementación necesarios para implantarlo en un DBMS concreto.

9.4 Diccionario de datos

Aunque un ERD sea claro y descriptivo, debe añadir un diccionario de datos para que sea aún más claro. El diccionario de datos mantiene la coherencia y la consistencia en el diseño de la base de datos, sobre todo cuando el número de objetos que la componen crece significativamente.

El objetivo principal del diccionario de datos es mantener un único repositorio de información de referencia sobre las entidades de un modelo de datos y sus atributos. El diccionario de datos debe contener los nombres de todas las entidades, los nombres de todos los atributos, sus formatos y tipos de datos, y una breve descripción de cada uno.

El diccionario de datos proporciona una guía clara y concisa de todos los elementos que componen la base de datos. Así se evita crear múltiples objetos que representan lo mismo, lo que dificulta saber a qué objeto recurrir cuando se necesita consultar o actualizar información.

9.5 Criterios de claves primarias

La decisión de utilizar claves naturales o claves sustitutivas debe ser coherente dentro de un modelo de datos. Si las entidades de un modelo de datos tienen identificadores únicos que pueden gestionarse eficazmente como claves primarias de sus respectivas tablas, no hay necesidad de crear claves sustitutas.

Pero es habitual que las entidades se identifiquen mediante múltiples atributos de distintos tipos -fechas, números y/o largas cadenas de caracteres- que pueden resultar ineficaces para formar claves primarias. En estos casos, es mejor crear claves sustitutas de tipo numérico entero, que proporcionan la máxima eficacia en la gestión de índices. Y la clave sustituta es la única opción si una entidad carece de atributos que la identifiquen de forma exclusiva.

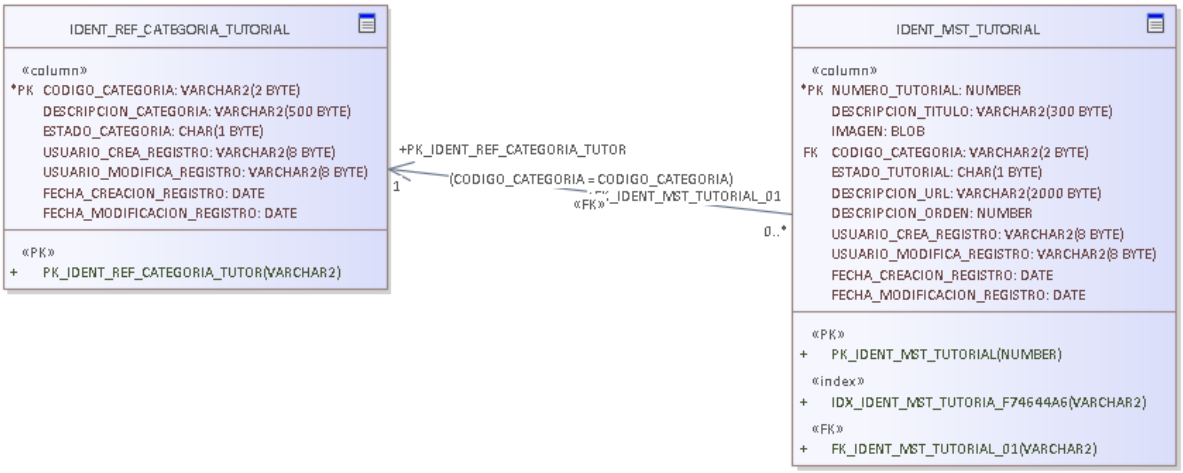


Ilustración 8 Comparación de Estrategias para Definir Claves Primarias; ubicado en la herramienta Enterprise Architect en la ruta: Model /3. Arquitectura Datos / 3.4. Modelo / 3.4.2. ToBe Reniec (Aplicaciones) / 3.4.2.1. IDENTIFICACION / IDO_PORTAL_CIUDADANO_IDENT

9.6 Tipos de datos para cada atributo.

Ciertos datos nos dan la opción de elegir qué tipo de datos utilizar para representarlos. Las fechas, por ejemplo. Se puede elegir almacenarlos en campos de tipo fecha, campos de tipo fecha/hora, campos de tipo varchar o incluso campos de tipo numérico. Otro caso son los datos numéricos que no se utilizan para operaciones matemáticas, sino para identificar a una entidad, como el número de un permiso de conducir o un código postal.

En el caso de las fechas, es conveniente utilizar el tipo de datos del motor, que facilita la manipulación de los datos. Si necesita almacenar sólo la fecha de un acontecimiento sin especificar la hora, el tipo de dato a elegir será simplemente Fecha; si necesita almacenar la fecha y la hora en que se produjo un determinado acontecimiento, el tipo de dato deberá ser DateTime.

Utilizar otros tipos, como varchar o numérico, para almacenar fechas puede ser conveniente, pero sólo en casos muy particulares. Por ejemplo, si no se sabe de antemano en qué formato se expresará una fecha, es conveniente almacenarla como varchar. Si el rendimiento de la búsqueda, la ordenación o la indexación son críticos en el manejo de campos de tipo fecha, una conversión previa a float puede marcar la diferencia.

Los datos numéricos que no intervienen en operaciones matemáticas deben representarse como varchar, aplicando validaciones de formato en el registro para evitar incoherencias o repeticiones. De lo contrario, se expone al riesgo de que algunos datos superen las limitaciones de los campos numéricos y le obliguen a refactorizar un diseño cuando ya está en producción.

9.7 Tablas de consulta

Algunos diseñadores inexpertos pueden creer que el uso excesivo de tablas de consulta para normalizar un diseño puede complicar innecesariamente el ERD de una base de datos porque añade un gran número de tablas «satélite» que a veces no tienen más que un puñado de elementos. Quienes piensen esto deberían comprender que el uso de tablas de consulta tiene muchas más ventajas que inconvenientes. Si la complejidad o el tamaño de un ERD suponen un problema, existen herramientas de diseño de ERD que permiten visualizar los diagramas de distintas formas para que se entiendan a pesar de su complejidad.

Las ventajas de las tablas de consulta son, entre otras, la reducción del tamaño de la base de datos, la mejora del rendimiento de las búsquedas y la imposición de restricciones sobre el conjunto de datos válidos que puede contener un campo. También es una buena práctica que todas las tablas de consulta incluyan un bit o un campo booleano que indique si un registro de la tabla está en uso o es obsoleto. Este campo puede utilizarse como filtro para evitar elementos obsoletos como opciones en la interfaz de usuario de la aplicación.

9.8 Normalizar/desnormalizar según el tipo de Base de Datos

En las bases de datos relacionales utilizadas para aplicaciones tradicionales, la normalización es imprescindible. Es bien sabido que la normalización reduce el espacio de almacenamiento necesario al evitar redundancias. Mejora la calidad de la información y proporciona múltiples herramientas para optimizar el rendimiento en consultas complejas.

Sin embargo, en otros tipos de bases de datos se aplica una técnica conocida como desnormalización. En las bases de datos dimensionales, utilizadas como almacenes de datos, la desnormalización añade cierta información redundante útil en las tablas del esquema.

Aunque parezcan conceptos opuestos, la desnormalización no significa deshacer la normalización. En realidad, es una técnica de optimización que se aplica a un modelo de datos después de normalizarlo para simplificar la escritura de consultas y la elaboración de informes.

9.9 Modelos físicos por partes

Por ejemplo, en un proyecto de desarrollo de software, el diseñador de la base de datos presenta a los interesados un modelo conceptual a gran escala, en el que no se muestran detalles de implementación. A su vez, para trabajar con los desarrolladores, el diseñador debe proporcionar un modelo físico con todos los detalles de cada entidad y atributo. Sin embargo, no es necesario que ambos modelos estén completamente creados al principio del proyecto.

Cuando se aplican metodologías ágiles, cada desarrollador toma al principio de cada ciclo de desarrollo una o varias historias de usuario con las que trabajar durante ese ciclo. El trabajo del diseñador de la base de datos consiste en proporcionar a cada desarrollador un submodelo físico que incluya sólo los objetos que necesita para una unidad de trabajo.

Al final de cada ciclo de desarrollo, los submodelos creados durante ese ciclo se fusionan para que el modelo físico completo tome forma paralelamente al desarrollo de la aplicación.

9.10 Vistas e índices

Las vistas y los índices son dos herramientas fundamentales en el diseño de bases de datos para mejorar el rendimiento de las aplicaciones. El uso de vistas permite manejar abstracciones que simplifican las consultas, ocultando detalles innecesarios de las tablas. A su vez, las vistas facilitan las tareas de optimización de las consultas a los motores de bases de datos, ya que les permiten anticipar cómo se obtendrán los datos y elegir las mejores estrategias para ofrecer los resultados de las consultas con mayor rapidez.

Los índices pueden mejorar el rendimiento de una consulta lenta en función de la experiencia del usuario una vez que la base de datos está en producción. Sin embargo, la creación de índices puede realizarse como parte de las tareas de diseño de la base de datos, anticipándose a las necesidades de la aplicación.

Para la creación de índices, es necesario tener una idea aproximada de la magnitud de cada tabla -en términos de recuento de registros- y, a continuación, crear índices para las tablas más grandes. Para elegir los campos que se incluirán en un índice, debe tener en cuenta principalmente los que representan claves externas y los que se utilizarán como filtros en las búsquedas.

10 Estándar de pruebas

10.1 Alcance

El propósito del capítulo de estándares de pruebas es definir el marco de trabajo, las técnicas, herramientas, criterios y métricas que se aplicarán en los distintos niveles de prueba del SIIRC. Con la finalidad de garantizar que el sistema cumpla con los requerimientos funcionales y no funcionales definidos, manteniendo altos estándares de calidad, confiabilidad y rendimiento conforme a las normas internacionales IEEE 829 e ISO/IEC/IEEE 29119 y bajo el marco internacional de referencia de pruebas ISTQB.

Dentro del alcance se contempla:

- **Unit Testing (responsabilidad Development):**
  Validar el correcto funcionamiento de las unidades más pequeñas del código (métodos, funciones, clases) de forma aislada, utilizando mocks para dependencias externas.

- **API Testing - Nivel Proceso:**
  Validan que cada servicio implementado cumpla con los requerimientos funcionales definidos en la historia de usuario y especificaciones técnicas. Se evalúa el correcto procesamiento de datos, respuestas esperadas, validación de campos obligatorios y opcionales, y cumplimiento de reglas de negocio.

- **Integration Testing - Nivel Proceso:**
  Validan las interacciones entre múltiples APIs y componentes que forman parte de un proceso de negocio completo. Se asegura que los datos fluyan correctamente entre servicios, manteniendo la integridad y consistencia de la información a través de todo el flujo.

- **E2E Testing - Nivel Bloque:**
  Las pruebas end-to-end verifican que los flujos de negocio completos funcionen correctamente desde el punto de vista del usuario final.

- **Performance Testing - Nivel Integral:**
  Evaluar el comportamiento del sistema bajo condiciones específicas de carga, midiendo tiempos de respuesta, throughput, escalabilidad y uso de recursos, para garantizar que cumple con los requisitos no funcionales de rendimiento.

- **Security Testing - Todos los niveles:**
  Identificar vulnerabilidades, debilidades y riesgos de seguridad en el sistema, validando la implementación correcta de controles de autenticación, autorización, cifrado y protección de datos sensibles conforme a estándares OWASP.

- **Funcional Testing:**
  Verificar que el sistema ejecuta correctamente las funciones especificadas en los requisitos, validando entradas, procesamiento y salidas esperadas sin considerar la estructura interna del código, enfocándose en el cumplimiento de las reglas de negocio y comportamientos funcionales definidos.

- **Non-functional Testing (performance, security, usability):**
  Evaluar atributos de calidad del sistema que determinan "cómo" funciona, midiendo características como rendimiento, seguridad, usabilidad, confiabilidad, escalabilidad y compatibilidad, conforme a estándares.

- **Pruebas de regresión:**
  Son realizadas después de algún cambio a una funcionalidad crítica y/o funcionalidades principales del sistema. Se encargan de hacer pruebas sobre la funcionalidad ya entregada, cuando alguna funcionalidad da error y se vuelve a entregar, la funcionalidad afectada debe pasar esta serie de pruebas de validación de que no se ha roto nada.

- **Smoke Testing:**
  Ejecutar un conjunto mínimo de pruebas críticas para verificar que las funcionalidades principales del sistema están operativas después de un despliegue, permitiendo determinar si el sistema es lo suficientemente estable para pruebas más exhaustivas.

- **Acceptance Testing:**
  Validar que el sistema cumple con los criterios de aceptación definidos por el negocio y satisface las necesidades del usuario final, confirmando que está listo para su despliegue en producción desde la perspectiva funcional y de usabilidad.

Componentes:

- Frontend (Web, Mobile)
- Backend APIs
- Base de datos
- Integraciones externas
- Servicios biométricos

10.2 Métricas de pruebas

De acuerdo con las buenas prácticas establecidas por el ISTQB, las métricas de prueba se recopilan con el propósito de evaluar el progreso del proceso de pruebas frente al cronograma y presupuesto planificados, así como para medir la calidad actual del producto

La monitorización de pruebas implica análisis de estas métricas para respaldar el control del proceso, la toma de decisiones y la finalización formal de las pruebas, asegurando que los objetivos definidos en el plan se cumplan de manera medible y trazable.

Tabla 16 Métricas y Beneficios de las Métricas de Pruebas de Software

Métrica	Beneficio
Métricas de progreso de las pruebas	Progreso de la implementación de casos de prueba, progreso de la preparación del entorno de prueba, número de casos de prueba ejecutados/no ejecutados, aprobados/reprobados, tiempo de ejecución de la prueba
Métricas de calidad del producto	Disponibilidad, tiempo de respuesta, tiempo medio hasta el fallo
Métricas de defectos	Número y prioridades de defectos encontrados/corregidos, densidad de defectos, porcentaje de detección de defectos
Métricas de riesgo	Nivel de riesgo residual
Métricas de cobertura	Cobertura de requisitos, cobertura de código
Métricas de costo	Coste de las pruebas, coste organizativo de la calidad

10.3 Criterios de Aceptación de las pruebas

Dentro de los criterios de aceptación de establecen condiciones que deben cumplirse para considerar que una prueba o un conjunto de pruebas, han sido completados de manera exitosa. Estos criterios actúan como una guía para determinar cuándo un producto o funcionalidad se puede considerar como “probado” y listo para avanzar a la siguiente etapa del ciclo de vida de desarrollo.

En términos simples, los criterios de aceptación permiten definir el punto de satisfacción tanto técnico como funcional, asegurando que los resultados obtenidos cumplen con lo esperado y que el producto es estable, usable y cumple los requerimientos definidos por el negocio. Estos criterios pueden variar según el tipo de prueba, o el nivel de prueba, pero en todos los casos garantizan una validación objetiva y medible del cumplimiento de los objetivos de calidad y aceptación del producto.

10.4 Propósito del Proceso de Pruebas de Software

De acuerdo con el estándar **ISTQB CTFL v4.0**, las pruebas de software se diseñan y ejecutan con el propósito de garantizar la calidad del producto, la detección temprana de defectos y la reducción de riesgos técnicos y de negocio. Los objetivos específicos del proceso de pruebas se agrupan de la siguiente manera:

10.4.1 Estándares de Pruebas

Basado en la sección 1.1.1 *“Test Objectives”* del ISTQB, las pruebas persiguen los siguientes fines:

- **Verificación de requisitos:**
  Confirmar que el sistema cumple con los requisitos funcionales y no funcionales especificados, aplicando pruebas en todos los niveles definidos.

- **Detección temprana de defectos:**
  Identificar fallas lo antes posible dentro del ciclo de vida del desarrollo de software (SDLC).
  Principio ISTQB: *"Early testing saves time and money" (Principio 3).*
  Beneficio: Reducir hasta diez veces el costo de corrección en fases tempranas.

- **Evaluación de la calidad del producto:**
  Proporcionar información objetiva sobre el nivel actual de calidad del sistema, facilitando la toma de decisiones informadas.

- **Reducción del riesgo técnico y de negocio:**
  Mitigar riesgos asociados al desarrollo, integración y operación mediante estrategias de prueba planificadas y trazables.

10.4.2 Nivel de Pruebas de Proceso

Las pruebas a nivel de proceso es validar la correcta integración y funcionamiento conjunto de los distintos módulos del sistema, verificando que la lógica de negocio implementada en los procesos transaccionales se ejecute conforme a los requerimientos definidos. Este nivel de prueba busca además asegurar la integridad y consistencia de los datos intercambiados entre servicios, así como detectar posibles errores en la comunicación, transformación o persistencia de la información dentro del flujo operativo end-to-end del sistema.

Para este nivel se adopta la metodología BDD (Behavior Driven Development), la cual permite describir el comportamiento esperado del sistema mediante escenarios escritos en lenguaje natural, entendible tanto para los equipos técnicos como para los usuarios del negocio.

A través del enfoque Given-When-Then, se garantiza una alineación clara entre los requerimientos, las reglas de negocio y los casos de prueba automatizados, fomentando la colaboración entre desarrolladores, testers y analistas funcionales.

Ejemplo BDD – Escenario de Login:

Scenario: Inicio de sesión exitoso

Given que el usuario ingresa su nombre de usuario y contraseña válidos

When presiona el botón "Iniciar sesión"

Then el sistema debe mostrar la página principal con el mensaje "Bienvenido"

Este enfoque metodológico facilita la compresión de las pruebas por todas las partes interesadas y permite automatizar los escenarios de una forma coherente con el comportamiento esperado.

10.4.3 Nivel de Pruebas de Bloque

Las pruebas a nivel de bloque son validar el correcto funcionamiento de los flujos funcionales específicos en los que interactúan uno o varios módulos del sistema, asegurando que cada bloque opere de acuerdo con los requerimientos funcionales y no funcionales establecidos. Además, se busca identificar de manera temprana defectos en la integración parcial de componentes, verificar la coherencia de las interfaces internas, evaluar la estabilidad y desempeño del bloque bajo condiciones controladas, y garantizar que las salidas generadas sean consistentes con las entradas y reglas de negocio definidas.

En este nivel se aplica la metodología ATDD (Acceptance Test-Driven Development), que promueve la colaboración directa entre desarrolladores, testers y el Product Owner para definir los criterios de aceptación antes del desarrollo de la funcionalidad.

Los casos de prueba se construyen a partir de estos criterios, permitiendo que la validación del bloque se base en comportamientos esperados y medibles, y que los defectos se detecten antes de la implementación final.

Ejemplo de implementación:

Historia de usuario

Como Dirección de Registro de Identificación,

quiero que el sistema agrupe automáticamente las solicitudes de DNIe por local y grupo de entrega,

para agilizar la generación de lotes sin necesidad de hacerlo manualmente.

Criterio de aceptación

CA02	El sistema debe identificar automáticamente los DNIe con errores y anular los números de TNP asignados incorrectamente.
CA11	El sistema debe reasignar los DNIe observados a una nueva TNP

Escenario con metodología ATDD

Feature: Personalización DNIe

Scenario: Destrucción y reprocesamiento en DNIe con errores de impresión

Given se hizo el registro de procesamiento de un lote

When se realiza la in spección de impresión y se detectan errores

Then el sistema debe anular los DNIe observados

And debe reprocesar los registros

And reasignar una nueva TNP

And repetir el flujo de impresión

10.4.4 Nivel de Pruebas de Sistema

El nivel de pruebas de sistema tiene como objetivo evaluar el comportamiento integral de la solución tecnológica en su entorno final, verificando que todos los componentes, servicios y módulos del sistema funcionen de manera conjunta y coherente conforme a los requerimientos funcionales y no funcionales definidos.

En este nivel se realizan pruebas bajo condiciones normales, extremas y de carga controlada, con el fin de validar atributos críticos de calidad como el rendimiento, la escalabilidad, la disponibilidad, la resiliencia y la seguridad operativa del sistema. Estas pruebas permiten identificar posibles cuellos de botella, degradaciones de desempeño, fallos de interoperabilidad o comportamientos inesperados cuando el sistema se enfrenta a volúmenes altos de transacciones o múltiples usuarios concurrentes. El resultado esperado es asegurar que la solución esté lista para ser evaluada en entornos de aceptación, reduciendo riesgos antes de su despliegue en producción.

El enfoque aplicado para este nivel se basa en los principios, procesos establecidos por el ISTQB (International Software Testing Qualifications Board), marco de referencia internacional (detallado más adelante) que estandariza las buenas prácticas en pruebas de software. La aplicación de estos principios asegura que todo el proceso de pruebas del sistema sea planificado, trazable, y que las actividades de verificación y validación se ejecuten conforme a criterios definidos de calidad.

Ejemplo ISTQB – Escenario de Login:

- Objetivo de prueba: Validar el comportamiento del sistema completo durante el inicio de sesión.

- Condición: 500 usuarios concurrentes intentan iniciar sesión al mismo tiempo.

Resultado esperado:

- El sistema mantiene un tiempo de respuesta inferior a 3 segundos.
- No se generan errores de disponibilidad ni caídas del servicio.

Cabe resaltar que el escenario, el objetivo y los resultados esperados dependerán del tipo de prueba que se busque realizar, sin embargo, este nivel de prueba demuestra la estabilidad y capacidad del sistema bajo condiciones reales, garantizando su preparación para el entorno de producción.

10.5 Metodologías y marcos de referencia

10.5.1 Metodología BDD

BDD (Desarrollo guiado por comportamiento) permite alinear a los equipos técnicos y funcionales mediante la especificación de casos de uso en lenguaje natural. BDD ofrece:

- Mejor entendimiento entre QA, desarrolladores y stakeholders.
- Facilita la automatización de pruebas.
- Mejora la trazabilidad entre requerimientos y pruebas.
- Reduce ambigüedades y malinterpretaciones funcionales.

10.5.2 Metodología ATDD

La metodología ATDD (Acceptance Test-Driven Development) es importante porque promueve una colaboración estrecha entre desarrolladores, testers y el cliente o Product Owner para definir claramente los criterios de aceptación antes del desarrollo de la funcionalidad. Esto asegura que todos los involucrados compartan una misma comprensión del comportamiento esperado del sistema, lo cual reduce malentendidos, errores y retrabajo.

10.5.3 Marco de referencia de pruebas ISTQB

El estándar de pruebas se fundamenta en los principios, procesos y terminología establecidos por el ISTQB (International Software Testing Qualifications Board), que constituye un marco de referencia internacional para la estandarización de las actividades de prueba de software.

Bajo este enfoque, se adoptan los niveles de prueba definidos por ISTQB (unidad, integración, sistema y aceptación), así como las técnicas de diseño de pruebas basadas en caja negra, caja blanca y caja gris.

10.5.4 Software Engineering Body of Knowledge (SWEBOK)

Esta guía de conocimientos, técnicas y mejores prácticas para la ingeniería de software define el testing como una actividad clave para el aseguramiento de la calidad en todos los productos de software, a fin de garantizar que se cumplan los requisitos especificados y que se logre la detección de defectos antes de la liberación. El swebok plantea algunas consideraciones prácticas para las pruebas, como, por ejemplo:

- **Planificación de pruebas**: definición de objetivos, alcance, recursos, cronograma.
- **Diseño y especificación de pruebas:** identificación de casos de prueba, selección de técnicas (caja negra, blanca o gris), tipos de pruebas funcionales y no funcionales, así como también la definición de datos de prueba.
- **Implementación y ejecución:** desarrollo de pruebas bien documentadas y suficientemente claras para que cualquier persona pueda replicar.
- **Evaluación de resultados:** validación de resultados obtenidos, análisis y depuración antes fallos y una revisión formal de pruebas en casos donde los resultados son importantes.

En general en la guía swebok se desglosa una serie de temas principales a aplicar en el testing, en donde se aborda lo que son fundamentos, los diferentes niveles, las técnicas, medidas (métricas) de pruebas, proceso de pruebas y las diferentes herramientas soportadas y categorizadas, lo cual permite ampliar no solo la visión si no también el alcance y cobertura de pruebas ante cualquier producto de software.

11 Estándar de Implementación de Inteligencia Artificial (IA)

El presente estándar define los lineamientos para el diseño, desarrollo, entrenamiento, evaluación y despliegue de modelos de Inteligencia Artificial (IA) en el entorno operativo del RENIEC.

Su objetivo es garantizar que el uso de algoritmos inteligentes contribuya al fortalecimiento de la verificación de identidad, validación documental, análisis biométrico, clasificación automatizada y detección de anomalías, promoviendo la transparencia algorítmica, la trazabilidad de resultados y la reducción de sesgos en los procesos automatizados.

11.1 Principios generales

El uso de Inteligencia Artificial en los sistemas del RENIEC se rige por principios técnicos y éticos que garantizan el tratamiento seguro, transparente, trazable y verificable de los datos personales, promoviendo la confianza ciudadana en los servicios digitales.

Los modelos de IA aplicados en los procesos de identificación, validación biométrica, análisis documental y automatización institucional deben operar dentro de un marco normativo que asegure precisión algorítmica, explicabilidad y cumplimiento legal, bajo los siguientes principios:

- **Transparencia y explicabilidad:**
  Los modelos deben permitir la comprensión del razonamiento detrás de sus resultados o predicciones. Se deben implementar mecanismos de interpretación (por ejemplo, SHAP, LIME o Grad-CAM) que garanticen la capacidad de auditar el comportamiento algorítmico ante decisiones críticas.

- **Trazabilidad y control de versiones:**
  Cada modelo, conjunto de datos y pipeline de entrenamiento debe estar documentado y versionado. La trazabilidad incluye identificación del modelo (model_id), fecha de entrenamiento, dataset utilizado y métricas de desempeño, asegurando la reproducibilidad y control del ciclo de vida de la IA.

- **Minimización y seguridad de datos:**
  Los datos utilizados en el entrenamiento o inferencia deben limitarse estrictamente a los necesarios para el propósito declarado. Se aplicarán medidas de anonimización, seudonimización y enmascaramiento para evitar la exposición de información personal o sensible.

- **Equidad y ausencia de sesgos:**
  Los modelos deberán ser evaluados regularmente para identificar sesgos demográficos o contextuales. Se deben establecer procedimientos para la corrección de desviaciones que afecten la imparcialidad en la toma de decisiones automatizadas.

- **Supervisión humana:**
  Ningún modelo de IA podrá operar de manera completamente autónoma en decisiones que afecten derechos ciudadanos. Se requiere intervención o validación humana en procesos críticos, asegurando la rendición de cuentas institucional.

- **Seguridad y resiliencia algorítmica:**
  Los entornos de entrenamiento, prueba y producción deben operar bajo políticas de ciberseguridad (control de acceso, cifrado, auditoría y detección de anomalías), garantizando la integridad y disponibilidad de los modelos frente a ataques o manipulaciones

- **Cumplimiento normativo y gobernanza:**
  Todo desarrollo o implementación de IA debe alinearse con la Ley N.º 29733 de Protección de Datos Personales, las Normas ISO/IEC 27001, 27701 y 23894, así como con las políticas internas de la institución en materia de calidad, seguridad y transformación digital.

11.2 Razones técnicas

Desde una perspectiva técnica, la implementación de un estándar unificado de Inteligencia Artificial en el RENIEC responde a la necesidad de garantizar precisión, seguridad, auditabilidad y coherencia operativa en el uso de modelos predictivos y de clasificación en procesos críticos de identificación ciudadana.
En entornos modernos distribuidos basados en microservicios, APIs, almacenamiento seguro y despliegue en nube híbrida, la IA requiere marcos normativos y arquitectónicos uniformes que aseguren interoperabilidad, explicabilidad y cumplimiento normativo.

Principales razones técnicas:

- **Interoperabilidad y consistencia:**
  Los modelos de IA deben integrarse de manera uniforme con los sistemas institucionales existentes (bases de datos, APIs, servicios biométricos), utilizando formatos estandarizados para intercambio de información (JSON, REST).

- **Trazabilidad y auditoría:**
  La gestión centralizada de versiones de modelos, datasets y resultados garantiza el seguimiento histórico de cada entrenamiento, evaluación y decisión algorítmica, alineándose con las exigencias de auditoría de la Ley N.º 29733 y la ISO/IEC 27001.

- **Explicabilidad y confianza:**
  Los sistemas deben incluir mecanismos que permitan interpretar los resultados del modelo, evitando el uso de “cajas negras” y fortaleciendo la confianza institucional en la toma de decisiones automatizadas.

- **Optimización del rendimiento y escalabilidad:** La estandarización de pipelines de IA (entrenamiento, validación, despliegue y monitoreo) mejora la eficiencia computacional y facilita la adopción de arquitecturas MLOps, garantizando un rendimiento estable incluso ante grandes volúmenes de datos.

- **Seguridad y protección de datos:** Los entornos de entrenamiento y despliegue deben implementarse bajo políticas de aislamiento, cifrado en tránsito y reposo, anonimización de datasets y control de accesos basado en roles, en cumplimiento de las políticas internas de RENIEC

- **Cumplimiento normativo y ética algorítmica:** La trazabilidad de decisiones automatizadas permite garantizar el cumplimiento de estándares internacionales como ISO/IEC 23894:2023 (gestión de riesgos de IA), el NIST AI RMF y las directivas del Gobierno Digital del Perú.

En conjunto, estas razones consolidan el estándar de IA como un **pilar técnico esencial** para fortalecer la seguridad, confiabilidad y transparencia de los servicios digitales que el RENIEC ofrece a la ciudadanía, permitiendo la adopción responsable y controlada de tecnologías inteligentes en procesos institucionales.

11.3 Aprendizaje supervisado

El aprendizaje supervisado se utiliza cuando los datos de entrada poseen etiquetas predefinidas que orientan el modelo durante el entrenamiento.

En el contexto RENIEC, este enfoque se aplica a tareas de:

- Validación de documentos: clasificación de imágenes de DNI o certificados.
- Detección de inconsistencias en campos estructurados (nombres, apellidos, fechas).

Los modelos deben registrar:

- Dataset de entrenamiento (origen, tamaño, balance de clases).
- Algoritmo empleado (p. ej., CNN, SVM, Random Forest).
- Métricas de validación (precisión, recall, F1-score).
- Versión y fecha de despliegue.

Adicionalmente, los procesos de entrenamiento deben garantizar la calidad y representatividad de los datos utilizados, evitando sesgos por procedencia geográfica, demográfica o de calidad de imagen. La actualización de los modelos deberá realizarse de manera controlada y documentada, asegurando la trazabilidad de las versiones y la posibilidad de auditoría posterior.

Toda predicción generada por el modelo deberá estar sujeta a revisión humana en los casos en que el resultado tenga impacto directo en la identidad o los derechos de los ciudadanos.

11.4 Aprendizaje no supervisado

El aprendizaje no supervisado se aplica a la detección de patrones ocultos, agrupamientos o anomalías sin etiquetas previas.

Casos de uso en RENIEC incluyen:

- Agrupamiento de ciudadanos por zonas geográficas o características demográficas sin intervención manual.
- Identificación de documentos con estructura irregular o sospechosa mediante clustering.

Estos modelos deben incorporar mecanismos de auditoría que permitan correlacionar los resultados con conjuntos de datos reales y explicar de forma trazable los criterios de agrupamiento generados. Su implementación debe priorizar la transparencia y la interpretabilidad, garantizando que los patrones detectados no conduzcan a decisiones automatizadas sin supervisión humana. Además, los algoritmos empleados (como K-Means, DBSCAN, Autoencoders o PCA) deben seleccionarse con base en su robustez, escalabilidad y capacidad para identificar outliers en grandes volúmenes de información, asegurando resultados consistentes y reproducibles.

Toda salida generada debe documentarse detallando los parámetros de configuración, la versión del modelo y las evidencias del proceso de validación técnica. Asimismo, los resultados derivados de técnicas no supervisadas deben someterse a revisiones periódicas por parte de las áreas técnicas y de cumplimiento, a fin de detectar posibles sesgos, desviaciones o interpretaciones erróneas derivadas de los datos de origen, preservando así la confiabilidad y legitimidad de los análisis automatizados dentro del ecosistema RENIEC.

11.5 Aprendizaje por refuerzo

El aprendizaje por refuerzo (Reinforcement Learning) permite que un agente aprenda a través de la interacción con un entorno, optimizando decisiones mediante recompensas. Este enfoque resulta especialmente útil en sistemas donde las acciones tienen consecuencias acumulativas y se busca la mejora continua del rendimiento operativo.

Aplicaciones potenciales en RENIEC:

- **Optimización de flujos de validación documental**: el sistema aprende a priorizar casos según niveles de riesgo y tiempo de procesamiento.
- **Gestión dinámica de colas de atención:** asignación inteligente de recursos en centros de registro, reduciendo tiempos de espera y mejorando la experiencia del ciudadano.
- Mejora progresiva en chatbots o asistentes virtuales para atención ciudadana, ajustando respuestas con base en la retroalimentación real y el nivel de satisfacción del usuario.

Cada agente debe contar con un marco de control formalmente definido, que incluya la política de acción, la función de recompensa y los parámetros de exploración. Estos elementos deberán configurarse bajo los siguientes lineamientos:

- **Trazabilidad de decisiones:** todas las acciones ejecutadas por el agente deben poder auditarse y explicarse mediante registros o logs verificables.
- **Límites éticos y de cumplimiento:** el diseño del entorno de entrenamiento debe evitar comportamientos discriminatorios, sesgos o decisiones que vulneren la imparcialidad institucional.
- **Evaluación continua:** los modelos deben someterse a ciclos de prueba y validación controlados antes de su despliegue en entornos productivos, garantizando que las recompensas no incentiven comportamientos no deseados.
- **Supervisión humana:** toda decisión automatizada con impacto en la ciudadanía debe estar sujeta a mecanismos de aprobación o revisión humana, especialmente en procesos de identidad y registro civil.

11.6 Visión por computadora

La visión por computadora (Computer Vision) se enfoca en la interpretación automática de imágenes y videos para extraer información estructurada a partir de contenido visual.

En RENIEC, este componente es clave en:

- Lectura óptica y validación de documentos escaneados (OCR avanzado).
- Detección de falsificaciones o manipulaciones en DNI y certificados.
- Seguimiento biométrico en entornos controlados, con total cumplimiento de las normas de privacidad vigentes.

Los modelos de visión deben entrenarse con datasets certificados, anonimizar imágenes personales y cumplir con la resolución NIST FRVT para evaluación de precisión biométrica. Además, deberán implementar mecanismos de control para prevenir sesgos raciales, de edad o género en los procesos de reconocimiento, asegurando la equidad algorítmica en la verificación de identidad.

El procesamiento de imágenes deberá realizarse bajo entornos seguros, evitando la exposición o almacenamiento no autorizado de material sensible. Toda arquitectura de visión por computadora deberá incorporar controles de auditoría, trazabilidad de inferencias y mecanismos de explicabilidad visual que permitan verificar la causa de las decisiones automatizadas.

La validación periódica de desempeño será obligatoria, incluyendo pruebas de degradación en escenarios de iluminación variable, baja resolución o alteración parcial del documento, con el fin de mantener la fiabilidad y robustez del modelo en condiciones operativas reales.

11.7 Vectorización de archivos

Este apartado establece los lineamientos técnicos para la vectorización semántica de documentos y datos textuales en los sistemas del RENIEC.

La vectorización permite transformar textos, imágenes o metadatos en representaciones

numéricas (embeddings), posibilitando el análisis automatizado, la búsqueda contextual y la comparación semántica de información institucional.

Aplicaciones dentro del entorno RENIEC:

- Indexación y búsqueda semántica de documentos normativos, resoluciones, actas y registros históricos.
- Comparación automatizada de contenido documental, identificando similitudes o inconsistencias entre versiones.
- Control de coherencia textual, detectando discrepancias entre campos documentales y registros ciudadanos.
- Soporte a procesos de interoperabilidad, facilitando la correlación de datos entre sistemas internos mediante embeddings vectoriales.

Aspectos técnicos y de cumplimiento:

- Registrar el algoritmo de embedding utilizado (por ejemplo, *Sentence-BERT*, *OpenAI Embeddings* o equivalentes institucionales).
- Documentar la fuente, tamaño y naturaleza del corpus sometido a vectorización.
- Garantizar el aislamiento del entorno de ejecución, evitando la exposición o transferencia de datos personales o sensibles fuera de la infraestructura del RENIEC.
- Mantener una política de control de versiones y validación humana para la interpretación de resultados.

La implementación de procesos de vectorización deberá seguir los principios de transparencia, trazabilidad y reproducibilidad, asegurando que los resultados generados puedan ser auditados y verificados conforme al marco de cumplimiento institucional.

11.8 Aseguramiento de calidad (Q/A) con LLM

Este apartado define las directrices para el uso de Modelos de Lenguaje Extendido (LLM) en procesos de aseguramiento de la calidad documental (Quality Assurance, Q/A) dentro de los sistemas del RENIEC.

El uso de LLM tiene como objetivo fortalecer la validación semántica, la revisión automatizada de contenido textual y la detección de inconsistencias en registros documentales.

Aplicaciones dentro del entorno RENIEC:

- Validación de integridad y consistencia documental, verificando coherencia entre campos, formularios y registros asociados.
- Detección automática de errores semánticos o duplicidades en documentos oficiales.
- Apoyo en auditorías internas de calidad, mediante análisis contextual de expedientes y resoluciones.
- Asistencia en control de versiones, alertando sobre documentos obsoletos o conflictivos en repositorios institucionales.

Requisitos técnicos y operativos:

- Registrar la versión y arquitectura del modelo LLM implementado.
- Asegurar que los modelos se ejecuten en entornos cerrados y controlados, sin conexión externa que comprometa la confidencialidad de la información.
- Incorporar mecanismos de validación humana supervisada para revisar las decisiones o resultados emitidos por los modelos.
- Mantener evidencia de entrenamiento, pruebas y ajustes de desempeño realizados antes del despliegue en producción.

El uso de LLM deberá observar los principios de ética algorítmica, responsabilidad institucional y verificabilidad, garantizando que su funcionamiento contribuya a la eficiencia y calidad del procesamiento documental, sin comprometer la integridad de los datos o la confianza ciudadana.

11.9 Gobernanza y cumplimiento

La implementación de Inteligencia Artificial (IA) en RENIEC debe garantizar una gestión responsable, ética y trazable de los modelos utilizados en procesos institucionales.

Su aplicación debe alinearse con los marcos y estándares internacionales de referencia, tales como:

- ISO/IEC 23894:2023 – Gestión de riesgos de IA, para la identificación, análisis, tratamiento y monitoreo de riesgos asociados al ciclo de vida de los sistemas inteligentes.
- NIST AI Risk Management Framework, que proporciona directrices para asegurar la confiabilidad, equidad, explicabilidad y resiliencia de los modelos.
- Guía de Ética y Gobernanza de IA del Gobierno Digital del Perú, que establece los principios de transparencia, responsabilidad y respeto de los derechos fundamentales en el uso de tecnologías automatizadas.
- Normas internas y procedimientos institucionales que regulen la gestión de datos, la seguridad de la información y la interoperabilidad de sistemas en el entorno del RENIEC.

Cada despliegue de modelo deberá contar con una ficha técnica documentada, que incluya como mínimo:

- Identificación del modelo (nombre, versión, fecha de creación y despliegue).
- Descripción funcional y propósito institucional.
- Origen y características del dataset de entrenamiento y prueba.
- Metodología de validación y resultados de desempeño.
- Identificación del responsable técnico y del responsable ético del modelo.
- Evaluación de impacto y revisión ética previa a la entrada en producción.

Asimismo, deberá establecerse un proceso continuo de supervisión y reevaluación de los modelos en operación, asegurando la detección temprana de sesgos, la actualización de datos, el cumplimiento normativo y la adecuada gestión de versiones.

El área de seguridad y cumplimiento deberá garantizar la trazabilidad completa de las decisiones algorítmicas, habilitando mecanismos de auditoría que permitan justificar los resultados obtenidos por los sistemas inteligentes ante instancias internas o regulatorias.

12 Revisión y Actualización

El documento forma parte del marco normativo técnico del Sistema Integrado de Identificación y Registro Civil (SIIRC) de RENIEC y, por lo tanto, requiere una revisión y actualización periódica para garantizar su alineación continua con:

- La legislación nacional vigente, en especial lo dispuesto por la Ley N.º 29733 – Ley de Protección de Datos Personales y su reglamento.
- Las directivas técnicas emitidas por la Presidencia del Consejo de ministros (PCM).
- Los estándares internacionales en seguridad de la información, como ISO/IEC 27001, ISO/IEC 27034 (Seguridad en el Ciclo de Vida del Software) y OWASP.
- La evolución tecnológica del ecosistema SIIRC, incluyendo nuevas plataformas, lenguajes, herramientas de desarrollo y mecanismos de protección.
- Cambios significativos en la arquitectura de software o infraestructura tecnológica de la organización.

12.1 Frecuencia de Revisión

Este estándar será **revisado anualmente**, como parte de la planificación del ciclo de mejora continua del área de Seguridad de la Información y de la Oficina de Tecnología de la Información y Comunicaciones (OTIC).
Podrá ser **actualizado de forma extraordinaria** cuando:

- Se promulgue una nueva ley o regulación relevante en materia de protección de datos personales o ciberseguridad.
- Se detecten brechas de seguridad o deficiencias en las auditorías internas o externas que lo ameriten.
- Se introduzcan tecnologías disruptivas o arquitecturas críticas (por ejemplo, migraciones a microservicios, adopción de blockchain o inteligencia artificial).
- Ocurran incidentes de seguridad que requieran mejoras inmediatas en los controles de desarrollo.

12.2 Responsables

La responsabilidad de la revisión recae sobre el **Comité de Seguridad de la Información** en coordinación con:

- La **Oficina de Seguridad de la Información (OSI)**.
- La **Oficina de Tecnología de la Información y Comunicaciones (OTIC)**.
- La **Unidad de Cumplimiento Legal y Normativo**.
- Las actualizaciones deberán ser aprobadas por la **Alta Dirección de RENIEC** y comunicadas oficialmente a todos los equipos de desarrollo y proveedores tecnológicos.

12.3 Control de Cambios

Toda modificación del documento deberá registrarse en la sección de control de versiones, incluyendo:

- Número de versión.
- Fecha de revisión.
- Breve descripción del cambio.

13 Referencias Bibliográficas y Normativas

13.1 Libros y Publicaciones Académicas

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-Based Software Architectures* (Doctoral dissertation, University of California, Irvine).

Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

Nygard, M. T. (2018). *Release It!: Design and Deploy Production-Ready Software* (2nd ed.). Pragmatic Bookshelf.

Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.

Sridharan, C. (2018). *Distributed Systems Observability*. O'Reilly Media.

Packt Publishing. (2020). *Observability with Prometheus and Grafana*. Packt Publishing.

13.2 Sitios Web y Documentos Técnicos

Internet Engineering Task Force (IETF). (2014). *RFC 7231: Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. https://tools.ietf.org/html/rfc7231

Internet Engineering Task Force (IETF). (2009). *RFC 5424: The Syslog Protocol* (R. Gerhards, Ed.). https://tools.ietf.org/html/rfc5424

Microsoft. (2023). *Microsoft REST API Guidelines*. https://github.com/microsoft/api-guidelines

Zalando. (2023). *RESTful API Guidelines*. https://opensource.zalando.com/restful-api-guidelines/

Fowler, M. (2010). *Richardson Maturity Model*. https://martinfowler.com/articles/richardsonMaturityModel.html

Google Cloud. (2023). *API Design Guide*. https://cloud.google.com/apis/design/

NGINX. (2015). *Building Microservices Using an API Gateway*. https://www.nginx.com/blog/building-microservices-using-an-api-gateway/

gRPC Authors. (2023). *What is gRPC?*. https://grpc.io/docs/what-is-grpc/

ISO. (2013). *ISO/IEC 27001:2013 – Information Technology – Security Techniques – Information Security Management Systems – Requirements*. https://www.iso.org/standard/54534.html

ISO. (2004). *ISO 8601:2004 – Data Elements and Interchange Formats – Representation of Dates and Times*. https://www.iso.org/iso-8601-date-and-time-format.html

Heroku. (2014). *Logfmt Logging Format*. https://brandur.org/logfmt

Microsoft. (2023). *Best Practices for Structured Logging with JSON*. https://learn.microsoft.com/en-us/azure/azure-monitor/logs/log-structure

OpenTelemetry. (2023). *Logging Specification*. https://opentelemetry.io/docs/specs/otel/logs/

CNCF. (2023). *OpenTelemetry Project*. https://www.cncf.io/projects/opentelemetry/

Google Cloud. (2023). *Logging and Monitoring Best Practices*. https://cloud.google.com/architecture/logging-best-practices

OpenTelemetry Authors. (2023). *Why OpenTelemetry?*. https://opentelemetry.io/docs/concepts/what-is-opentelemetry/

Sigelman, B., et al. (2017). *Tracing Systems at Scale: Lessons from Google and Beyond*. *Communications of the ACM, 60*(6). https://queue.acm.org/detail.cfm?id=3022186

(1)(2) *Política de Seguridad Aplicada a los Modelos ToBe*.

13.3 Normativas y Legislación

Congreso de la República del Perú. (2011). *Ley N° 29733 – Ley de Protección de Datos Personales*. https://www.gob.pe/institucion/minjus/normas-legales/250602-29733

Presidencia del Consejo de Ministros – Secretaría de Gobierno y Transformación Digital. (2021). *Decreto Supremo N° 029-2021-PCM: Política Nacional de Transformación Digital*. https://sgtd.presidencia.gob.pe/

13.4 Conferencias y Opiniones Expertas

Hightower, K. (2021). *Observability Done Right* [Conferencia]. KubeCon 2021.

KubeCon. (2024). *Observability Insights from KubeCon 2024 – Summary*.

13.5 Sitios Web Institucionales y Normativos

Ministerio de Justicia y Derechos Humanos del Perú. (s.f.). *Protección de Datos Personales*. https://www.minjus.gob.pe/proteccion-de-datos-personales/

Presidencia del Consejo de Ministros. (s.f.). *Sede Digital*. https://www.gob.pe/pcm/digital

13.6 Normas ISO

International Organization for Standardization. (2013). *ISO/IEC 27001:2013 – Information Technology – Security Techniques – Information Security Management Systems – Requirements*. https://www.iso.org/standard/54534.html

International Organization for Standardization. (2013). *ISO/IEC 27002:2013 – Code of Practice for Information Security Controls*. https://www.iso.org/standard/27001

International Organization for Standardization. (2009). *ISO/IEC 27005:2009 – Information Security Risk Management*. https://www.iso.org/standard/44378.html

International Organization for Standardization. (2013). *ISO/IEC 27034-1 – Application Security – Part 1: Overview and Concepts*. https://www.iso.org/standard/44378.html

13.7 OWASP (Open Web Application Security Project)

OWASP Foundation. (2021). *OWASP Top Ten: The Ten Most Critical Web Application Security Risks*. https://owasp.org/www-project-top-ten/

OWASP Foundation. (2023). *Application Security Verification Standard (ASVS)*. https://owasp.org/www-project-application-security-verification-standard/

OWASP Foundation. (2023). *Software Assurance Maturity Model (SAMM)*. https://owasp.org/www-project-samm/

13.8 NIST (National Institute of Standards and Technology)

National Institute of Standards and Technology. (2020). *SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations*. https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final

National Institute of Standards and Technology. (2022). *SP 800-218: Secure Software Development Framework (SSDF) Version 1.1*. https://csrc.nist.gov/publications/detail/sp/800-218/final

13.9 Políticas Internas

Política de calidad de datos

Política de seguridad de datos